

CONSERVATOIRE NATIONAL DES ARTS ET MÉTIERS

CENTRE RÉGIONAL ASSOCIÉ DE NOUVELLE-AQUITAINE

État de l'art

Présenté en vue d'obtenir le

DIPLÔME D'INGÉNIEUR CNAM

SPÉCIALITÉ : INFORMATIQUE ET MULTIMÉDIA

Par

FLOC'H LE CAROUR Jame

Les techniques de rendu des aspects de la
lumière dans le jeu vidéo 3D et leurs
évolutions jusqu'à nos jours

Soutenu le 26/04/2024

JURY

PRÉSIDENT : BARTHOLOMEY Paul

Remerciements

La réalisation de cet état de l'art n'aurait pas été possible sans le soutien et l'aide de nombreuses personnes que je tiens à remercier.

Je suis reconnaissant envers l'ensemble des professeurs et des membres du *Cnam* qui m'ont apporté leurs connaissances et leur expertise tout au long de mon cursus. Je suis également reconnaissant envers *Virtuos Games* ainsi que mes collègues qui m'ont encadré et soutenu tout au long de mes deux premières années d'alternance et qui m'ont permis de réaliser cet état de l'art dans un cadre professionnel stimulant.

Je remercie aussi les personnes qui ont pris le temps de relire et de corriger mon état de l'art et mon rapport d'activité. Leurs commentaires constructifs et leur attention portée aux détails ont contribué à améliorer considérablement la qualité de mon travail. Ma gratitude s'étend également à mes camarades et collègues qui m'ont orienté vers des pistes intéressantes et m'ont permis d'étendre mon horizon de recherches.

Je remercie les chercheurs et les entreprises qui publient leurs travaux, partageant ainsi leurs connaissances et contribuant à l'avancement de la recherche dans l'imagerie numérique. Ma gratitude s'étend enfin aux auteurs d'articles et de vidéos de recherche et de vulgarisation ainsi qu'aux différentes communautés de passionnés qui archivent et présentent les différentes méthodes de rendus de la lumière dans le jeu vidéo. Leurs travaux m'ont grandement aidé à retracer une partie de l'histoire de l'imagerie numérique en temps réel.

Liste des abréviations

2D : deux dimensions

3D : trois dimensions

API : Application Programming Interface

BRDF : Bidirectional Reflectance Distribution Function

BSDF : Bidirectional Scattering Distribution Function

BSSDF : Bidirectional Scattering-Surface Distribution Function

BSSRDF : Bidirectional Scattering-Surface Reflectance Distribution Function

BSSTDF : Bidirectional Scattering-Surface Transmittance Distribution Function

BTDF : Bidirectional Transmittance Distribution Function

DLSS : Deep Learning Super Sampling

DRTV : Deferred Radiance Transfer Volume

FSR : FidelityFX Super Resolution

EA : Electronic Arts

GI : Global Illumination

GIBS : Global Illumination Based On Surfels

HBAO : Horizon Based Ambient Occlusion

HDAO : High Definition Ambient Occlusion

HDDAGI : Hierarchical Digital Differential Analyzer Global Illumination

HDRI : High Dynamic Range Image

IA : Intelligence Artificielle

IES : Illuminating Engineering Society

IOR : Indice Of Refraction

LED : Light Emmiting Diode

LOD : Level Of Detail

NeRF : Neural Radiance Field

PBR : Physically Based Rendering

PCF : Percentage Closer Filtering

PRT : Precomputed Radiance Transfert

ReSTIR : Reservoir-Based Spatio-Temporal Importance Resampling

ReSTIR GI : Reservoir-Based Spatio-Temporal Importance Resampling Global Illumination

SDF : Signed Distance Field

SDFGI : Signed Dittance Field Global Illumination

SMRT : Shadow Map Ray Tracing

SSAO : Screen Space Ambient Occlusion

SSBump Map : Self Shadwing Bump Map

SSDO : Screen Space Directional Occlusion

SSIL : Screen Space Indirect Lighting

SSR : Screen Space Reflection

SSS : Screen Space Shadow / SubSurface Scattering

URP : Universal Render Pipeline

VR : Virtual Reality

VXAO : Voxel Ambient Occlusion

VXGI : Voxel Global Illumination

Glossaire des termes techniques

Albédo : couleur intrinsèque d'une surface

Algorithme : ensemble d'instructions pour résoudre un problème

Application Programming Interface : interface logicielle permettant aux applications de communiquer entre elles

Micro-Facettes : algorithme permettant la réflexion de la lumière sur des surfaces plus ou moins rugueuses

Algorithme Monte-Carlo : algorithme utilisant le hasard pour générer des solutions à des problèmes complexes

Alpha Blending : mélange de deux images en fonction de leur canal alpha, il existe plusieurs méthodes différentes de blending dans les moteurs de jeu

Ambient Occlusion : phénomène d'assombrissement des zones creuses

Anisotropie : propriété d'une surface à réfléchir la lumière différemment selon sa direction

Anti-Aliasing : algorithme réduisant les crénelages sur les bords des images et adoucissant les transitions entre les surfaces

AO Map : texture d'occlusion ambiante précalculée

Azimut : angle horizontal d'un point par rapport au nord

Baking : précalcul d'éclairage d'une scène pour améliorer les performances en jeu

Bump Map : carte de relief simulant des détails sur une surface plane

Cache : structure de stockage des données pour un accès ultérieur plus rapide

Clear-Coat : revêtement transparent sur une surface

Concave : surface incurvée vers l'intérieur

Cone Tracing : technique de lancer de rayons utilisant des cônes pour échantillonner la lumière

Contact Shadows : ombres créées par le contact entre deux objets

Cube Map : texture représentant l'environnement autour d'un point

Debruitage / Denoising : réduit le bruit dans les images

Deep Learning Super Sampling : algorithme de machine learning développé par *Nvidia* pour améliorer la résolution des images

Deferred Rendering : technique permettant de différer le rendu de certains éléments pour améliorer les performances

Deferred Radiance Transfer Volume : technique de rendu global développée par *Ubisoft*

Shadow Map : texture d'ombrage précalculée

DirectX : API de programmation pour les jeux vidéo et les applications graphiques de Microsoft, créé pour les plateformes *Windows* et *XBox*

Echantillonnage : processus de sélection de points dans une image ou une scène

Équation De Rendu : équation déterminant la couleur d'un point en fonction de la lumière et des propriétés de la surface

Espace De L'Écran : coordonnées 2D correspondant aux pixels sur l'écran

Fidelity FX : suite de technologies développée par *AMD* pour améliorer la qualité des images

Fidelity FX Super Resolution : technologie d'amélioration de la résolution d'images créée par *AMD*

Fonction ayant une complexité quadratique : fonction dont le temps d'exécution est proportionnel au carré de la taille des données qu'elle traite

Fonction de distribution bidirectionnelle de diffusion de la lumière : fonction décrivant comment la lumière est diffusée par une surface

Fonction de distribution bidirectionnelle de transmission : fonction décrivant comment la lumière est réfractée par une surface

Fonction de distribution de réflectance bidirectionnelle : fonction décrivant comment la lumière est réfléchie par une surface.

Fonction de distribution bidirectionnelle de diffusion de la lumière de surface translucide : fonction décrivant comment la lumière est diffusée par une surface translucide

Framework : ensemble d'outils et de composants logiciels facilitant le développement d'applications

Full HD : résolution d'image de 1920 x 1080 pixels

G-Buffer : texture de données utilisée pour le rendu différé (Deferred Rendering)

Gaussian Spatting : algorithme de rendu de nuage de points permettant d'afficher une scène 3D à partir d'une suite d'images

Global Illumination : phénomène décrivant les rebonds de la lumière dans une scène

Global Illumination Based On Surfels : technique d'illumination globale basée sur des surfels, développée par Electronic Arts

Goniométre : instrument permettant de mesurer la réflexion de la lumière

Grain : imperfection dans une image sous forme d'un bruit de pixels

Height Map : texture de hauteur représentant le relief d'une surface

Hierarchical Digital Differential Analyzer Global Illumination : technique d'illumination globale créée par *Dreamworks* actuellement en développement dans le moteur de jeu *Godot*

High Definition Ambient Occlusion : algorithme de calcul d'occlusion ambiante de haute qualité

High Dynamic Range Image : image à large plage dynamique, souvent panoramique

Horizon Based Ambient Occlusion : algorithme de calcul d'occlusion ambiante basée sur l'horizon de la scène

Illumination Directe : lumière provenant directement d'une source lumineuse

Illumination Indirecte : lumière provenant des rebonds de la lumière sur les surfaces d'une scène

Implémentation : réalisation concrète d'un algorithme ou d'une technique.

Importance Sampling : technique d'échantillonnage favorisant les zones importantes de la scène

Index Of Réfraction (Indice de Réfraction) : une mesure de la vitesse à laquelle la lumière se propage dans un matériau par rapport à sa vitesse dans le vide, cette valeur est proportionnelle à la densité du matériau

Instant Radiosity : technique d'illumination globale calculant l'éclairage indirect instantanément en plaçant un grand nombre de lampes dans la scène

Intelligence Artificielle : domaine informatique visant à créer des machines capables de simuler des comportements intelligents

Lancer de rayons (Ray Tracing) : technique de rendu simulant le chemin de la lumière pour générer une image réaliste

Level Design : conception des environnements dans un jeu vidéo

Level Of Detail : technique permettant d'optimiser les performances en affichant des modèles 3D plus simples à distance

Light Map : texture contenant les informations d'éclairage précalculées pour une scène

Light Mapper : logiciel générant les Light Maps

Light Probe : sphère capturant l'éclairage ambiant d'un point dans la scène

Light Probes Group : objet du moteur *Unity* permettant de gérer des light probes

Lumen : algorithme de calcul d'illumination globale en temps réel présent dans le moteur Unreal Engine 5

Machine Learning : sous-domaine de la recherche en intelligence artificielle qui permettant aux machines d'apprendre à partir de données

Many Light Rendering : problème statistique de difficulté des calculs de lumière en présence de nombreuses sources lumineuses

Mipmap : ensemble de textures de tailles décroissantes permettant d'optimiser les performances graphiques

Modèle 3D : représentation mathématique d'un objet en trois dimensions

Modèle Empirique : modèle basé sur des observations plutôt que sur des lois physiques

Moteur de jeu : logiciel permettant de développer des jeux vidéo

Neural Radiance Field (NeRF) : technique de représentation de scènes 3D à partir d'images utilisant des réseaux de neurones virtuels

Ninite : système de virtualisation de géométrie présent dans Unreal Engine permettant d'afficher un très grand nombre de polygones

Normal Map : texture contenant les informations de normales (orientation) d'une surface pour simuler des détails

Normale (d'une surface) : vecteur perpendiculaire à la surface en un point donné

Ombrage de Blinn-Phong : modèle d'ombrage simulant la réflexion diffuse et spéculaire

Ombrage de Gouraud : technique d'ombrage appliquant la couleur de sommet à toute la face

Ombrage de Lambert : modèle d'ombrage ne tenant compte que de la diffusion de la lumière

Ombrage de Phong : modèle d'ombrage plus complexe que celui de Gouraud simulant la réflexion spéculaire

Ombrage de Trowbridge-Reitz GGX : modèle d'ombrage avancé pour les surfaces micro-facettes

Ombres portées : ombres projetées par les objets sur une scène

Open World : jeu vidéo en monde ouvert permettant une exploration libre

OpenGL : API de programmation pour le rendu de graphiques 2D et 3D

Optix Denoiser : algorithme de débruitage développé par Nvidia

Package (Unity) : élément téléchargeable regroupant des ressources dans Unity

Parallax Corrected Cube Map : technique corrigeant l'effet de parallaxe sur les Cube Maps

Parallax Occlusion Mapping : technique simulant la profondeur d'une surface sans déformer réellement la géométrie

Patch : élément de base d'une carte du jeu *Quake* utilisé pour l'optimisation des collisions

Path Tracing : technique de rendu basée sur le traçage du chemin de la lumière pour un réalisme élevé

Photoréalisme : image qui est si réaliste qu'elle n'est pas distinguable d'une photographie

Physically Based Rendering : rendu basé sur les principes physiques de la lumière et des matériaux

Pixel (Picture Element) : élément d'une image

Planar Reflection Probe : light Probe capturant les réflexions de l'environnement sur un plan

Post-Processing : effets appliqués à l'image finale pour en améliorer la qualité

Precomputed Radiance Transfert : structure de stockage des informations d'illumination globale utilisée par *Ubisoft* dans son jeu *Far Cry 3*

Percentage Closer Filtering : technique de filtrage permettant d'adoucir les ombres

Profil **Illuminating Engineering Society** : format de fichier contenant des informations sur la distribution de la lumière d'une source lumineuse, ces informations étant issues des informations d'ampoules réelles

Radiosité : algorithme d'illumination globale simulant la lumière indirecte

Rastérisation : processus de conversion d'une scène polygonale en image

Ray Casting : technique de lancer de rayon tirant des rayons depuis la caméra. *Dans le contexte de ce document, le ray casting s'effectue uniquement dans un espace en 2D*

Ray Marching : technique de lancer de rayons parcourant une scène en volume

Ray Tracing : algorithme de lancer de rayon dans une scène à partir de la caméra permettant de calculer les rebonds de lumière dans une scène

Réalité virtuelle : technologie permettant une immersion simulée dans un environnement artificiel

Reflection Capture : objet spécifique au moteur de jeu *Unreal Engine* capturant de l'environnement utilisé pour simuler les réflexions sur les surfaces

Reflection Probe : sphère capturant l'environnement visible d'un point pour simuler les réflexions locales

Réflexion diffuse : réflexion de la lumière dans toutes les directions à partir de la surface mate d'un objet

Réflexion environnementale : réflexion de l'environnement sur la surface d'un objet

Réflexion spéculaire : réflexion de la lumière dans une direction opposée à la source de lumière à partir de la surface d'un objet

Reservoir-Based Spatio-Temporal Importance Resampling : technique d'éclairage direct développée par *Nvidia* permettant d'utiliser un très grand nombre de sources lumineuses

Sample : échantillon utilisé dans les techniques de rendu pour représenter une zone de la scène

Screen Space Ambient Occlusion : algorithme de calcul d'occlusion ambiante réalisé dans l'espace écran

Screen Space Directional Occlusion : algorithme de calcul d'occlusion ambiante réalisé dans l'espace écran amélioré

Screen Space Indirect Lighting : technique d'illumination indirecte réalisée dans l'espace écran disponible dans *Godot Engine*

Screen Space Radiance Cache : structure de donnée de stockage de l'information de lumière utilisée par l'algorithme Lumen d'*Unreal Engine*

Screen Space Reflection : algorithme de calcul de réflexion réalisé dans l'espace écran

Screen Space Shadow : technique d'ombrage réalisée dans l'espace écran

Self Shadowing Bump Map : texture de normales tenant compte de l'occlusion ambiante

Shader : programme informatique exécuté sur une puce graphique permettant d'effectuer des calculs de lumière de manière parallélisés

Shadow Cascade : technique de découpage des ombres en cascades permettant d'améliorer leur qualité

Shadow Map : texture contenant les informations d'ombrage

Shadow Map Ray Tracing : technique d'ombrage disponible dans Unreal Engine

Shadow Volume : algorithme de calcul des ombres basé sur la projection des volumes des modèles dans une scène

Sheen : effet de reflet subtil sur certaines surfaces comme le tissu

Signed Distance Field : représentation d'une surface par la distance au plus proche point de la surface

Signed Distance Field Global Illumination : technique d'illumination globale basée sur les Signed Distance Fields disponibles dans *Godot*

Spherical Harmonics Function : fonction utilisée pour représenter la lumière ambiante

Structure réservoir : mémoire tampon utilisée pour stocker des données intermédiaires de rendu

Subsurface Profile : profil définissant la diffusion de la lumière sous la surface d'un matériau

Subsurface Scattering : diffusion de la lumière à l'intérieur d'un matériau translucide

Surfels : représentation d'une surface par des éléments surfaciques circulaire

Texture : image appliquée sur la surface d'un modèle 3D pour en définir l'apparence

Uncanny Valley : phénomène psychologique décrivant une sensation de malaise face à des humanoïdes presque réalistes

Universal Render Pipeline : solution de rendu légère et optimisée proposée par Unity

Upscaling : technique d'amélioration de la résolution d'une image

UV Map : correspondance entre une texture et la surface d'un modèle 3D

Virtual Shadow Map : technique de shadow map améliorée présente dans *Unreal Engine* permettant de décupler sa résolution

Volumetric Light Map : texture 3D contenant les informations d'éclairage pour une scène de manière volumétrique

Voxel : élément d'un volume en 3D, similaire à un pixel pour une image

Voxel Ambient Occlusion : technique d'occlusion ambiante utilisant des voxels

Voxel Global Illumination : technique d'illumination globale développée par *Nvidia* utilisant des voxels

Workflows : enchaînement de tâches à réaliser pour effectuer un travail

Workflow Métallique : pipeline de rendu PBR basé sur un paradigme de surface métallique

Workflow Spéculaire : pipeline de rendu PBR basé sur un paradigme de surface spéculaires

World Space Radiance Cache : structure de donnée de stockage de d'information de lumière utilisée par l'algorithme Lumen d'*Unreal Engine*

Zénith : point directement au-dessus de l'observateur dans des coordonnées sphériques

Sommaire

Remerciements	2
Liste des abréviations	3
Glossaire des termes techniques	5
Sommaire	13
Introduction.....	15
Aspects physiques de la lumière	15
Relation avec le ressenti du joueur	16
La vision	16
Les couleurs	16
Les émotions.....	16
La lumière comme élément de gameplay dans le jeu vidéo	17
Les différents aspects du rendu de la lumière dans le jeu vidéo en 3D	18
La surface des objets et les matériaux	18
Description	18
Historique	19
Modèles polygonaux	19
Le Physically Based Rendering.....	28
Les ombres portées	43
Description	43
Différentes sources de lumière	45
Les ombres statiques.....	48
Les ombres dynamiques.....	50
La lumière environnementale	60
Description	60
La réflexion environnementale	60

L'illumination Globale.....	70
L'occlusion ambiante.....	91
Spéculations sur l'évolution des techniques de rendu en temps réel dans les années à venir dans le jeu vidéo	98
Conclusion	100
Liste des figures.....	102
Annexe.....	108
Liste de sources	108

Introduction

Aspects physiques de la lumière

La lumière est un phénomène physique électro-magnétique qui est à l'origine de la vision, l'un des principaux sens nous permettant de percevoir le monde. Chez les personnes voyantes, la vision est le sens le plus utilisé pour percevoir l'environnement. Les différents aspects physiques de la lumière jouent donc un rôle majeur dans la compréhension du monde qui nous entoure. Le cerveau humain s'est adapté au fil de millions d'années, afin de donner du sens aux informations électro-chimiques transmises par nos yeux au contact de la lumière, il est donc normal que nous puissions détecter des changements très précis dans la lumière.

Au vu de l'importance de la lumière dans nos vies, il n'est pas surprenant que la représentation de la lumière dans l'art date de la préhistoire et occupe une place si importante. La lumière a une importance culturelle et symbolique dans la majorité des civilisations connues, on étudie son comportement physique depuis l'Antiquité. L'optique est donc une science ancienne qui prend racine dans la Grèce antique. Elle a beaucoup progressé au Moyen Âge, particulièrement en Asie et en Orient. Notre compréhension de la lumière a ensuite continué à s'améliorer au fil des siècles à partir de la Renaissance jusqu'à nos jours.

Les formules de calculs de lumières sont donc maîtrisées depuis longtemps, la difficulté majeure dans la simulation de la lumière est encore aujourd'hui son temps de calcul. La complexité du calcul de lumière vient du fait qu'elle est composée de milliards de photons qui rebondissent dans tout l'environnement. La quantité d'informations à traiter est donc très importante et l'objectif des algorithmes de rendu de lumière est de considérablement réduire ce temps de calcul.

C'est l'apparition de l'informatique qui aura permis aux chercheurs de simuler le comportement de la lumière dans un environnement virtuel pour la première fois. Grâce aux ordinateurs programmables, on a pu simuler les équations de la lumière décrites par la science de l'optique et ainsi générer des images virtuelles. La génération de ces images a d'abord été réservée aux usages scientifiques mais l'avancée de l'informatique a permis de démocratiser les algorithmes de calculs de lumière pour des usages artistiques.

Dans le domaine du jeu vidéo et du rendu en temps réel, il est d'autant plus important de pouvoir réaliser ces calculs dans un temps très court pour permettre l'interaction en direct. Pour obtenir entre 30 et 60 images par seconde, il faut pouvoir rendre la scène 3D dans une durée

d'entre 16 et 30 millisecondes. C'est pour cela que depuis les prémices de la 3D temps réel jusqu'à ce jour, les développeurs de jeux se sont reposés sur des techniques d'approximations du rendu de la lumière créée pour donner un aspect assez convainquant à leurs graphismes. Ces techniques ont beaucoup évolué grâce aux avancées scientifiques et à l'amélioration exponentielle de la puissance du matériel de calcul de rendu 3D. Ce sont ces techniques que nous aborderons dans ce document.

Relation avec le ressenti du joueur

La vision

Comme décrit précédemment, la vision est le principal sens utilisé par les personnes voyantes pour interagir et se déplacer dans leur environnement. Les différentes propriétés physiques de la lumière qui sont captées par les yeux et permettent au cerveau humain, en se reposant également sur d'autres sens, de comprendre son environnement.

Les interfaces Homme-machines utilisées pour interagir avec le jeu vidéo ne permettent pas de simuler tous les sens. Les joueurs et joueuses doivent se reposer sur leur vision et sur leur ouïe. De plus, la majorité des écrans ne permet pas l'affichage stéréoscopique des environnements, ce qui oblige le cerveau à distinguer les volumes en fonction de leur intensité lumineuse.

La représentation de la lumière est donc très importante pour la navigation dans les espaces 3D proposés par les jeux vidéo.

Les couleurs

L'œil humain interprète les différentes fréquences électromagnétiques du spectre visible de la lumière comme des couleurs. La théorie des couleurs étudie l'impact de ces dernières sur les émotions et les perceptions humaines. Différentes couleurs peuvent avoir des sens différents, par exemple le bleu pour le calme ou le rouge pour le danger. Cette théorie est utilisée dans les jeux vidéo pour influencer le ressenti du joueur, communiquer des informations implicites ou explicites en jeu et créer des atmosphères spécifiques.

Les émotions

La lumière dans les images permet de communiquer des émotions de manière très efficace aux observateurs. Les couleurs peuvent avoir des sens plus ou moins universels, l'intensité lumineuse également. On peut jouer sur l'intensité des ombres et lumière et leur distribution pour donner une sensation de confort ou un sentiment de danger imminent.

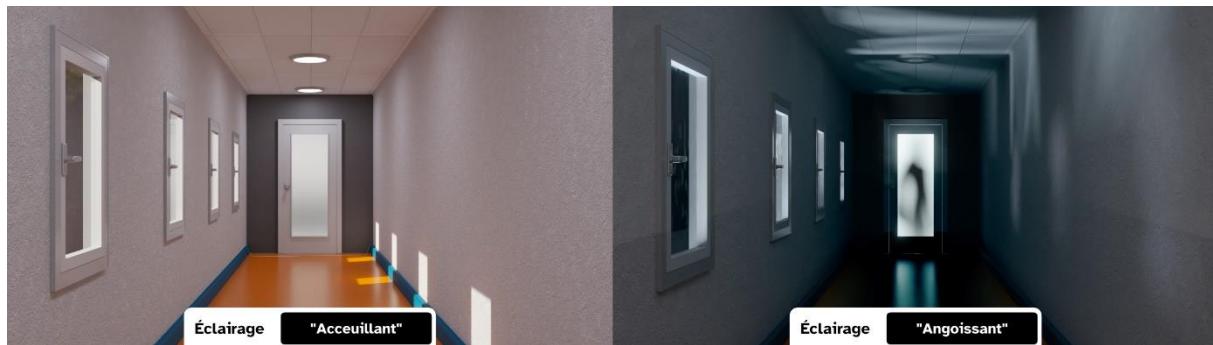


Figure 1 Comparaison d'une scène de couloir avec un éclairage accueillant et avec un éclairage angoissant

La lumière comme élément de gameplay dans le jeu vidéo

La lumière est un aspect très important de l'immersion dans les visuels des jeux vidéo. Sans lumière, il n'est pas possible de voir et donc de créer des visuels. Comme la lumière a un impact majeur sur la psychologie humaine, on peut l'utiliser à des fins de communications avec les joueurs et joueuses. On utilise par exemple le placement des sources de lumière comme un des éléments majeurs du Level Design afin de guider les joueurs et joueuses dans les environnements virtuels. On peut indiquer beaucoup de signaux en changeant la lumière, par exemple le déclenchement d'une alarme peut déclencher des lumières clignotantes pour signifier visuellement que le personnage doit fuir une zone.



Figure 2 La scène du couloir avec une lumière rouge signifiant un danger imminent

Dans ce document nous aborderons des équations et algorithmes qui nous permettent aujourd'hui de simuler un large éventail d'interactions physiques de la lumière. Nous aborderons les méthodes historiquement utilisées, les standards actuels et les algorithmes spécifiques de certains moteurs de jeux et de certains constructeurs de puces graphiques.

Les différents aspects du rendu de la lumière dans le jeu vidéo en 3D

La lumière est un élément omniprésent dans notre monde et joue un rôle crucial dans la perception de notre environnement. Dans les jeux vidéo, la simulation de la lumière est un défi constant pour les développeurs qui cherchent à créer des images immersives et réalistes. Depuis les débuts du jeu vidéo, on essaye de reproduire la lumière de manière plus ou moins fidèle pour immerger les joueurs et joueuses. Il s'agit toujours d'une illusion, d'un tour de passe-passe technique mais au fil des années, les techniques de simulation de la lumière se sont considérablement améliorées, permettant de reproduire de plus en plus des phénomènes physiques de la lumière.

Dans le jeu vidéo, on peut séparer les types d'interaction de la lumière en cinq familles de simulations spécifiques : les matériaux des surfaces, les ombres portées, l'illumination environnementale, la lumière volumétrique et les phénomènes liés aux capteurs de lumière. Ce document se concentre sur les trois premiers types de simulations dans le jeu vidéo, car ils forment la base du rendu de la lumière dans le jeu vidéo.

La surface des objets et les matériaux

Description

La surface d'un objet peut être décrite comme sa région extérieure en contact avec le reste de l'environnement. Dans la grande majorité des cas, c'est la seule partie des objets que l'on peut voir. L'interaction de la lumière avec ces surfaces avant d'atteindre nos yeux est ce qui nous permet de voir notre environnement dans le monde réel. C'est ce phénomène que l'imagerie numérique tente de reproduire. Ces interactions sont en fait des transferts d'énergie entre des photons et d'autres éléments physiques présents dans l'environnement. La composition et l'arrangement de ces éléments physiques, que l'on appellera plus communément des matériaux, affectent grandement l'apparence des objets à nos yeux.

Il est donc primordial, dans la simulation de la lumière, de pouvoir calculer avec une grande précision les différentes interactions de la lumière avec les surfaces. Il est tout aussi important de pouvoir simuler l'impact des matériaux des objets sur cette interaction si l'on veut obtenir des images fidèles à la réalité.

Historique

Moteurs de ray casting

Les moteurs de ray castings sont des moteurs de jeux permettant de projeter un environnement 2D dans une perspective simili 3D. On retrouve comme exemples de jeux notables les premiers jeux de la société *Id Software* tels que *Wolfenstein 3D* ou *Doom*. Le premier n'utilise aucun calcul de lumière lors de son affichage, hormis les informations de lumières déjà présentes dans les textures de l'environnement.



Figure 3 Image du jeu Wolfenstein 3D sorti en 1992



Figure 4 Image du jeu Doom sorti en 1993

Le second, qui utilise une version plus avancée du moteur de la société, permet d'assombrir l'environnement en fonction de sa distance avec le joueur. Cette technique donne au joueur l'opportunité de mieux distinguer les distances dans l'environnement et/ou d'afficher du brouillard dans le jeu. Elle permet également d'utiliser différentes couleurs pour simuler rudimentairement du brouillard. Bien que cette technique soit très élémentaire et loin d'être réaliste, elle est encore utilisée dans certains jeux pour simuler la perspective atmosphérique et pour pouvoir cacher le chargement et le déchargement des modèles 3D lointains.

Modèles polygonaux

La grande majorité des moteurs de rendu 3D existants représente les objets 3D en utilisant une multitude de polygones (principalement des triangles) pour représenter leurs surfaces. L'enjeu du calcul de lumière est donc d'éclairer correctement ces polygones. Pour cela, plusieurs modèles et algorithmes d'éclairage ont été développés au fil des années. Les deux principaux sont la rastérisation et le lancer de rayons.

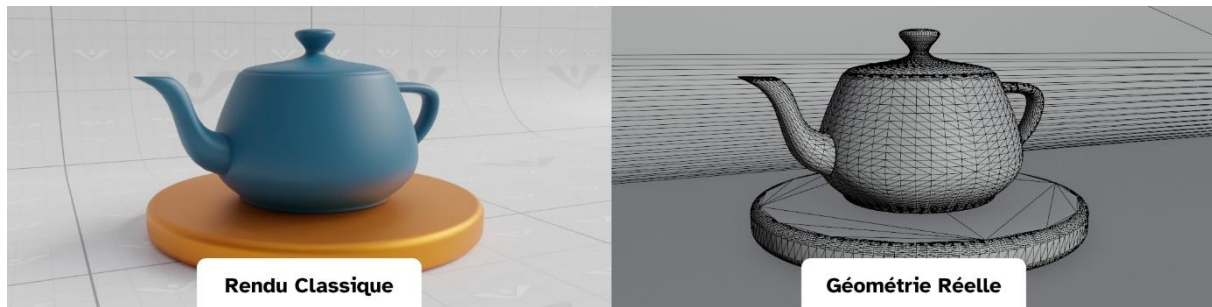


Figure 5 Comparaison d'une scène rendue en utilisant un algorithme de rendu photoréaliste (lancer de rayons) avec sa géométrie triangulaire réelle

La rasterisation

La rasterisation est une méthode de rendu 3D qui transforme une scène composée de polygones en une image finale visible à l'écran. C'est la méthode utilisée pour rendre les scènes 3D dans le jeu vidéo. Ce processus se déroule en plusieurs étapes : c'est un modèle de rendu 3D qui commence par la projection des triangles de la scène sur l'écran puis qui calcule la couleur de chaque pixel de l'écran en fonction des propriétés visuelles des triangles. Pour calculer la couleur des pixels, on applique un microprogramme appelé un shader qui calcule leurs couleurs en fonction des propriétés visuelles de la surface comme ses matériaux, sa position et son orientation. Ce modèle est très rapide à calculer mais il ne permet pas de simuler les rebonds de lumière de manière réaliste sans utiliser de techniques de rendu complexes.

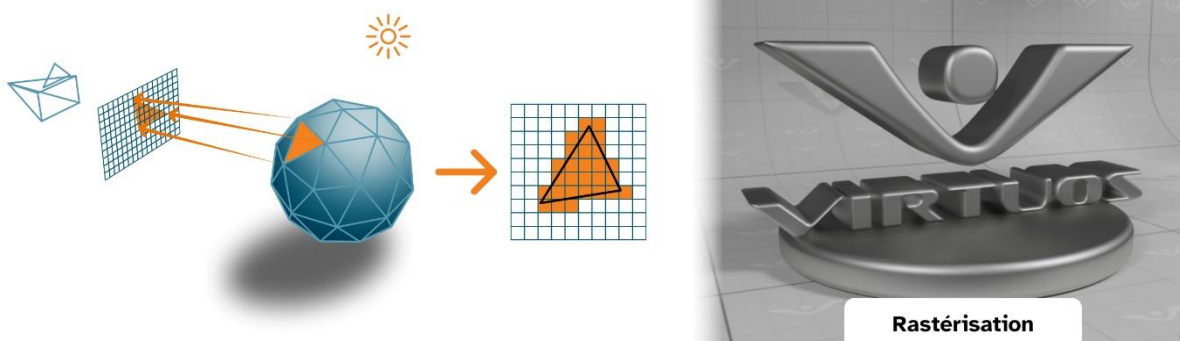


Figure 6 Schéma de l'algorithme de rasterisation et exemple de rendu rasterisé. On voit dans le schéma que la géométrie de l'objet est projetée en 2D sur l'écran.

Le lancer de rayons

Dans le cinéma d'animation et les effets spéciaux, on utilise une autre méthode de rendu appelé le lancer de rayons. Le processus de lancer de rayons consiste à tracer des rayons de lumière virtuelle dans la scène et de les laisser rebondir dans l'environnement jusqu'à toucher une source lumineuse.

On peut ensuite remonter le chemin que la lumière a prise depuis la source lumineuse jusqu'à l'observateur pour calculer une partie de l'énergie lumineuse du pixel. On répète cette opération plusieurs fois pour chaque pixel de l'écran pour obtenir l'image finale éclairée. Cette méthode peut être très lente à calculer mais modélise plus fidèlement le comportement physique de la lumière et permet d'obtenir des images très réalistes.

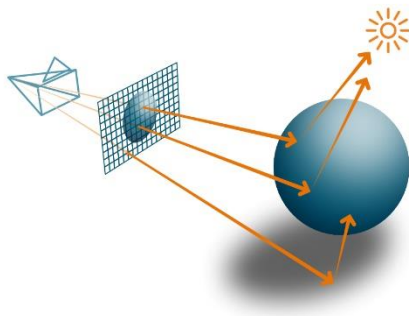


Figure 7 Schéma de l'algorithme lancé de rayons et exemple de rendu en lancé de rayons, on voit dans le schéma que l'on projette des rayons dans la scène depuis la caméra



Figure 8 Comparaison du rendu d'un modèle 3D métallique en utilisant la rastérisation et le lancer de rayons

Dans la comparaison ci-dessus on remarque que les calculs de rebonds de lumières comme les reflets ou l'occlusion ambiante sont plus précis en utilisant un algorithme de lancer de rayons. On peut remarquer que l'algorithme de rastérisation calcule les rebonds de lumière de manière grossière comparativement au lancer de rayons. En effet les reflets et l'ombre rastérisés sont plus approximatifs que les ombres obtenues par lancer de rayons. De plus, le matériau métallique est plus sombre dans l'image rastérisée, cela est dû au fait que l'algorithme de rastérisation ne permet pas de calculer autant de reflets que l'algorithme de lancer de rayons.

La différence de qualité des deux images est très visible quand on les compare l'une à côté de l'autre, mais c'est moins le cas en jeu. Hormis dans certaines situations, la différence de qualité n'est pas réellement perceptible. La différence de performance entre les deux algorithmes est cependant très

perceptible. L'image rastérisée a été générée en moins de 10 millisecondes tandis que l'image créée à partir du lancer de rayons classique a été générée en plusieurs secondes. Il est possible d'effectuer du lancer de rayons plus rapidement, mais on peine encore aujourd'hui à atteindre un temps de rendu en dessous de 16 millisecondes.

Ces deux modèles de rendus fonctionnent de manière très différente. L'un est optimisé pour sa rapidité et projette la géométrie de la scène sur l'écran pour calculer la lumière tandis que l'autre est optimisé pour simuler plausiblement la lumière et projette des rayons sur la géométrie de la scène pour calculer la lumière. Comme la rastérisation permet de simuler très rapidement les interactions simples de la lumière et que le lancer de rayons est efficace pour calculer les interactions complexes de la lumière, on peut combiner les deux méthodes pour obtenir des images qui offrent un bon compromis entre la qualité visuelle et la rapidité de calcul.

Ombrage de Lambert / Flat Shading

L'ombrage de Lambert définit les propriétés d'un matériau mat idéal, c'est-à-dire un matériau qui produit uniquement des réflexions diffuses. Cette technique a été conceptualisée par *Johann Heinrich Lambert* dans son livre intitulé "Photometria" en 1760. Elle permet de calculer la luminosité D_S d'une surface en fonction du produit scalaire de l'angle entre la normale $\vec{N}$ d'une surface et la direction vers la lumière $\vec{L}$, la couleur C de la surface et l'intensité de la lumière I .

$$D_S = \vec{L} \cdot \vec{N} * C * I$$

Figure 9 Équation de la réflexion diffuse de Lambert

Cette fonction donne une approximation d'une surface diffuse mais ne permet pas l'interpolation de la direction d'une surface discrétisée, tel qu'un modèle 3D polygonal. Ces modèles ont donc une luminosité ressemblant à des facettes avec des discontinuités visibles.



Figure 10 Torus polygonal utilisant un éclairage de Lambert

L'ombrage de Lambert est très simple conceptuellement et très rapide à calculer. Comme il se calcule une fois par face, il est également très rapide pour l'affichage d'objets polygonaux.

Parmi les jeux utilisant ce mode d'éclairage, on peut retrouver "*I Robot*" d'Atari en 1984, l'un des premiers jeux d'arcade en 3D.

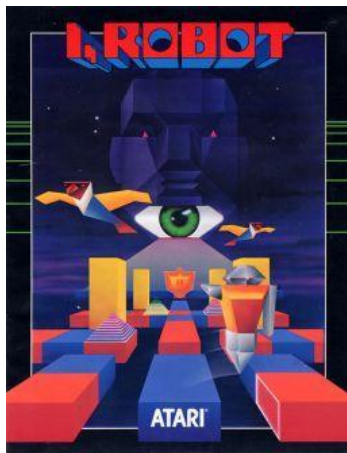


Figure 11 Affiche du Jeu I Robot



Figure 12 Image Issue du jeu I Robot

Ombrage de Gouraud / Smooth Shading

L'ombrage de Gouraud est une technique d'interpolation permettant de faire disparaître les discontinuités dans l'éclairage des surfaces polygonales. Cette technique a été publiée par *Henri Gouraud* en 1971 dans son ouvrage nommée "*Computer Display of Curved Surfaces*".

Cette technique consiste à estimer la normale moyenne de la surface à chaque sommet des polygones en faisant la moyenne des normales des polygones adjacents. Ensuite, elle calcule la luminosité en se basant sur un algorithme comme celui de *Lambert*. Enfin, pour chaque *pixel* du polygone, elle effectue

une interpolation linéaire dans l'espace de l'écran des valeurs de luminosité de chaque sommet du polygone.

Cette technique permet de faire disparaître les discontinuités de l'ombrage des modèles 3D sans devoir recalculer l'ombrage pour chaque *pixel* de l'écran. Elle est donc peu coûteuse et améliore grandement l'aspect graphique des rendus 3D par rapport à un ombrage de Lambert.

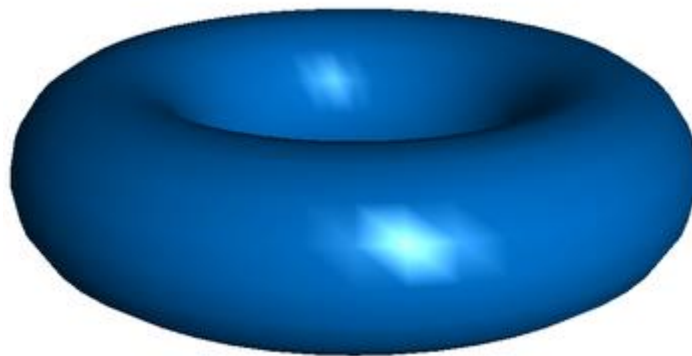


Figure 13 Torus polygonal utilisant un éclairage de Gouraud

Bien que cet algorithme ne permette pas d'afficher correctement des reflets spéculaires, l'ombrage de Gouraud a été une très grande avancée dans le monde du jeu vidéo car il a permis l'affichage de formes complexes et organiques sans devoir multiplier exponentiellement le nombre de polygones.

Parmi les jeux notables utilisant l'ombrage de *Gouraud* on peut trouver "Mario 64", sorti en 1996 sur Nintendo 64, et "Final Fantasy 7", sorti en 1997 sur Playstation.

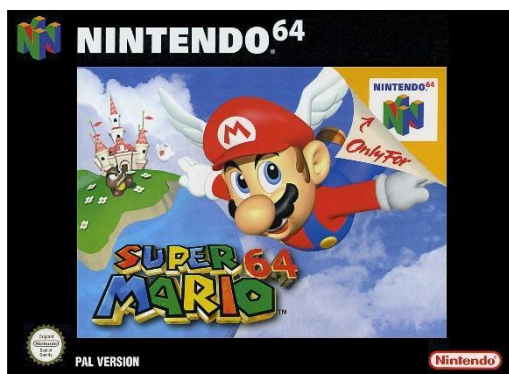


Figure 14 Affiche du jeu Mario 64



Figure 15 Image issue du jeu Mario 64

Ombrage de Phong

L'ombrage de Phong englobe deux techniques : le modèle de réflexion de *Phong* et l'interpolation de *Phong*.

Le modèle de réflexion de *Phong* a été publié par *Bui Tuong Phong* en 1973. Il s'agit d'un modèle de réflexion empirique qui définit les propriétés d'un matériau en prenant en compte non seulement les réflexions diffuses comme le modèle de *Lambert* mais également les réflexions spéculaires ou brillantes, ainsi que la luminosité ambiante optionnellement. En ajoutant ces termes à la formule, on peut obtenir un modèle de réflexion englobant la majorité des matériaux. Pour le terme diffus, on utilise la formule de *Lambert*, et pour le terme spéculaire, on utilise la formule suivante :

On calcule la réflexion spéculaire R_s d'une surface en fonction du produit scalaire de l'angle entre la direction vers la caméra $\vec{C}$ d'une surface et la direction de la lumière reflétée sur la surface de l'objet $\vec{R}_L$, l'intensité spéculaire S de la surface, la brillance de la surface B et l'intensité de la lumière I .

$$R_s = (\vec{C} \cdot \vec{R}_L)^B * S * I$$

Figure 16 Formule de la réflexion spéculaire de *Phong*

$$\vec{R}_L = \vec{L} - 2\vec{N} * (\vec{L} \cdot \vec{N})$$

Figure 17 Formule du vecteur direction de la lumière reflétée sur une surface de normale N

Les réflexions spéculaires permettent d'afficher des objets ayant des matériaux bien plus complexes que le modèle de *Lambert*. En effet, l'ajout des réflexions spéculaires permet d'afficher des objets plus ou moins brillants, comme du plastique, de la peau, de la céramique ou même du métal. Pour afficher du métal, la formule doit être légèrement modifiée en multipliant le reflet spéculaire par la couleur de la surface et en supprimant le terme diffus. Cette modification est due à la spécificité physique des réflexions des matériaux métalliques.

Le modèle d'interpolation de *Phong*, quant à lui, n'effectue pas son interpolation sur les couleurs des sommets des polygones, mais sur leurs normales.

Ici encore, on estime la normale moyenne de la surface pour tous les sommets des polygones en fonction des normales des faces adjacentes aux sommets. Ensuite, pour chaque pixel de la surface, on

effectue une interpolation linéaire de la normale des sommets du polygone et on calcule la luminosité en se basant sur le modèle de réflexion de *Phong* (ou autre modèle).

En utilisant ces deux techniques, on peut afficher des modèles 3D avec une qualité bien supérieure à l'éclairage de *Lambert* et de *Gouraud*. Le modèle de réflexion permet d'afficher les reflets des matériaux brillants et l'interpolation permet d'avoir un détail d'éclairage à l'échelle des pixels. Comme l'éclairage de Phong effectue des calculs de lumière plus complexes et pour chaque pixel, cette technique est bien plus coûteuse que les techniques précédentes, mais elle permet d'obtenir une apparence des matériaux réaliste. On peut utiliser le modèle de réflexion de *Phong* avec une interpolation de *Gouraud* pour gagner en performance, mais cette technique ne permet pas d'afficher des reflets détaillés en raison de l'échantillonnage trop faible de la lumière sur la surface de l'objet.

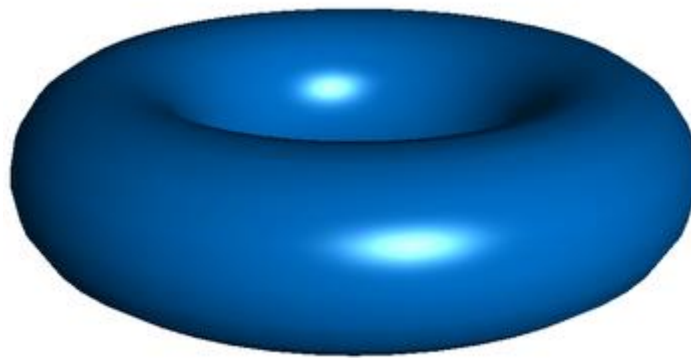


Figure 18 Torus polygonal utilisant un éclairage de Phong

Ce modèle d'éclairage étant bien plus coûteux que les alternatives antérieures à cette technique, son utilisation a d'abord été réservée aux machines d'arcade avec l'introduction du jeu "*Brave Firefighters*" sur le système *Hikaru* de *Sega* en 1999.

Parmi les jeux notables utilisant l'ombrage de Phong, on peut retrouver "*Half Life 2 : Episode 2*" en 2006, "*Titanfall*" en 2014, "*Doom*" en 2016, "*Fortnite*" en 2017 ou "*Uncharted : Legacy of Thieves Collection*" en 2022.



Figure 19 Image issue du jeu Titanfall (2014)



Figure 20 Image issue du jeu Doom (2016)

Ce modèle est aujourd'hui l'un des standards des modèles d'éclairage dans le jeu vidéo et la majorité des autres modèles d'éclairage standards dans le jeu vidéo s'inspirent du modèle de *Phong*.

Ombrage de Blinn-Phong

L'ombrage de Blinn-Phong est une modification du modèle de réflexion de *Phong*. Ce modèle a été publié par *James F. Blinn* en 1977 et est une modification mineure du modèle de réflexion spéculaire qui permet d'obtenir des réflexions potentiellement plus réalistes et étant moins coûteuses que le modèle de *Phong*. Ici on ne calcule pas l'intensité de la réflexion grâce au produit scalaire de la direction vers la caméra $\vec{C}$ et la direction reflétée de la lumière $\vec{R}_L$ reflété par rapport à la normale $\vec{N}$ de la surface. Ce produit scalaire est remplacé par un produit scalaire entre la normale de la surface $\vec{N}$ et le vecteur milieu $\vec{M}$ entre la direction vers la caméra $\vec{C}$ et la direction vers la lumière $\vec{L}$.

$$R_s = (\vec{N} \cdot \vec{M})^B * S * I$$

Figure 21 Formule de la réflexion spéculaire de Blinn-Phong

$$\vec{M} = \frac{\vec{L} + \vec{C}}{\|\vec{L} + \vec{C}\|}$$

Figure 22 Formule du vecteur milieu $\vec{M}$ entre la direction vers la caméra $\vec{C}$ et la direction vers la lumière $\vec{L}$

Le modèle de *Blinn-Phong* a de nombreux avantages par rapport au modèle de *Phong*. Cette formule est généralement plus efficace que celle de *Phong* car le calcul de la réflexion d'un vecteur par rapport à une normale est plus coûteux que le calcul d'un vecteur ayant une direction au centre de deux autres vecteurs. Les réflexions spéculaires de *Phong* peuvent avoir des coupures abruptes quand l'angle de

vue entre l'observatoire et la normal de la surface est raide. De plus, les réflexions spéculaires qui sont produites par cette formule sont plus ellipsoïdales que le modèle de *Phong* et donc sont une meilleure approximation de la fonction de distribution de réflectance bidirectionnelle, un algorithme de réflexion avancé et très réaliste. Enfin, le modèle de *Blinn-Phong* a tendance à avoir des réflexions plus floues que le modèle de *Phong*. Pour remédier à ce problème, il est généralement conseillé d'augmenter légèrement le coefficient de brillance des matériaux pour obtenir un résultat similaire au modèle de *Phong*.

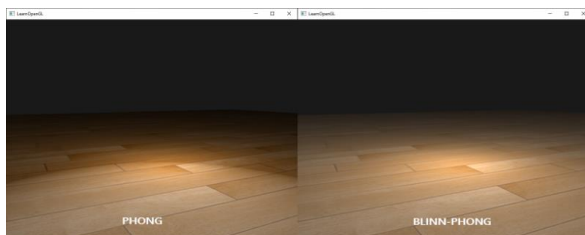


Figure 23 Illustration des problèmes de réflexion spéculaire du modèle de Phong comparé au modèle de Blinn-Phong

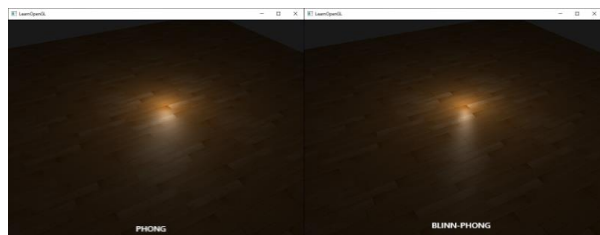


Figure 24 Comparaison des reflets spéculaires du modèle de Phong et du modèle de Blinn-Phong

Le modèle *Blinn-Phong* s'est imposé comme le modèle de réflexion standard dans le jeu vidéo au début des années 2000 en raison de son efficacité et de son apparence. Il a été le modèle de réflexion par défaut dans OpenGL et DirectX.

Les modèles d'ombrage que nous avons abordés jusqu'à présent sont très utiles et ont permis d'augmenter grandement la qualité des images de synthèse dans le jeu vidéo. Il faut cependant savoir que ces modèles ont été développés de manière empirique, c'est-à-dire qu'il se basent sur des observations du monde réels et non sur des données scientifiques. Ces modèles ont tendance à ne pas respecter certaines lois de l'optique comme celle de la conservation de l'énergie que l'on abordera dans le document dans la prochaine partie. Pour obtenir des images de synthèse ayant une simulation de la lumière plausible, une nouvelle approche de l'illumination a été développée, connue sous le nom de Physically Based Rendering.

Le Physically Based Rendering

Description

Le Physically Based Rendering (*PBR*) est une approche de l'infographie dont le but est de se baser sur des données physiques et des équations se rapprochant du comportement réel de la lumière. Cette approche permet d'effectuer l'approximation du comportement de la lumière en fonction de différents aspects des matériaux des objets. Les matériaux créés en utilisant les techniques de *PBR*

doivent avoir leurs propriétés basées sur des données réelles. Par exemple, un objet doré n'est pas juste jaune et brillant, mais est bien un objet métallique ayant des reflets d'une couleur spécifique. Le comportement des rayons de lumière doit également être simulé en se basant sur des propriétés réelles de la lumière. De plus, comme l'approche du PBR sépare l'aspect visuel du matériau de l'éclairage, un matériau doit garder une apparence correcte quel que soit l'environnement lumineux, ce qui n'est généralement pas le cas des modèles plus anciens.

Les théories sur lesquelles se base le PBR ont été principalement établies dans les années 1980 par les chercheurs de l'*Université de Cornell*. Ces chercheurs en rendu ont commencé à établir une base théorique solide pour le rendu, en mettant l'accent sur l'exactitude physique.

C'est à cette époque que *Turner Whitted* a introduit l'idée d'utiliser le Ray Tracing pour les effets d'éclairage global, ouvrant la voie à une simulation précise de la distribution de la lumière dans les scènes. *Turner Whitted* est également le créateur du premier film utilisant du Ray Tracing. Nous verrons dans ce document que les travaux de *Turner Whitted* ont impacté l'ensemble des domaines de l'imagerie numérique, en commençant par le cinéma d'animation et les effets spéciaux jusqu'aux jeux vidéo aujourd'hui. Son papier de recherche nommé "*An improved illumination model for shaded display*", publié le 1er juin 1980, est considéré comme l'un des piliers de l'imagerie numérique moderne.

Le terme Physically Based Rendering a ensuite été popularisé par *Matt Pharr*, *Greg Humphreys* et *Pat Hanrahan* dans leur livre du même nom en 2004. Cet ouvrage est considéré comme un des fondamentaux de l'informatique graphique moderne et a permis aux créateurs du livre de remporter un *Oscar technique*. Il en existe maintenant dans sa quatrième édition.

Cette technique a été implémentée dans un jeu vidéo grand public pour la première fois en 2013 nommé "*Remember Me*". Le PBR a révolutionné la qualité visuelle des jeux vidéo, des films d'animation et des effets spéciaux dans le cinéma. Il permet des rendus plus réalistes, des matériaux plus convaincants et une meilleure interaction entre la lumière et les surfaces.

Cette approche a également permis d'améliorer grandement la compatibilité des modèles 3D et des moteurs de rendu 3D. En effet, une fois que cette approche est devenue standard, les modèles 3D ont pu être transférés entre les moteurs de rendu sans devoir modifier leurs matériaux pour s'afficher correctement dans chaque logiciel.

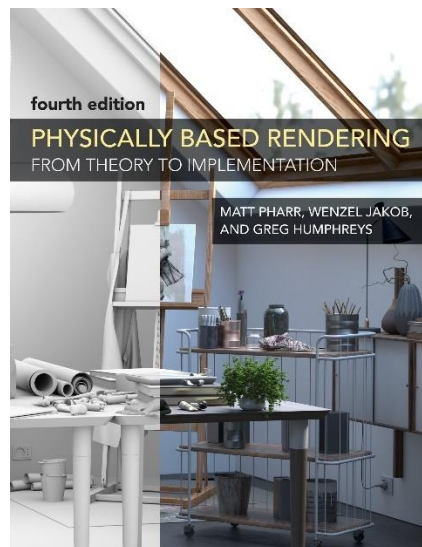


Figure 25 Première de couverture de la quatrième édition du livre "Physically Based Rendering"

Équations de la lumière

Il existe différents modèles permettant de simuler les différents comportements de la lumière au contact d'une surface. Les modèles les plus utilisés en infographie sont des Fonctions de distribution bidirectionnelle. Il en existe plusieurs types, parmi lesquels BRDF, BTDF, BSDF et BSSRDF. En d'autres termes, il s'agit de fonctions permettant de décrire la distribution d'un aspect de la lumière sur deux axes, généralement le zénith et l'azimut, en fonction de la direction vers la lumière, la normale de la surface et la direction vers l'observateur.

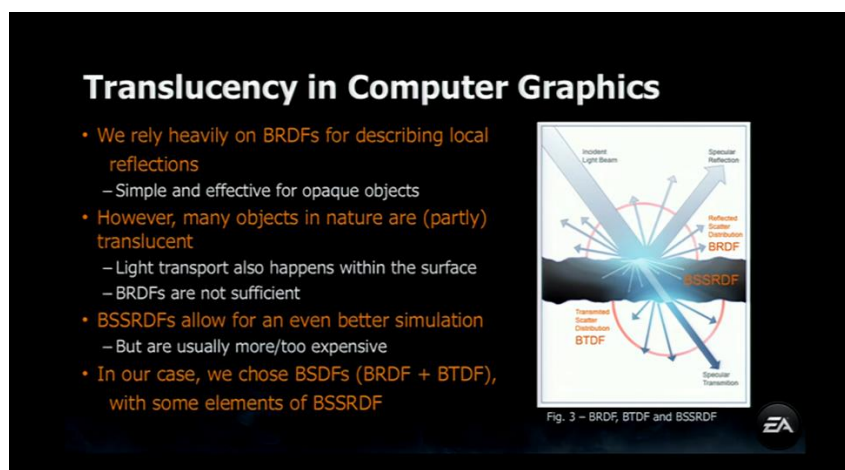


Figure 26 Schéma des différentes Fonctions de Distributions Bidirectionnelles par Electronic Arts

Ces fonctions permettent de décrire les propriétés d'un rayon de lumière au contact d'une surface, par exemple sa couleur, son intensité, sa direction et sa position de départ. Pour chaque type de

fonction de distributions bidirectionnelles, il existe plusieurs implémentations d'algorithmes pour différents usages. Certaines implémentations sont très précises pour des usages scientifiques, tandis que d'autres sont plus rapides mais moins précises et sont généralement utilisées en infographie.

La Nasa utilise par exemple ces fonctions pour pouvoir trouver les matériaux des surfaces photographiées par satellites en calculant leurs Fonction de distribution bidirectionnelle de réflectance (BRDF) à partir de photographies sous plusieurs éclairages et en les comparant avec des bases de données préexistantes. On peut également mesurer la BRDF d'une surface à l'aide d'un gonioréfectomètre.

En infographie, ces fonctions permettent de calculer la lumière en utilisant des algorithmes de lancer des rayons pour créer des effets spéciaux dans les films d'animations et les images de synthèses. Dans les jeux vidéo, on utilise des algorithmes de rasterisation qui permettent l'approximation des résultats des fonctions de distributions bidirectionnelles.

On fera la distinction entre des équations de lumière basées sur la réalité physique et des fonctions empiriques non basées sur la physique réelle de la lumière.

Fonction de distribution bidirectionnelle de réflectance (BRDF)

La Fonction de Distribution Bidirectionnelle de Réflectance (BRDF) est une fonction décrivant comment une source de lumière est reflétée sur une surface opaque. Cette fonction permet de décrire la grande majorité des matériaux existants et des surfaces opaques.

Il en existe beaucoup :

BRDF ZOO I

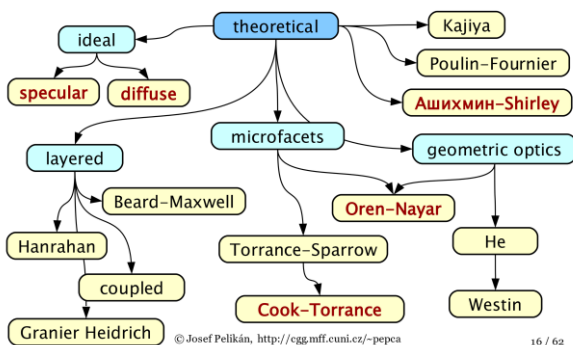


Figure 27 Historique des BRDF basées sur la physique par Josef Pelikán

BRDF ZOO II

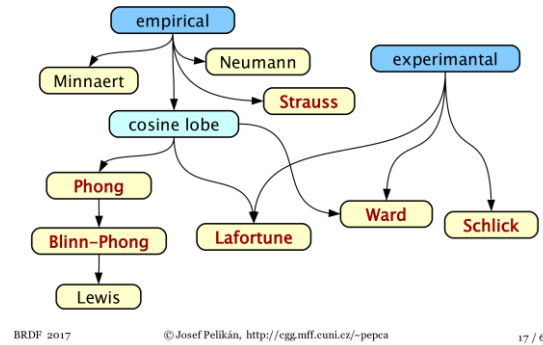


Figure 28 Historique des BRDF empiriques et expérimentales par Josef Pelikán

Pour qu'une BRDF soit considérée comme physiquement plausible (*Physically Based*), elle doit satisfaire la loi de la conservation d'énergie : la quantité de lumière reflétée par une surface ne doit jamais être supérieure à la quantité de lumière entrante sur la surface. La quantité de lumière reflétée peut cependant être inférieure à la quantité entrante, c'est d'ailleurs le cas de tous les matériaux existants qui convertissent une partie de la lumière visible en chaleur, en réémettant des ondes infrarouges. C'est pour cela qu'un matériau sombre chauffe plus au soleil qu'un matériau clair.

Le modèle de *Blinn-Phong* n'est par exemple pas considéré comme basé sur la réalité et il ne respecte pas la loi de la conservation d'énergie, car le modèle peut produire plus de lumière qu'il n'en reçoit dans certaines situations.

La BRDF la plus utilisée aujourd'hui dans le PBR dans le jeu vidéo est le modèle de *Cook-Torrance* ou des modèles qui s'en sont inspirés. Ce modèle utilise une diffuse de *Lambert* tout comme *Phong* et *Blinn-Phong*. On peut également utiliser d'autres modèles de diffuses comme par exemple le modèle de *Burley*, celui de *Oren-Nayar* ou celui de *Hanrahan-Krueger*.

Pour la réflexion spéculaire, le modèle utilisé est celui des Micro-Facettes. Il s'agit d'un modèle qui simule la rugosité / brillance des surfaces en considérant une surface plane comme une surface faite de micro-facettes d'angles plus ou moins alignés avec la surface réelle, l'alignement de ces facettes étant contrôlé par un paramètre de rugosité.

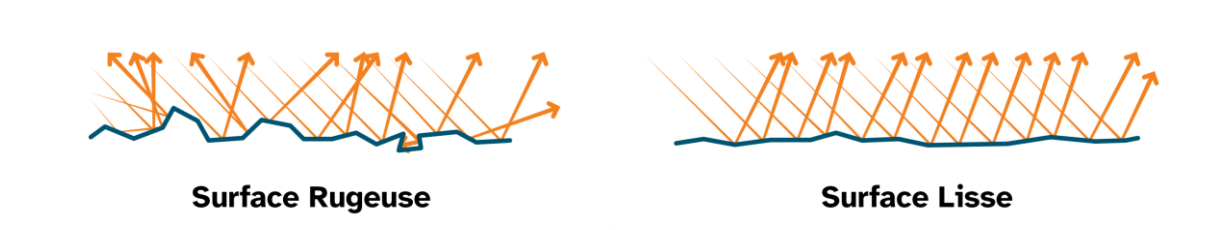


Figure 29 Simulation de la rugosité par algorithme de micro-facettes

La BRDF de Cook-Torrance est définie comme tel :

$$f_r = k_d * f_{\text{lambert}} + k_s * f_{\text{cook-torrance}}$$

Figure 30 Fonction BRDF de Cook-Torrance

Ici k_s et k_d sont les ratios entre les réflexions spéculaires $f_{\text{cook-torrance}}$ et les réflexions diffuses f_{lambert} . Pour des matériaux métalliques le ratio sera de $k_s = 100\%$ / $k_d = 0\%$ et pour des matériaux non métalliques (isolants/diélectriques) le ratio sera autour de $k_s = 4\%$ / $k_d = 96\%$.

$$f_{\text{cook-torrance}} = \frac{D * F * G}{4 * (\omega_c * n_o) * (\omega_l * n_o)}$$

Figure 31 Fonction de réflexion spéculaire de Cook-Torrance

Définition des termes :

- D - Fonction de distribution de la loi normale
 - Simule la distribution des directions des micro-facettes et leurs effets sur la forme de la réflexion spéculaire
 - Est contrôlée par un paramètre de rugosité.
- G - Fonction géométrique
 - Simule l'assombrissement des surfaces rugueuses dû à l'occlusion possible des micro-surfaces qui peuvent faire apparaître des zones concaves.
- F - Équation de *Fresnel*
 - Équation décrivant le ratio de la réflexion de la surface en fonction de l'angle.
 - Une surface vue de face aura des réflexions moins visibles et plus floues qu'une surface vue d'un angle quasiment parallèle.
 - La distribution de la réflexion en fonction de la normale est contrôlée par un paramètre d'Indice de Réfraction (IOR) basé sur la densité d'un matériau.
- ω_c - Angle solide aligné au vecteur de la surface vers la caméra.

- ω_l - Angle solide aligné au vecteur de la surface vers la source lumineuse.



Figure 32 Distribution normale des reflets spéculaires en fonction du paramètre de rugosité

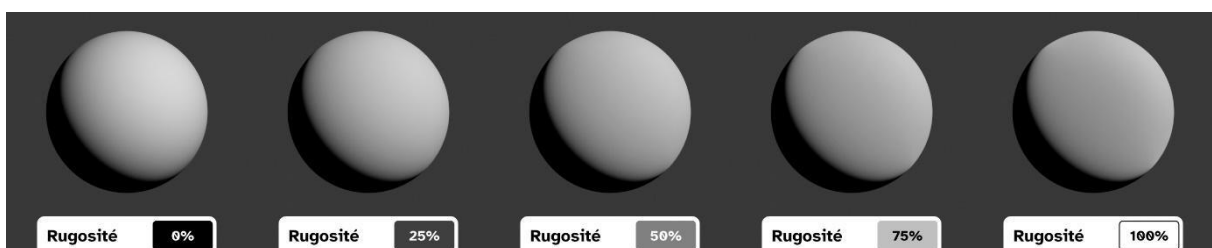


Figure 33 Assombrissement d'une surface en fonction de sa rugosité

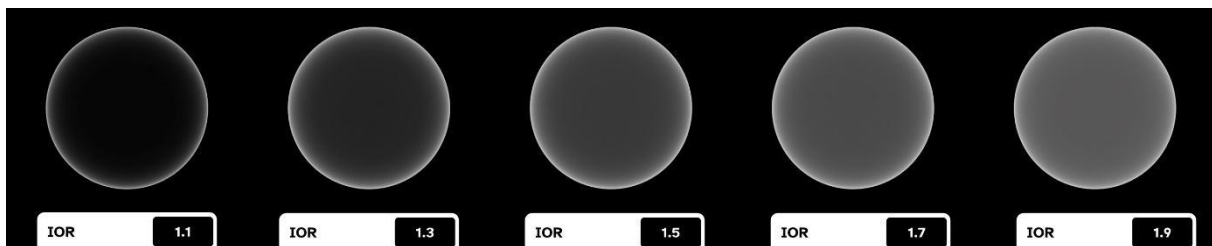


Figure 34 Distribution de la fonction de Fresnel en fonction de l'Indice de Réfraction et impact sur la réflexion spéculaire d'un matériau

Dans le jeu vidéo, on remplace les termes D , G et F par des algorithmes qui simulent leurs comportements tout en ayant des performances acceptables. Il existe un grand nombre d'algorithmes étant plus ou moins rapides ou réalistes. Le choix des algorithmes est donc un choix de compromis entre la performance de rendu et la fidélité de l'image.

Bien que chaque moteur de rendu utilise ces propres algorithmes pour le rendu 3D des BSDF, la majorité est grandement basée sur la *Disney's Principled BRDF* publiée par Brent Burley en 2012. Ce modèle a été utilisé pour la première fois pour la production du film *Le Monde De Ralph* et a rapidement inspiré différents logiciels pour la création de leurs matériaux physique : *Unreal Engine 4 et 5*, *Unity URP*, *Blender*, *Substance Painter*, *Autodesk* etc. Aujourd'hui, la BRDF de *Disney* ou des modèles similaires sont les standards dans tous les outils de création de jeux vidéo.

Unreal Engine 4 a par exemple utilisé l'algorithme Trowbridge-Reitz GGX pour l'approximation de D , l'algorithme Smith's Schlick GGX pour l'approximation de G et l'algorithme Fresnel-Schlick pour l'approximation de F ..

Fonction de distribution bidirectionnelle de transmission (BTDF)

La Fonction de Distribution Bidirectionnelle de Transmission (BTDF) est une fonction décrivant comment une source de lumière est réfractée par une surface transparente. Ce modèle est utilisé pour pouvoir décrire le comportement de la lumière au contact de matériaux réfractant la lumière tels que le verre, le plastique, l'eau, le diamant ou l'acrylique.



Figure 35 Comparaison de matériaux ayant chacun un indice de réfraction différent, la valeur de leur indice de réfraction est affichée

Tout comme la BRDF, il existe plusieurs méthodes pour calculer la BTDF, mais l'approche utilisée dans le jeu vidéo et l'infographie en temps réel sont très peu précises. En effet, la transparence et la réfraction sont les aspects les plus compliqués à rendre fidèlement dans le jeu vidéo car ils nécessitent de simuler la lumière en utilisant du lancer de rayon qui est encore aujourd'hui extrêmement lourd à calculer en temps réel.

Pour pallier cela, les moteurs de rendu en temps réel utilisent différentes techniques pour pouvoir obtenir des fausses réfractions convaincantes, la plupart du temps en utilisant l'image de la scène déjà rendue et en la distordant en fonction des angles la surface. Cette image peut également être floutée pour donner un aspect rugueux à la réfraction. Cette méthode arrive assez rapidement à sa limite quand une partie de la scène réfractée n'est pas visible de base. Comme l'information de couleur n'existe pas, on ne peut pas l'afficher correctement. Certains moteurs de rendu en temps réel comme *Unreal Engine* permettent d'utiliser des algorithmes de Ray Tracing pour obtenir des réfractions réalistes qui se basent sur des BTDF classiques.

Fonction de distribution bidirectionnelle de diffusion de la lumière (BSDF)

La Fonction de Distribution Bidirectionnelle de Diffusion de la lumière, (Bidirectional Scattering Distribution Function, ou BSDF) est une fonction englobant la BRDF et la BTDF. C'est le modèle

d'équation de la lumière standard pour les moteurs de rendu 3D PBR, qu'il s'agisse des moteurs de lancer de rayon ou d'une application en temps réel utilisant la rastérisation.

Comme expliqué précédemment, beaucoup de moteurs de rendu en lancer de rayon et également en temps réel s'inspirent de la BRDF de *Disney* et de différentes BTDF pour la simulation de la réfraction.

Cette standardisation a permis aux artistes de garantir que les modèles 3D aient un rendu similaire dans les logiciels de création et dans les moteurs de jeux.

Bidirectional Scattering-Surface Distribution Function (BSDF)

La Fonction de Distribution Bidirectionnelle de Diffusion de la lumière de Surface translucide, (Bidirectional Scattering-Surface Distribution Function, ou BSSDF) est une fonction très proche d'une BSDF classique mais qui prend aussi en compte l'aspect translucide des matériaux ou Subsurface Scattering. Un objet translucide peut réfracter la lumière comme les objets transparent mais a la particularité de diffuser une partie de la lumière qui le traverse dans toutes les directions. La lumière va rebondir dans son volume et une partie va être réémise vers l'extérieur.

La fonction BSSDF est composée de deux autres fonctions à la même manière que la BSDF. La première fonction est la BSSRDF, elle calcule la réflexion de la lumière dans les matériaux généralement opaque mais qui deviennent translucides quand leur surface sont fines. La BSSRDF est utile pour afficher les matériaux comme le papier, les feuilles ou la peau. La seconde fonction est la BSSTDF, elle calcule la réfraction de la lumière dans les surfaces semi-transparentes qui diffusent une partie de la lumière réfractée. La BSSTDF est utile pour afficher des objets transparents qui comportent des impuretés comme le quartz, le raisin ou la glace. Les fonctions BSSRDF et BSSTDF pour la BSSDF sont des analogues à la BRDF et la BTDF pour la BSDF.

Parmi les matériaux qui ont des propriétés translucides, on peut retrouver la peau, la cire, la glace, les feuilles, les plastiques fins, le lait, le quartz etc. Comme cette fonction est plus lourde à calculer qu'une BSDF classique, on réservera son utilisation pour les matériaux spécifiques comme ceux mentionnés plus précédemment dans le document.

En 2013, *Electronic Arts* a présenté sa méthode de rendu d'une BSSDF pour des applications en temps réel. Cette approximation est assez simpliste mais permet d'obtenir des résultats convaincants sans grandes pénalités de performances.

$$L_{SSR} = L_a * (\vec{N} \cdot \vec{L} + \vec{C} \cdot (-L))$$

Figure 36 Equation du rendu de la translucidité d'une surface

Ici, l'équation décrit l'intensité de la lumière retransmise L_{SSR} en fonction d'une diffuse de Lambert $\vec{N} \cdot \vec{L}$, du produit scalaire entre la direction vers la caméra $\vec{C}$ et la direction inverse de la lumière $-\vec{L}$, le tout étant atténué par un facteur d'atténuation de la lumière L_a déterminé par des paramètres artistiques et l'épaisseur de la surface.

L'aspect translucide de la peau est très important dans l'imagerie numérique si l'on cherche à obtenir des images photoréalistes. L'absence de translucidité cause la peau à ressembler à du plastique et fait ressortir la nature numérique de l'image. Ce problème a été présent longtemps au cinéma et dans le jeu vidéo. L'absence de cet effet est l'une des raisons du phénomène de l'Uncanny Valley ou vallée de l'étrange. Ce phénomène peut être décrit par une sensation de malaise envers les images quasiment réalistes de visages humains mais pas assez pour les prendre pour une photographie.



Figure 37 Comparaison du réalisme de la peau en fonction de la prise en compte de sa translucidité, le modèle éclairé sans translucidité a une apparence de plastique.

Pour remédier à ce problème, le moteur *Unreal Engine 5* utilise des modèles d'éclairages différents pour le rendu des surfaces translucides. Le premier modèle est le Subsurface Scattering model, qui est rapide et utilisé pour la majorité des surfaces. Il permet d'obtenir des performances correctes tout en ayant un réalisme suffisant pour des objets de l'environnement. Le second modèle est appelé le Subsurface Profile model. Il est plus complexe et spécialisé dans le rendu de la peau. Il se base sur des informations réelles de la translucidité de la peau mais peut être plus lent à calculer, il est donc utilisé exclusivement pour le rendu de la peau.

L'application du Physically Based Rendering dans le jeu vidéo

L'arrivée du Physically Based Rendering dans le jeu vidéo a permis d'augmenter grandement la qualité des graphismes. Cette approche de l'infographie a également permis de standardiser la création des

matériaux. La plupart des moteurs de rendu dans le jeu vidéo ont des matériaux aux paramètres similaires.

Il existe deux grandes familles de matériaux PBR qui utilisent deux workflows de création de textures différents :

Le workflow spéculaire est le plus ancien. Il est très flexible mais peut être peu intuitif pour des matériaux complexes et il a tendance à être moins optimisable que son nouvel homologue. Les termes de sa BRDF sont contrôlés par trois paramètres : la diffuse, le spéculaire et la brillance.

Le workflow métallique est plus récent. Il permet, à l'aide de textures, de séparer les matériaux en deux catégories principales, les matériaux métalliques et les non métalliques. Il est légèrement moins flexible mais a tendance à être plus intuitif et est légèrement moins lourd que son prédécesseur. Les termes de sa BRDF sont contrôlés par trois paramètres : l'albédo, la rugosité et l'aspect métallique.

Ces deux workflows sont tous les deux utilisés mais le workflow métallique est aujourd'hui plus standardisé dans le milieu du jeu vidéo. C'est pour cette raison que cette étude se focalisera principalement sur le paramètre des matériaux basé sur le workflow métallique.

Les autres paramètres de ces matériaux sont quant à eux standard entre eux, et peuvent être utilisés de manière interchangeable.

L'Albédo



Figure 38 Différents aspects visuels d'un matériau en fonction de la couleur de son albédo

L'albédo désigne la couleur intrinsèque du matériau sans prendre en compte des informations de lumière. Pour les matériaux diélectriques (non métalliques), ce paramètre contrôle le terme diffus de la BRDF. Pour les métaux, ce paramètre contrôle la couleur des réflexions spéculaires. Le terme diffus de la BRDF est également rendu nul.

L'aspect Métallique



Figure 39 Différents aspects visuels d'un matériau en fonction de son aspect métallique

Le paramètre d'aspect métallique est dans 99% des cas un masque binaire qui décrit si une surface est métallique ou non. La différence entre un matériau métallique et diélectrique est la suivante :

Un matériau métallique ne contient pas de thème diffus et reflète la lumière uniquement de manière spéculaire. Il a également la propriété de colorer ce reflet, contrairement aux matériaux diélectriques dont les reflets spéculaires sont non colorés et autour de 4% de réflexion.

La rugosité



Figure 40 Différents aspects visuels d'un matériau en fonction de sa rugosité

La rugosité contrôle l'aspect brillant des réflexions spéculaires, la brillance des réflexions de cet objet est inversement proportionnelle à sa rugosité.

Les textures de normales

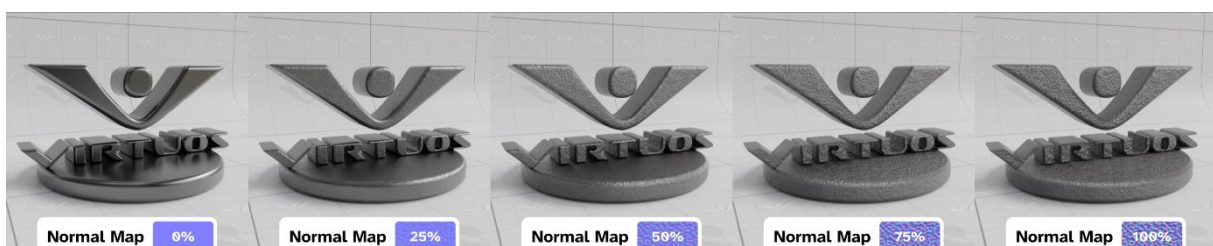


Figure 41 Différents aspects visuels d'un matériau en fonction de l'intensité de sa texture de normales

Les textures qui contrôlent l'orientation de la surface sont appelées Normal Maps d'une surface. Cette direction est un vecteur XYZ stocké généralement dans une texture en couleur en utilisant les canaux

de couleur Rouge, Vert et Bleu. Ces textures permettent d'augmenter artificiellement le niveau de détail des modèles 3D sans devoir augmenter le nombre de polygone. Elles peuvent être créées à partir d'un modèle 3D de haute résolution ou dans un logiciel d'édition d'image.

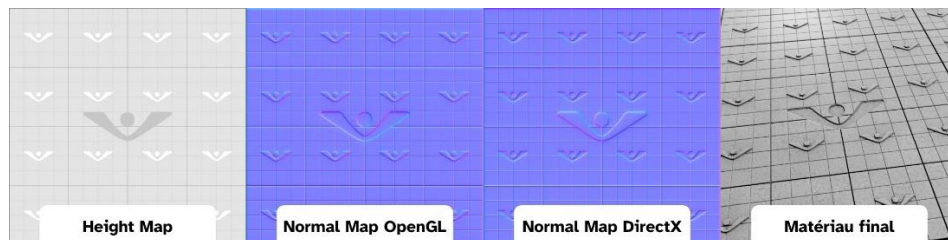


Figure 42 Comparaison de l'apparence des textures de normales au format OpenGL et DirectX en fonction de la hauteur de la surface, le canal vert de la texture de normale est inversé entre le format OpenGL et le format DirectX

Le Subsurface Scattering



Figure 43 Différents aspects visuels d'un matériau en fonction de sa translucidité

Les paramètres de Subsurface Scattering contrôlent l'aspect translucide d'un matériau, ils sont très utiles pour représenter la peau, les feuilles et d'autres matériaux translucides.

L'opacité



Figure 44 Différents aspects visuels d'un matériau en fonction de sa transparence

L'opacité ou alpha définit si un matériau est opaque ou non. On peut choisir comment le matériau réagit à cette transparence grâce à différents paramètres de blending d'alpha (alpha blending). On peut généralement choisir si un matériau peut avoir une transparence classique ou une transparence binaire, c'est-à-dire que chaque partie de la surface ne peut être que complètement opaque ou complètement transparente.

La réfraction



Figure 45 Différents aspects visuels d'un matériau en fonction de son indice de réfraction

L'indice de réfraction ou IOR est une mesure physique qui détermine la vitesse de propagation de la lumière dans un matériau par rapport à sa vitesse dans le vide. Ce paramètre dépend de la densité du matériau et impacte donc à quel point un matériau distord la lumière qui le traverse. En application de la loi de réflexion de *Fresnel*, ce paramètre affecte aussi les réflexions spéculaires, on doit donc prendre en compte cette valeur pour les matériaux opaques également. Pour des raisons de clarté, beaucoup de moteurs de rendu remplacent cet indice dans les matériaux opaques par un paramètre de spécularité qui décrit la spécularité minimum lorsqu'on fait face à cette surface. La grande majorité des matériaux opaques ont un indice de réfraction de 1.5, c'est donc généralement la valeur par défaut dans les logiciels.

Autres paramètres



Figure 46 Comparaison d'un matériau ayant ou non un éclat de tissu

Figure 47 Comparaison du rendu d'un matériau utilisant ou non du clear-coat

Figure 48 Comparaison du rendu d'un matériau anisotropique ou non

Il existe beaucoup d'autres paramètres plus spécifiques qui permettent de créer des matériaux plus distinctifs, par exemple l'émission qui permet à un objet d'émettre de la lumière, l'éclat (sheen) qui donne un aspect de tissu, le clear-coat qui permet de donner un aspect de film transparent et brillant par-dessus la surface comme sur une peinture de voiture et l'anisotropie qui permet aux objets d'avoir des réflexions dépendantes d'une orientation, qui est particulièrement utilisée pour donner l'aspect du métal brossé.

Ces paramètres sont peu utilisés mais peuvent avoir de l'importance pour atteindre un niveau de fidélité graphique très élevé. Il existe d'autres paramètres dans les matériaux PBR qui peuvent servir à

décrire des matériaux très spécifiques, mais les paramètres décrits dans ce document regroupent la grande majorité des paramètres utilisés aujourd'hui.

Les ombres portées

Description

Les ombres portées d'un objet peuvent se définir comme les ombres que cet objet projette sur les objets environnants. On les distingue de l'ombre propre d'un objet qui est une ombre due au fait qu'une surface d'un objet ne fait pas face à une source lumineuse.

Les ombres portées ont une grande importance dans le rendu 3D car elles nous permettent de distinguer les formes et les volumes. Leur présence et leur qualité influent énormément sur la qualité perçue des graphismes des images de synthèses.



Figure 49 Comparaison de l'aspect d'un modèle 3D en fonction d'une absence d'ombres portées, d'une ombre obtenue par rastérisation et d'une ombre obtenue par lancer de rayons.

La comparaison d'ombres ci-dessus fait apparaître le problème de distinction des volumes lorsque les ombres portées sont absentes. On peut également remarquer que les techniques d'ombrages rastérisés peuvent, dans certains cas, se rapprocher grandement d'une simulation complète de la lumière.

L'ombre, la pénombre et l'éclairage direct

Dans le contexte de la simulation des ombres portées, on peut séparer les zones en fonction de trois zones d'éclairage principales : la zone d'ombre, la zone de pénombre et la zone éclairée.

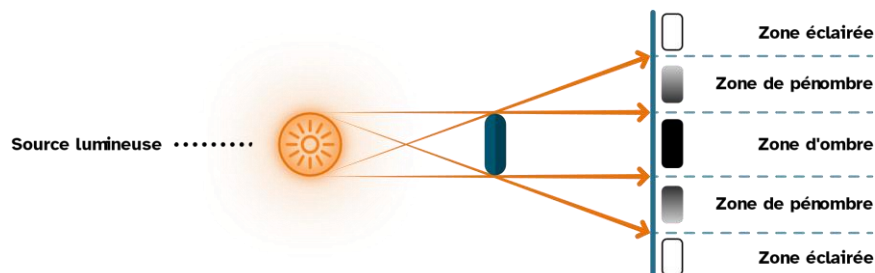


Figure 50 Schéma de la distribution de l'ombre, la pénombre et la zone éclairée d'un objet.

La zone d'ombre est la zone sombre qui ne reçoit aucun éclairage direct d'une source lumineuse. Cette zone est couverte par une surface opaque qui la cache de la source de lumière. Elle peut être considérée comme la projection de la silhouette d'un objet sur un autre du point de vue d'une source lumineuse.

La zone de pénombre est la zone qui comporte un éclairage partiel de la part d'une ou plusieurs sources de lumières. Son niveau d'éclairage à chaque point dépend de la proportion de la source lumineuse visible à ce même point.

La zone éclairée est une zone qui comporte un éclairage direct des sources lumineuses. Elle est la surface la plus proche du point de vue de la source lumineuse. Son niveau d'éclairage dépend de l'intensité des sources lumineuses et de la distance qui la sépare de ces sources.

Les ombres dures et douces

On peut distinguer les ombres dures des ombres douces par leur netteté et leurs contrastes. Les ombres dures passent de l'ombre à la lumière très rapidement et sont plus marquées tandis que les ombres douces ont une zone de transition plus large et sont plus floues. Cette différence est due à la différence de taille de leurs pénombres respectives, les ombres douces ayant une pénombre plus large que les ombres dures.

D'un point de vue physique, on peut calculer la largeur de la zone de pénombre en se basant sur la taille, la distance de la source lumineuse et la distance entre la surface projetant l'ombre et la surface la recevant. Plus la source lumineuse est grande, proche de la surface projetant l'ombre et distante de la surface recevant l'ombre, plus la taille de la zone de pénombre sera grande.

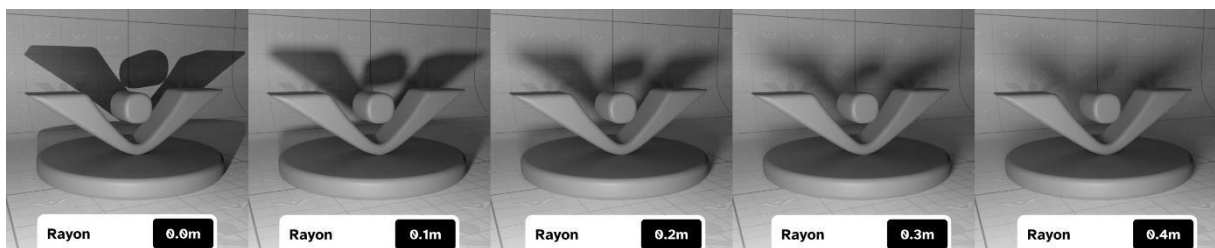


Figure 51 Comparaison des ombres projetées sur une surface en fonction de la taille d'une source lumineuse.

Cette zone se calcule en appliquant le théorème de Thalès en configuration dite papillon, mais dans un espace 3D. Le schéma de la distribution de l'ombre, de la pénombre et de la zone éclairée permet de voir la configuration papillon du théorème de Thalès dans un espace 2D. Pour le calculer dans un espace 3D, on peut projeter les triangles et les points comme des cônes et des silhouettes.

Il existe deux cas précis qui permettent d'obtenir une ombre marquée sans aucune pénombre.

La première est pour une source de lumière infiniment petite comme celle montrée dans le schéma précédent.

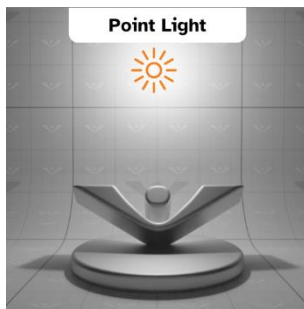
La seconde est pour une source de lumière infiniment éloignée de la surface : comme ses rayons sont tous parallèles, la pénombre est infiniment petite. On peut retrouver une situation s'approchant du second cas dans la nature avec le soleil, bien que cette étoile ait un rayon équatorial de 696342 km, sa distance médiane avec la terre de 150 500 000 km crée un phénomène de rayons quasiment tous parallèles et crée donc une ombre marquée.

Différentes sources de lumière

Bien qu'il existe une infinité de sources de lumières possibles dans le monde réel, le jeu vidéo s'est contenté d'un groupe de sources de lumière assez réduit pour des raisons de performance.

Malgré leurs différents aspects, elles ont dans la majorité des cas des paramètres similaires comme leurs couleurs, l'intensité de leurs rayonnements, leur rayon d'impact, le fait que ces lumières protègent des ombres ou non et d'autres options de direction artistique.

Points lumineux ou Point Light



Les points lumineux simples sont parmi les sources de lumières les plus utilisées dans le jeu vidéo. Ils sont généralement utilisés pour représenter des ampoules et éclairent dans toutes les directions uniformément. Ils sont également souvent utilisés pour augmenter artificiellement la luminosité des environnements en les ajoutant à des environnements sombres pour les éclairer légèrement.

Figure 52 Éclairage d'un point lumineux

On peut parfois leur appliquer des profils IES (*Illuminating Engineering Society*) qui représentent l'illumination de sources de lumières en fonction de leurs angles. Ces profils sont mesurés sur des modèles de lampes réels et permettent d'augmenter le réalisme des lampes dans le jeu vidéo et dans les effets spéciaux.

On peut également souvent contrôler leurs rayons afin de pouvoir contrôler la douceur de leurs pénombres. *Unreal Engine* permet aussi de choisir leur longueurs pour leur donner un aspect de capsules, leur donnant des ombres plus douces dans certaines directions.

Projecteurs ou Spot Light

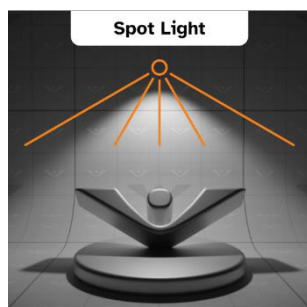


Figure 53 Éclairage d'un projecteur

Les projecteurs sont très similaires aux points lumineux et sont tout autant utilisés dans le jeu vidéo. Ils ont les mêmes propriétés que les points lumineux mais n'éclairent que dans la forme d'un cône de lumière. On peut contrôler l'angle de ce cône et la douceur de sa démarcation entre l'éclairage complet et l'absence d'éclairage.

On peut parfois également utiliser des profils IES comme les points de lumière mais également projeter des textures dans le cône de lumière pour, par exemple, donner des effets de vitraux ou de projecteurs de cinéma.

Lumière directionnelle ou Directional Light

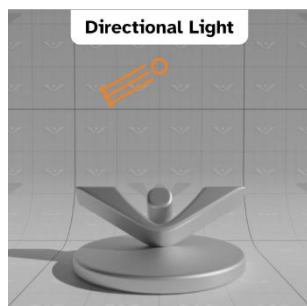


Figure 54 Éclairage d'une lumière directionnelle

Les lumières directionnelles sont généralement utilisées pour représenter le soleil. Elles ont pour particularité d'éclairer uniformément les scènes 3D à la manière du soleil et leur intensité lumineuse n'est pas affectée par leurs distances avec les objets.

Beaucoup de jeux vidéo utilisent une lumière directionnelle comme lumière principale dans leurs niveaux et leur accordent une importance particulière. Par exemple, une technique d'optimisation consiste à garder l'entière des lumières dans un niveau statique à l'exception de la lumière du soleil, afin d'obtenir des ombres dynamiques dans les extérieurs sans devoir calculer l'ombre sur des milliers de sources de lumières.

Lumières de Zones ou Area Light

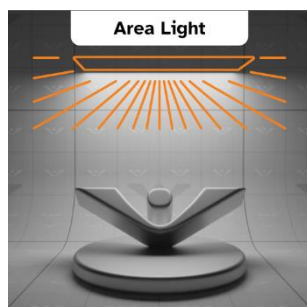


Figure 55 Éclairage d'une lumière de zone

Les lumières de zones sont généralement des lumières rectangulaires ayant une aire paramétrable. Elles peuvent être utilisées comme des projecteurs rectangulaires qui n'illuminent que l'hémisphère dans sa direction. Ces lumières sont utilisées principalement pour éclairer des environnements ou des personnages avec une lumière très douce à la manière des soft-boxes en photographie. Elles peuvent représenter également des lumières rectangulaires industrielles comme des néons d'usines.

Lumières d'environnement ou Skylight



Figure 56 Éclairage d'une lumière environnementale

Les lumières d'environnement représentent l'éclairage diffus créé par l'atmosphère. Ces lumières augmentent la luminosité des scènes de manière à empêcher d'avoir des zones d'ombres complètes dans les environnements. Elles peuvent être créées à partir d'algorithmes simulant le comportement de l'atmosphère ou à l'aide de photographies panoramiques de ciels comme des cube maps ou des High Dynamic Range Images (HDRIs), il s'agit d'images panoramiques de haute fidélité.

Ces lumières entrent également beaucoup en jeu dans le calcul des réflexions spéculaires de l'environnement.

Lumière Globale ou Global Light



Figure 57 Éclairage d'une lumière globale

La forme la plus simple de lumière est la lumière globale. Elle donne une intensité lumineuse globale à la scène. C'est ce type de lumière que les jeux ne calculant pas la lumière utilisent. Tout comme les lumières d'environnement, leur principale utilité est d'empêcher d'afficher des ombres trop sombres.

Ces lumières n'ont aucune équivalence dans le monde réel et sont le type de lumière le moins réaliste. Néanmoins, elles peuvent toujours avoir une grande utilité dans des jeux aux graphismes réalistes quand elles sont discrètes et accompagnées d'autres types de lumières.

Lumière de surface ou Surface Light



Figure 58 Éclairage d'une surface lumineuse

Les lumières de surface sont des sources de lumière émanant des matériaux des modèles 3D présents dans la scène. Elles sont généralement créées à partir de textures et permettent de représenter des sources lumineuses très complexes comme des LEDs ou des écrans de télévision.

Comme calculer leur impact sur l'environnement peut être très lourd, on aura tendance à ne pas le faire et à "tricher" en plaçant d'autres types de lampes à côté de ces surfaces qui auront des couleurs similaires pour simuler l'impact des surfaces sur l'environnement.

Les ombres statiques

Le calcul d'ombre dans le jeu vidéo fait partie des tâches de rendu les plus complexes, mais il existe beaucoup de techniques d'optimisation de ces calculs. Comme il n'est pas toujours possible de calculer le rendu de chaque lumière en temps réel, on peut précalculer le rendu de la lumière. On générera alors les informations d'ombre et de lumière au préalable, que l'on stockera ensuite pour le rendre en temps réel. On appelle cette technique le baking de lumière.

Le baking d'ombres permet aux jeux tournant sur des machines n'ayant pas beaucoup de puissance de calcul d'obtenir des ombres sans réduire drastiquement les performances. Les premières consoles de jeux grand public ou les appareils mobiles d'aujourd'hui comme les smartphones ou les casques de réalité virtuelle n'ont généralement pas la puissance nécessaire pour simuler la lumière en temps réel, il en est de même pour les expériences 3D sur le Web.

Cette technique a pour avantage d'améliorer grandement les performances de calcul de lumières en jeu, au prix d'une absence d'ombres dynamiques. Le processus de baking peut parfois ralentir la production des jeux car la mise à jour des lumières nécessite un temps d'attente en raison du pré-calcul des ombres. Il peut également augmenter la taille des jeux significativement car les informations d'ombres de chaque niveau doivent être stockées dans les niveaux au lieu d'être calculées en temps réel.

Vertex color baking

La manière la plus simple de stocker les informations d'ombres est de les stocker dans la couleur des sommets des polygones des modèles 3D. Il suffira d'effectuer l'interpolation des couleurs des sommets pour chaque pixel de l'écran en fonction de sa distance avec les sommets, puis de multiplier la couleur des matériaux par cette couleur d'ombre.



Figure 59 Création de l'illumination d'une scène du jeu "Kingdom Hearts" grâce à la couleur des sommets des polygones.

C'est la technique la moins coûteuse car elle utilise des informations déjà existantes pour stocker les ombres et lumières. Elle permet également d'obtenir des ombres douces, mais pas d'ombres dures car

la résolution des ombres dépend de la quantité de polygones des modèles 3D. On peut également décider d'éditer manuellement la couleur des sommets afin d'obtenir des résultats plus stylisés comme dans l'image ci-dessus.

Shadow Map

La méthode la plus standard pour enregistrer des ombres utilise des textures que l'on appelle des shadow maps. Il s'agit de textures en noir et blanc dans lesquelles on enregistre les informations d'ombres et de lumières. Pour ce faire, on place tous les polygones d'une scène dans l'espace de cette texture. On évalue ensuite pour chaque pixel l'intensité lumineuse de la partie du polygone qui y est placée. Enfin, on multiplie la couleur de chaque pixel du polygone par la texture d'ombre pour donner l'illumination finale.

Cette méthode nécessite de créer des UV maps secondaires pour chaque modèle 3D afin de pouvoir le projeter sur la texture d'ombrage. Il est possible d'effectuer ce processus manuellement ou de l'automatiser.

L'algorithme utilisé pour générer les textures d'ombres est appelé un light mapper. Chaque moteur supportant le baking des ombres a son propre light mapper supportant différentes fonctionnalités.

A titre d'exemple, le light mapper du jeu *Quake* fonctionne de la manière suivante : on vérifie pour chaque pixel de la texture d'ombre si le chemin entre ce pixel et la lumière est bloqué en utilisant un lancer de rayon appelé Ray Cast. Si c'est le cas, alors le pixel est à l'ombre, sinon, le pixel est illuminé. On calcule ensuite l'intensité de l'illumination du pixel en l'atténuant linéairement avec sa distance de la source de lumière et en prenant en compte l'angle d'incidence de la lumière. On répète ces opérations pour chaque lampe proche de la surface.

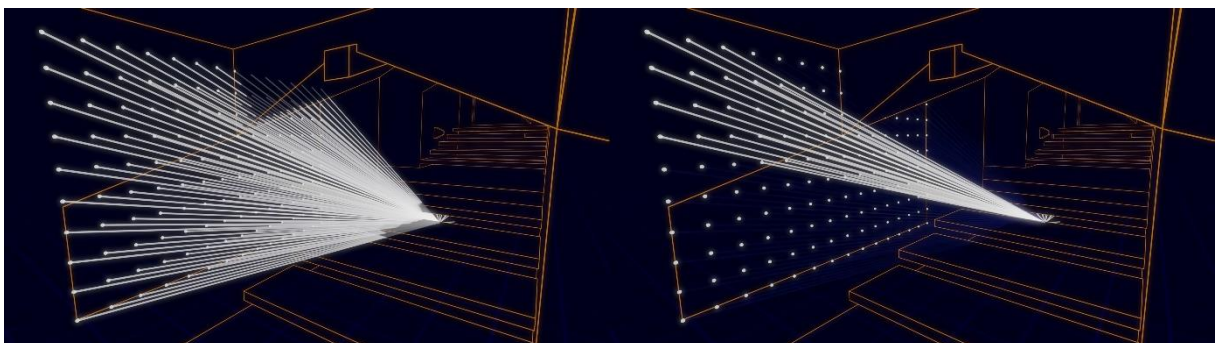


Figure 60 Schéma de la détection de l'illumination par Ray Casting dans le moteur de *Quake*, les rayons bloqués par la géométrie sont considérés à l'ombre.

Ce fonctionnement crée une démarcation binaire entre les pixels éclairés et l'ombre. Pour remédier à ce problème, le light mapper de *Quake* permet d'utiliser quatre rayons pour la vérification de l'illumination et d'effectuer une moyenne pour adoucir les ombres. Cette méthode porte le nom d'Anti-Aliasing.

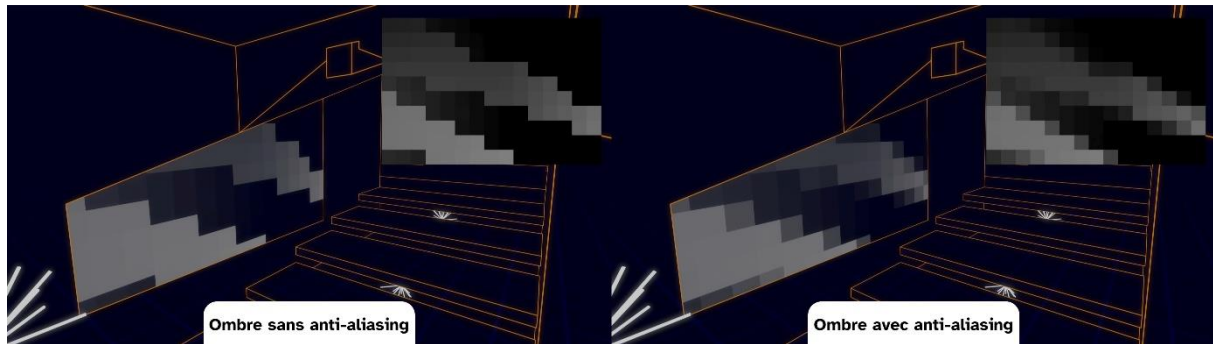


Figure 61 Comparaison de l'ombre calculée sans et avec Anti-Aliasing.

L'algorithme du light mapper de *Quake* a un fonctionnement assez basique dans son calcul de lumière, mais il est extrêmement rapide pour un jeu datant de 1996, grâce à différentes techniques d'optimisation du moteur.

Pour *Quake 2*, le light mapper utilise l'algorithme Radiosity, qui permet de calculer les rebonds de la lumière que l'on appelle le phénomène d'illumination globale. *Quake 3 Arena* n'a pas utilisé Radiance pour son light mapper, le choix a été fait d'utiliser un light mapper ne supportant pas l'illumination globale pour des raisons de direction artistique. Malgré cela, le light mapper de *Quake 2* a été utilisé dans le jeu *Half Life 2* et ses suites.

Aujourd'hui, la majorité des light mappers permet de calculer l'illumination globale comme dans *Quake 2*. A titre d'exemple, *Unity*, *Unreal Engine* ou encore *Godot Engine* et bien d'autres moteurs de jeu supportent cette fonctionnalité. Grâce à un processus de lancer de rayon, on peut obtenir des ombres réalistes ayant des zones de pénombres correctes.

Les ombres dynamiques

Le calcul d'ombres en temps réel a permis aux jeux vidéo d'augmenter significativement la qualité de ses graphismes. Les ombres dynamiques permettent non seulement d'augmenter l'immersion des joueurs et joueuses mais sont aussi très importantes pour le gameplay. Par exemple, l'ombre projetée d'un personnage permet de jauger sa position dans l'environnement et de pouvoir changer l'environnement lumineux d'une scène, ce qui donne une réelle opportunité aux développeurs de guider les joueurs.

Premières méthodes

Les premières consoles de jeu et les machines d'arcades de la transition des jeux vidéo vers la 3D n'avaient pas la puissance nécessaire pour calculer fidèlement des ombres dynamiques. Pour pallier cela, les développeurs ont créé différentes techniques pour donner l'illusion d'ombres dynamiques dans les niveaux de jeu.

Projection de textures



Figure 62 Ombre circulaire projetée sous Mario dans "Mario 64"

L'une des techniques les plus simples pour donner l'illusion d'une ombre dynamique est de placer un rectangle sombre semis transparent au niveau des pieds des personnages. Sur ce polygone, on appliquera une texture décrivant la forme de l'ombre, souvent un cercle. Cette technique a été utilisée par le jeu "Mario 64". Projeter directement une ombre sous le personnage malgré l'orientation des sources de lumière est d'ailleurs très utile pour les jeux de plateformes, cette ombre permettant de localiser la position du personnage même en plein saut. Pour cette raison, tous les jeux de plateforme 3D de la série *Mario* utilisent cette technique.

On utilise encore cette technique dans des jeux plus récents comme *Minecraft*.

Dynamic Shadow Map

Aujourd'hui, la majorité des jeux modernes comporte une forme d'ombre dynamique basée sur des shadow maps dynamiques. Il s'agit du même type de texture que pour les ombres statiques, mais on les recalcule en temps réel.

Pour ce faire, le moteur de rendu calcule la distance des sources de lumière par rapport aux surfaces de la scène et enregistre ces distances dans des textures. Le moteur vérifie ensuite pour chaque pixel si la distance de ce pixel avec la lampe est plus grande que la distance enregistrée sur la texture. Si c'est le cas, alors la surface de ce pixel est à l'ombre.

Cette technique est très efficace à calculer avec les ordinateurs d'aujourd'hui mais pose plusieurs problèmes.

En premier lieu, l'ombre est binaire et ne comporte pas de pénombre. Pour régler cela, on peut utiliser un filtrage comme l'algorithme Percentage Closer Filtering (PCF) qui vérifie une grille pixels de la texture de la lampe pour adoucir l'ombre en fonction du pourcentage de pixels éclairés. Cette

technique peut être améliorée en ajoutant un léger décalage aléatoire de chaque élément de la grille afin de rendre le résultat plus naturel.

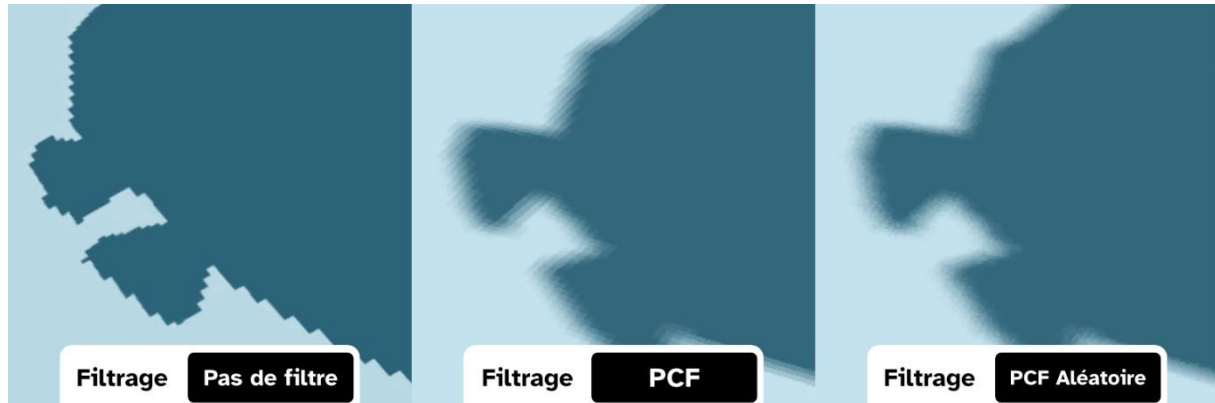


Figure 63 Comparaison d'une shadow map sans filtrage, avec filtrage PCF et avec filtrage PCF aléatoire

En second lieu, cette technique entraîne un problème de résolution des textures d'ombres. En effet, utiliser des textures avec des résolutions trop grandes consomme beaucoup de puissance de calcul et de mémoire, mais utiliser des textures d'ombres de trop basse résolution fait apparaître une ombre floue et pixélisée. Il existe beaucoup de méthodes pour pallier la pixellisation des ombres, en voici quelques-unes.

Shadow Cascade

Les cascades de textures d'ombres (shadow cascade) sont une technique qui sépare la scène 3D en plusieurs niveaux de détails en fonction de la distance avec la caméra. C'est actuellement la technique la plus utilisée pour optimiser les ombres dynamiques. Elle est utilisée dans tous les moteurs de jeux 3D les plus populaires comme *Unreal Engine*, *Unity*, *Cry Engine*, *Godot* ou encore *Open 3D Engine*.

Cette technique consiste à utiliser plusieurs textures pour les objets proches, à moyenne distance et à longue distance. Elle permet de ne pas gaspiller d'espaces de textures pour des objets petits et lointains à l'écran. On peut également choisir la vitesse de mise à jour des ombres en fonction de la distance pour gagner en performance.

La difficulté majeure pour l'implémentation est de gérer la zone de transition entre le niveau de détails, il est recommandé de rendre cette transition plus douce pour éviter d'avoir des artéfacts à l'écran.

Virtual Shadow Map

Bien que les cascades de textures d'ombres (cascaded shadow maps) permettent l'augmentation significative du niveau de détail des ombres, elles peuvent encore manquer de détail. Cette technique

utilise par ailleurs la même quantité de détails pour la zone de transition de la pénombre que des zones éclairées ou complètement à l'ombre, ce qui crée une inefficacité d'utilisation de la mémoire.

Une solution permettant de régler ce problème consiste en l'utilisation des textures d'ombres virtuelles (virtual shadow maps). Il s'agit d'une technique permettant d'augmenter significativement la qualité des textures en virtualisant les textures d'ombres.

Unreal Engine utilise cette technique en conjonction avec son système Nvite pour augmenter grandement la quantité de détails possibles dans son moteur de rendu. Toutes les ombres sont stockées dans un atlas de très haute résolution 16000px*16000px par exemple, mais seules les zones visibles à l'écran sont chargées en blocs de 120px*128px. Seules les zones de pénombres nécessitent des blocs de textures uniques car les grandes zones d'ombres ou éclairées sont uniformes et peuvent donc utiliser les mêmes blocs. Les ombres sont enregistrées et sont mises en cache, seuls les blocs ayant un changement de lumière sont recalculés, ce qui permet de baisser significativement le calcul de lumière car il est localisé autour des zones d'objets en mouvements.

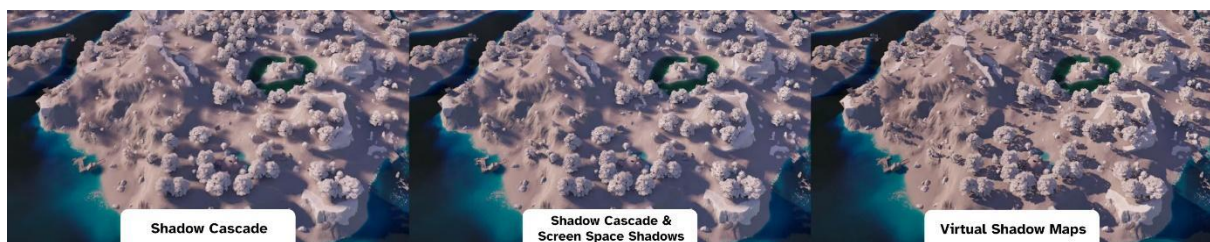


Figure 64 Comparaison des cascaded shadow maps et des virtual shadow maps dans Fortnite.

Pour obtenir une pénombre plausible, *Unreal Engine* utilise la technique du Shadow Map Ray Tracing (SMRT), qui consiste à effectuer un lancer de rayon vers la source de lumière, mais au lieu de tester des collisions avec la géométrie environnante, le test est effectué sur la texture d'ombre virtuelle. La distribution de ces rayons est calculée en fonction du rayon de la source de lumière, ce qui permet d'obtenir une pénombre de taille variable et réaliste.

Shadow Volume

Bien que les ombres à base de textures soient la technique la plus utilisée dans le jeu vidéo aujourd'hui, il existe également une autre technique qui a été popularisée en 2004 par le jeu *Doom 3* : les Shadow Volumes.

Les shadow volumes fonctionnent de la manière suivante : pour chaque lampe projetant une ombre, on calcule la silhouette des objets et on les extrude/projette vers l'infini dans la direction opposée à la

lampe. Ces volumes relativement coniques représentent une zone assombrie. Pour chaque pixel de l'écran, on calcule le nombre d'intersections avec des volumes d'ombres. Pour cela, on crée une image tampon qui stocke le nombre d'intersections. On commence par lancer un rayon entre la caméra et la surface visible, pour chaque intersection rentrant dans un volume on ajoute 1 et pour chaque intersection sortant d'un volume on retire 1. Tous les pixels de l'image tampon différents de 0 sont à l'ombre et les autres pixels sont éclairés.

On a donc l'information exacte des zones d'ombre pour chaque pixel de l'écran, ce qui est très difficile à mettre en œuvre avec des techniques plus classiques. Cela permet ainsi d'obtenir des ombres très nettes dans des scènes complexes, de manière peu coûteuse en mémoire par rapport à des solutions à base de shadow maps si l'on cherche le même niveau de netteté.



Figure 65 Unreal Tournament 2004 utilisant un ombrage classique



Figure 66 Doom 3 (2004) utilisant des shadow volumes pour son ombrage

ID Software a popularisé les Shadow Volumes en 2004 avec *Doom*. Toutefois, en se penchant sur les productions antérieures du studio, on remarque que l'idée d'utiliser des modèles 3D pour définir les ombres date de 1999 dans le jeu *Quake 3 Arena*. En effet, le jeu utilise une technique appelée Stencil Shadow qui fonctionne de manière très similaire mais n'utilise pas de lancer de rayons pour obtenir l'information d'ombre. On peut donc observer que le studio fondé par *John Carmack* a utilisé des techniques d'ombrages ayant un détail à l'échelle des pixels de l'écran 10 ans avant que les shadow maps puissent atteindre un niveau de détail comparable.

Bien que les shadow volumes permettent d'afficher des ombres très nettes, ils ont un désavantage majeur par rapport aux autres techniques d'ombrage. En effet, cette technique crée une absence totale de zones de pénombres et d'ombres douces. Ce compromis n'était pas un problème pour le jeu *Doom 3* ayant une esthétique avec des lumières très marquées, mais l'avancée de la puissance du

matériel de rendu 3D l'a rendu relativement obsolète quand la taille de textures des shadow maps a été grandement améliorée dans les années 2010, grâce à l'augmentation de la quantité de mémoire graphique dans les machines. Les shadow volumes sont aujourd'hui utilisés plus sporadiquement pour des utilisations spécifiques comme sur certains casques en réalité virtuelle, mais peuvent encore être utiles pour des jeux ayant un ombrage simple ou très marqué.

Screen Space Shadow

Aujourd'hui, les ombrages à base de shadow maps ont atteint un niveau de détail très élevé mais qui permet rarement d'afficher des ombres détaillées sur les micro-détails de l'environnement et des personnages. Pour atteindre ce niveau de détail, on peut utiliser une technique appelée screen space shadows, qui permet de calculer une approximation des zones d'ombre sur une très petite échelle. Elle ne permet pas de calculer l'ombrage d'une scène mais permet d'obtenir des ombres nettes sur les détails des modèles 3D.

Pour cette raison, cette technique fonctionne très bien en tandem avec les shadow maps, puisqu'ils fournissent les détails médium et larges et l'ombre d'espace d'écran fait apparaître les micro-détails de la scène.

Cette technique utilise le ray marching, un algorithme qui effectue un lancer de rayon en plusieurs étapes. Pour chaque pixel, on envoie un rayon dans la direction d'une source de lumière, on va faire avancer ce rayon petit à petit et calculer sa position dans l'espace de l'écran. À chaque avancée, on calcule la distance du rayon par rapport à la caméra et on la compare avec la distance de l'environnement au même pixel. Si l'environnement est plus proche que le point de fin du rayon, alors on en déduit que la surface d'origine est à l'ombre.

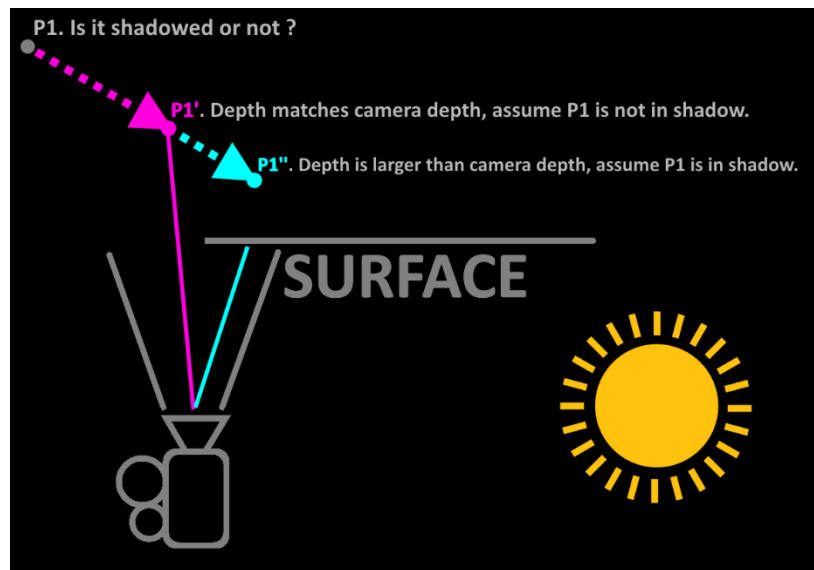


Figure 67 Schéma de l'algorithme de calcul des screen space shadows

Cette technique manque de précision pour des grandes distances et ne permet pas d'obtenir de zones de pénombre mais ces deux points négatifs n'affectent pas réellement son apparence. Comme elle est généralement utilisée pour des petits détails à l'écran, son manque de précision n'est pas visible et à cette échelle, une zone de pénombre réaliste serait invisible sur l'écran.

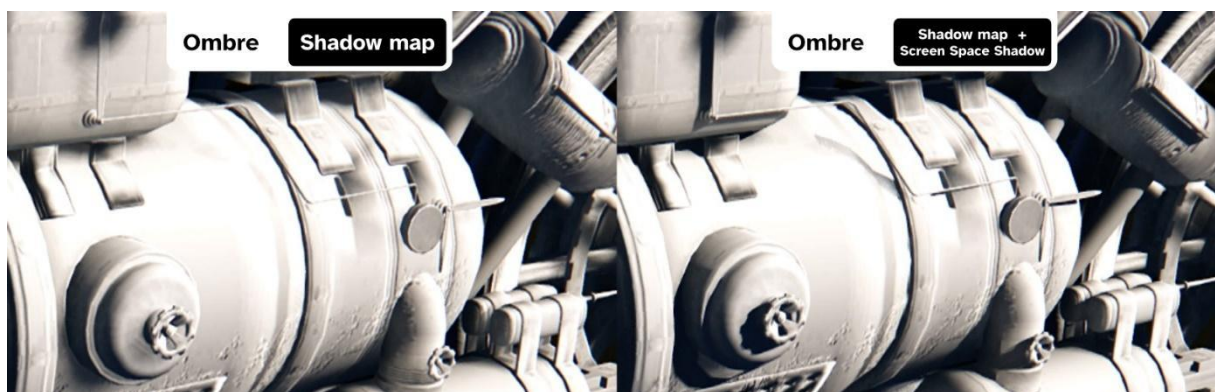


Figure 68 Comparaison des détails d'une ombre issue de shadow map avec et sans ajout de screen space shadows

Pour ces raisons, on utilise très souvent cette technique sous le nom d'ombres de contact (Contact Shadows). Le studio de jeu vidéo *Remedy Entertainment* l'utilise par exemple dans le jeu "*Quantum Break*" et dans le jeu "*Control*" avec une multitude d'autres effets pour obtenir des ombres de très grande qualité.



Figure 69 Implémentation des screen space shadows dans les jeux "Quantum Break" et "Control" du studio Remedy Entertainment

Unreal Engine utilise les contact shadows pour ajouter des détails à l'ombre quand les techniques de virtual texturing ne peuvent pas être appliquées à des environnements. De plus, le système de matériaux s'intègre à cette technique pour ajouter des ombres au matériaux utilisant du parallax occlusion mapping.

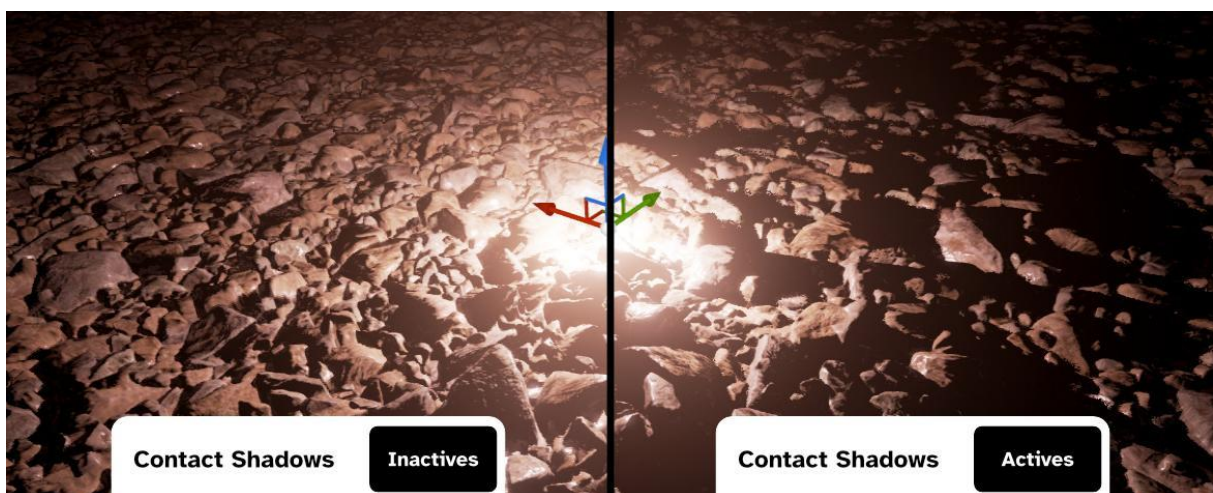


Figure 70 Image comparative des matériaux utilisant du parallax occlusion mapping avec et sans contact shadows

Nvidia Reservoir-based Spatio-Temporal Importance Resampling (ReSTIR)

Le fabricant de carte graphique *Nvidia* est un acteur qui pousse l'innovation dans le rendu 3D au moyen de matériels spécialisés et en publiant beaucoup d'algorithmes d'optimisation du rendu de la lumière. L'entreprise en partenariat avec le *Dartmouth College* a publié une méthode de rendu de l'éclairage direct au Siggraph 2020 appelée Reservoir-based Spatio-Temporal Importance Resampling (ReSTIR). Cette technique de rendu des ombres basée sur le lancer de rayons permet d'afficher des ombres dynamiques et réalistes pour des scènes comportant plusieurs millions de sources de lumières, y compris des modèles 3D émettant de la lumière.



Figure 71 Image d'une scène rendue avec ReSTIR, chaque élément lumineux de la scène est comptabilisé dans les calculs d'illumination directe.

Elle consiste en une optimisation des algorithmes de lancer de rayon classique en stockant les informations dans le temps et en réutilisant ces informations ainsi que les informations de lumière physiquement proches pour réduire la quantité de grain à l'image. Ces informations permettent également d'obtenir la direction des sources lumineuses importantes, afin de concentrer le lancer de rayon sur celle-ci pour ne pas gaspiller de rayons dans des zones ayant peu d'effet sur l'illumination des objets. Ces différentes techniques d'optimisation permettent à l'algorithme d'obtenir des images de qualité similaire à des techniques plus classiques de lancer de rayon en six à soixante-cinq fois moins de temps.

Bien que cette technique augmente significativement la vitesse de rendu de la lumière par lancer de rayon, elle ne permet pas d'obtenir des images sans grain dans des temps assez courts pour être interactifs. Pour résoudre ce problème, *Nvidia* utilise son algorithme Optix denoiser, permettant le débruitage par machine learning développé par la compagnie, créé dans le but de faire disparaître le grain dans les images pour des applications en temps réel. Ces deux algorithmes permettent de rendre

une scène contenant 11 000 sources lumineuses en 16ms, soit 60 images par seconde et de rendre une scène contenant 3 400 000 sources lumineuses en 50 ms, soit 20 images par seconde.



Figure 72 Comparatif par Nvidia de la qualité de rendu d'image de différents algorithmes de lancer de rayons avec et sans débruitage par l'Optix Denoiser

Ce gain de vitesse substantiel permet de rendre des scènes extrêmement complexes en temps réel tout en simulant de manière réaliste le comportement de la lumière, son ombre et sa pénombre correctement.

La lumière environnementale

Description

La lumière environnementale englobe les différents phénomènes dus aux rebonds multiples de la lumière sur la surface des objets. Ces multiples rebonds de la lumière ont un impact très fort sur l'apparence des objets et permettent d'augmenter grandement la cohérence physique des scènes 3D dans le jeu vidéo. Parmi tous ces phénomènes, il en existe trois majeurs qui sont simulés dans le jeu vidéo.

La réflexion environnementale est le processus de calcul des reflets spéculaires de l'environnement sur les modèles 3D qui le composent. Ces reflets sont très présents sur les objets brillants, les vitres ou même les miroirs qui ont la particularité de refléter la majorité de la lumière.

L'illumination globale (GI) détermine l'illumination des surfaces en prenant en compte la lumière qui a subi des réflexions et des réfractions sur d'autres surfaces avant d'atteindre la surface observée. La lumière qui rebondit sur les murs et les objets dans une pièce et fini par éclairer une zone ombragée est un exemple d'illumination globale.

L'occlusion ambiante est, quant à elle, un phénomène qui est décrit par l'apparition d'ombres à l'endroit de contact entre deux surfaces. La lumière a tendance à se faire emprisonner dans des zones creuses, elle rebondit dans les zones concaves et se fait absorber par la surface. Pour cette raison, les zones creuses sont assombries. L'espace entre un meuble et un mur ou les pieds d'une table au contact du sol présentent par exemple une ombre qui est due à l'occlusion ambiante.

La réflexion environnementale

Description

Les réflexions environnementales sont les réflexions spéculaires des objets sur les autres objets dans une scène. Elles englobent tous les reflets complexes que l'on peut trouver dans le monde réel. Le calcul des réflexions environnementales réalistes est un processus complexe et chronophage pour le matériel de rendu, on cherche donc souvent à effectuer une approximation grossière des reflets afin de tromper l'œil des joueurs et joueuses. Pour créer ces illusions de reflets, il existe beaucoup de techniques plus ou moins complexes et plus ou moins convaincantes.

Réflexions standards

CubeMaps

Les cubemaps sont des textures panoramiques qui représentent l'environnement visible d'un point de vue. Grâce à ce type de texture, on peut simuler un reflet pour un objet proche du point de vue de la cube map.



Figure 73 Réflexions de d'un environnement sur un des modèles 3D avec cube map qui correspond à l'environnement reflété

Ces textures peuvent être de plusieurs formes. Les cube maps classiques sont composées de six images individuelles qui représentent les six faces d'un cube, d'où le nom de la texture. Les cube maps modernes utilisent maintenant une seule image panoramique à 360°, similaire à celle que l'on peut prendre avec un téléphone. Pour ces deux types d'images, on obtient la couleur de la réflexion à chaque pixel de l'écran en calculant l'angle reflété de la vue par rapport à l'orientation de la surface d'un modèle 3D. Avec cette direction, on peut échantillonner la couleur de la réflexion et afficher la réflexion sur le modèle 3D. Pour afficher des réflexions sur des surfaces plus ou moins rugueuses, il faut pouvoir flouter ces réflexions. Pour cela, il existe plusieurs techniques, la plus simple consistant à échantillonner les différents mipmaps de la texture. Il s'agit donc de dupliquer des textures qui ont une résolution bien moins élevée. En utilisant des mipmaps plus élevées, on perd en résolution et en détail, ce qui donne une apparence de flou.

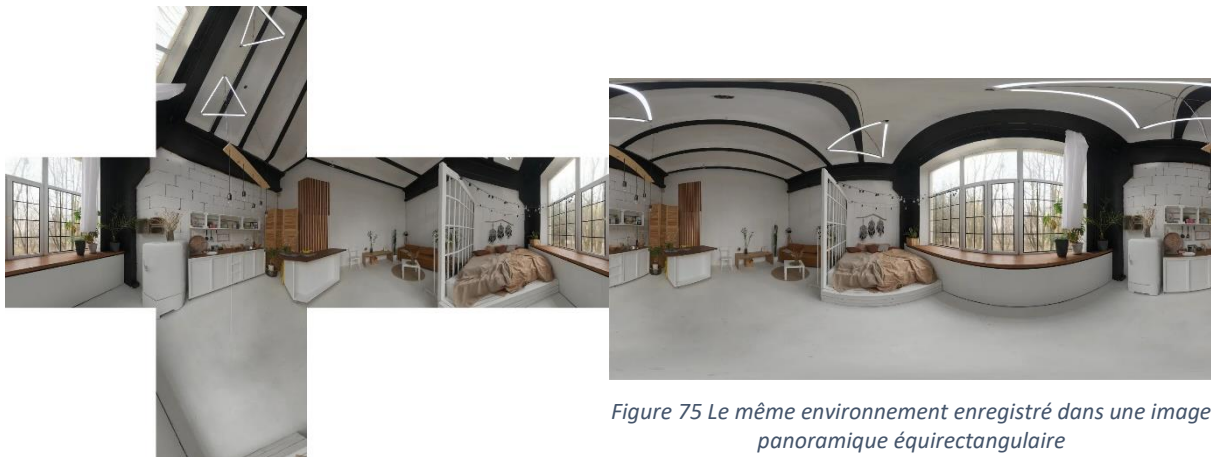


Figure 74 Environnement enregistré dans une cube map

Figure 75 Le même environnement enregistré dans une image panoramique équirectangulaire

Pour créer des cube maps, il existe deux techniques principales. La première consiste à les créer à la main en utilisant des logiciels de traitement photo. Cela permet de créer des réflexions stylisées ou plus détaillées que l'environnement réel de la scène mais cela peut être chronophage si l'on veut obtenir des réflexions de la scène, car il faut effectuer des captures d'écran sous différents angles et les éditer.

La seconde technique consiste à placer des objets dans la scène que l'on appelle des reflection probe dans *Unity* et *Godot* ou des reflection captures dans *Unreal Engine*. Ces objets permettent de générer automatiquement des réflexions et de les appliquer dans un volume choisi. Elles peuvent avoir une forme sphérique ou rectangulaire en fonction de la géométrie des scènes, ce qui permet d'obtenir des réflexions plus réalistes.

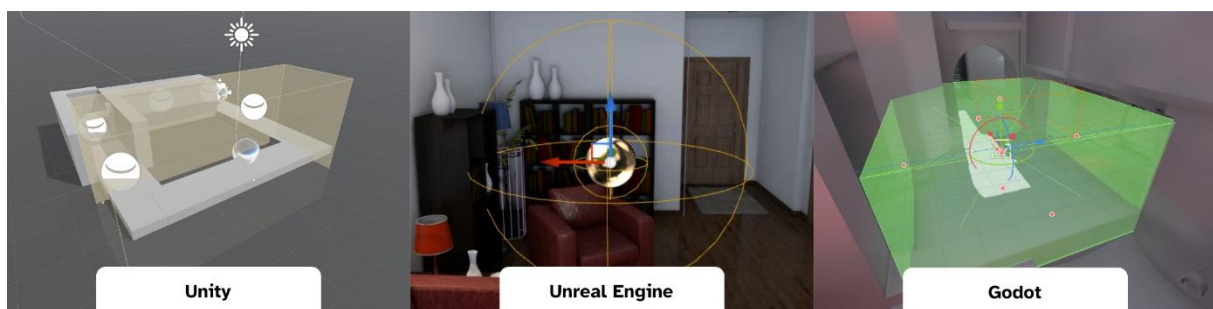


Figure 76 Comparaison des réflexions probes/captures dans le moteur Unity, le moteur Unreal Engine et le moteur Godot

Les cube maps ont l'avantage d'être très peu coûteuses en performance graphique et d'augmenter significativement la qualité des réflexions dans le rendu 3D. Elles ont malgré tout quelques désavantages. Premièrement il faut soit les créer à la main, soit placer des réflexions probes partout dans les scènes pour obtenir des réflexions convaincantes. Deuxièmement, comme les réflexions des

cube maps sont calculées du point de vue du probe, tous les objets reflètent l'environnement comme s'ils étaient placés au centre des cube maps. Ceci n'est pas un problème pour des surfaces peu réfléchissantes mais cela devient très visible pour les surfaces très réfléchissantes. Pour résoudre ce problème, on peut distordre la réflexion en prenant en compte la taille du volume de la cube map pour corriger la perspective, on parle alors de parallax corrected cube maps. Ces cube maps sont présentes dans la majorité des moteurs et permettent de réduire l'artéfact visuel des réflexions.



Figure 77 Comparaison d'une réflexion utilisant une cube map classique et une cube map ayant sa perspective corrigée dans le moteur Source, la réflexion est bien plus convaincante quand sa perspective est correcte

Par défaut, les réflexions probes sont statiques car la création des cube maps est coûteuse en ressources comme il faut effectuer le rendu 3D à 360° du point de vue des réflexions probes. Malgré cela, il existe des réflexions probes dynamiques qui permettent d'obtenir un reflet qui est mis à jour en temps réel. Ces cube maps peuvent être très lourdes à calculer mais peuvent être utiles dans les situations où les reflets sont très importants pour l'immersion et que l'environnement change drastiquement, par exemple les jeux de courses automobiles.

Screen Space Reflections

On appelle Screen Space Reflection une technique de rendu des reflets qui se base sur les informations de rendu présentes dans l'image déjà rendue. Tout comme la technique des screen space shadow, il s'agit d'un effet que l'on applique à la fin du rendu de l'image (post processing effect) qui consiste à effectuer du ray marching dans l'espace de l'écran. Pour ce faire, l'algorithme a besoin de trois textures de la scène rendue que l'on appelle des G Buffers : la texture de profondeur de la scène, la texture des normales (orientation des surfaces) de la scène des surfaces et la texture du rendu de la scène classique.



Figure 78 Utilisation des G buffers de profondeur, de normales et de rendu classique de la scène pour générer un G buffer de réflexion suivi de l'ajout de ce buffer au rendu de base pour obtenir une image finale.

L'algorithme fonctionne de la manière suivante : Pour chaque pixel de l'écran, on lance un rayon vers la surface que l'on reflète en utilisant la carte des normales, on va ensuite parcourir la direction du rayon réflétié petit à petit en vérifiant la profondeur de la scène au niveau du pixel où se termine le rayon. Si la profondeur de la scène est la même que celle du rayon réflétié, alors on obtient la couleur du reflet, sinon on continue dans la même direction. Tout comme pour le cube maps, on peut utiliser les mipmaps de la couleur de la scène pour obtenir des réflexions floues sur les surfaces rugueuses.

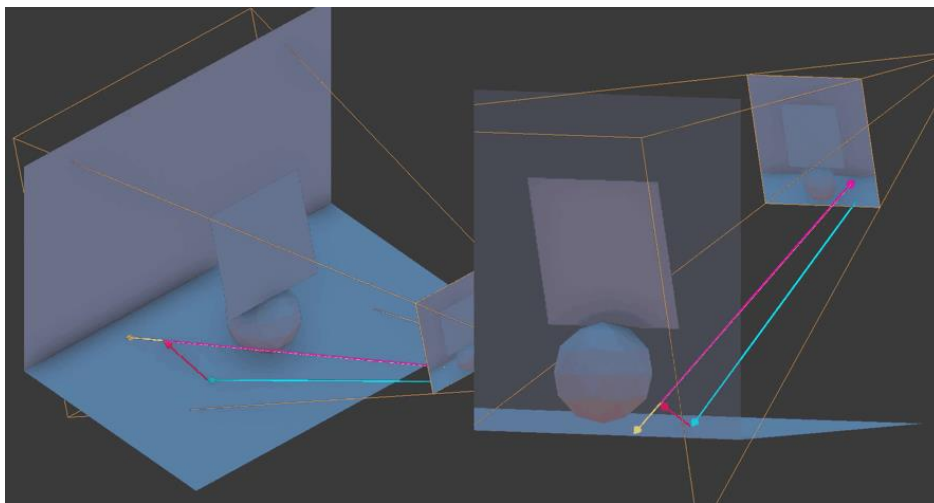


Figure 79 Schéma de l'algorithme de calcul des Screen Space Reflection

Cette technique permet d'obtenir des réflexions détaillées et dynamiques pour les objets présents à l'écran mais elle présente beaucoup d'artéfacts. Si la réflexion est dans la direction générale de l'observateur, alors il y a de fortes chances que la couleur reflétée soit celle d'une surface non visible par la caméra. La couleur présente sur l'image de rendue est alors incorrecte, il faut donc généralement ne pas prendre en compte les réflexions des surfaces pointant vers la caméra. De plus, comme on utilise des informations rendues à l'écran, l'algorithme ne dispose pas des informations de couleurs qui sont hors champ, il ne peut donc pas refléter des objets hors du champ de vision de la caméra.

L'algorithme des screen space reflections est une technique permettant d'obtenir des réflexions très détaillées localement mais qui peuvent manquer de stabilité et créer des artefacts visibles et des trous dans la réflexion. Dans les cas où cette technique ne permet pas d'obtenir des réflexions convaincantes, on peut alors l'utiliser en la combinant avec des cube maps pour ajouter les détails à plus grande échelle. En combinant ces deux techniques, on peut obtenir des réflexions très convaincantes tout en gardant des performances convenables.

Ray Tracing

Bien que la simulation de lumière par lancer de rayon soit algorithmiquement simple et maîtrisée depuis 1979 avec le film *Compleat Angler* des laboratoires de Bell, l'utilisation de cette technique pour les applications en temps réel est encore aujourd'hui très restrictive.

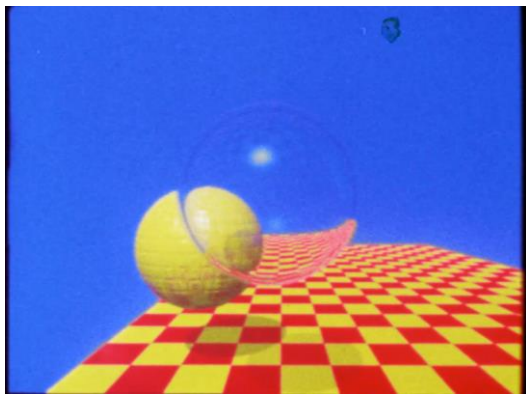


Figure 80 Image du film *Compleat Angler* par Turner Whitted en 1979 ayant un rendu prenant un temps considérable pour chaque image



Figure 81 Image de la cinématique *Reflections* en 2020 ayant un rendu en 20 ms en utilisant une Nvidia RTX 2080Ti

Pour atteindre des performances de l'ordre de 16 ms pour rendre une image complète sur des machines grand public, il a fallu attendre la sortie de nouvelles générations de cartes graphiques ayant des structures d'accélération de lancer de rayon avec la série *Nvidia GeForce RTX 2000* sortie en 2018 et la série *Amd Radeon RX 6000* sortie en 2020. Les premières consoles de jeu à supporter cette technologie en temps réel sont la *PS5* et les *Xbox Series X* et *S*, toutes les trois sorties en 2020 également.

Cette technologie permet d'effectuer des lancers de rayon depuis la caméra du jeu et de les faire rebondir sur les surfaces des modèles 3D pour obtenir des réflexions réalistes. Elle permet également de simuler les autres aspects de la lumière environnementale comme l'illumination globale. Bien qu'elle permette d'obtenir des réflexions très réalistes, son impact sur les performances des jeux est très élevé et requiert du matériel spécialisé pour pouvoir l'utiliser. Pour ces raisons, elle est

généralement optionnelle est réservée à des jeux à très haute-fidélité graphique, nécessitant du matériel de dernière génération pour tourner.



Figure 82 Image de la réflexion sur l'eau en utilisant des Screen Space Reflections dans le jeu Cyberpunk 2077

Figure 83 Image de la réflexion sur l'eau en utilisant du Ray Tracing dans le jeu Cyberpunk 2077

Miroirs

Les miroirs sont des surfaces réfléchissantes et planes, deux propriétés qui les rendent spéciales dans le contexte du rendu 3D en temps réel. En effet, il est plus facile de donner l'illusion d'une réflexion planaire que de simuler des réflexions sur des surfaces arbitraires mais cela peut être très lourd de calculer des réflexions de miroirs car il faut dupliquer le nombre de points de vue à rendre à l'écran. Pour ces raisons, les miroirs simples existent dans le jeu vidéo depuis plus longtemps que les autres types de réflexions et les miroirs complexes sont encore très rares.

Ray Casting 2.5D

Les premières instances de miroirs dans des jeux en perspective 3D ont débuté avec les moteurs de ray casting. En effet, ces moteurs effectuent un lancer de rayon en 2D dans l'espace de la scène pour chaque colonne de pixel et effectuent le rendu 3D en fonction de la distance parcourue par ce même rayon. Comme le lancer de rayon est effectué en 2D et qu'il ne faut un seul rayon pour afficher une colonne entière de pixel de l'écran, il est possible d'effectuer une réflexion sur les murs comportant un matériau de miroir sans augmenter drastiquement le temps de rendu 3D. Cette solution a été implémentée en premier dans le jeu *Duke Nukem 3D*.



Figure 84 Image d'un miroir capturée dans le jeu Duke Nukem 3D

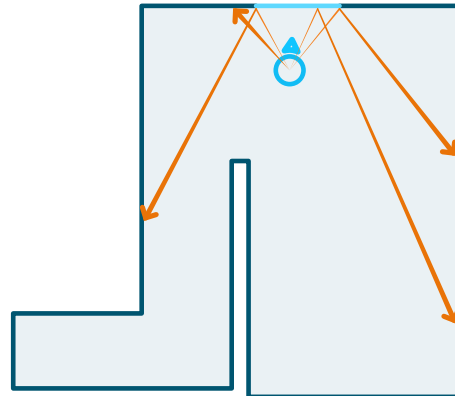


Figure 85 Schéma du ray casting 2D dans la géométrie de la scène de l'image précédente, le moteur lance un rayon par colonne de pixel de l'écran pour afficher la scène.

Duplication de géométrie

La technique la plus simple pour donner l'illusion d'un miroir ou d'une réflexion planaire est de dupliquer entièrement une salle, de l'inverser dans le sens du miroir et de la placer derrière ledit miroir. Cette technique ne nécessite aucun algorithme complexe et est extrêmement rapide à rendre.



Figure 86 Faux reflet de miroir dans la salle au miroir dans le jeu Mario 64



Figure 87 Salle en réalité dupliquée avec une surface semi transparente

On peut également changer l'apparence de ce type de miroir très facilement. Pour tinter la réflexion, il suffit de placer un objet semi transparent au niveau du miroir. Pour refléter les personnages, il suffit de les dupliquer, de les inverser et de les déplacer en même temps que les personnages originaux. Ces deux techniques ont été utilisées dans le jeu *Mario 64* et dans le jeu *Wii Sport* pour l'épreuve de Bowling.

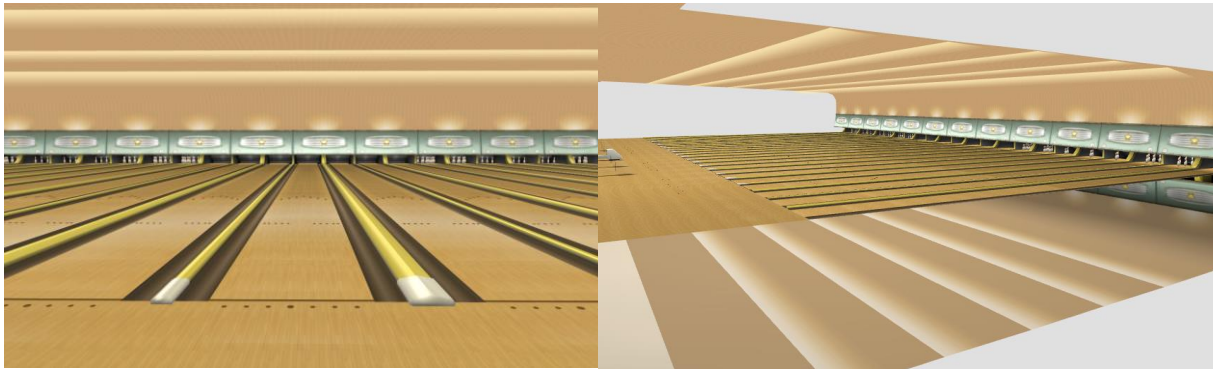


Figure 88 Faux reflets du plafond sur le bois laqué de la piste de bowling dans Wii Sport

Figure 89 Salle en réalité symétrique sur sa hauteur et ayant un sol semi transparent pour faire apparaître les reflets.

On peut également tordre la réflexion pour donner l'apparence d'un miroir déformant ou de la surface de l'eau. Si la surface est moins réfléchissante qu'un miroir, on peut également utiliser des modèles de moins haute qualité du côté de la réflexion du miroir. Ces deux techniques ont par exemple été utilisées dans *Mario Galaxy* pour donner l'illusion de la réflexion sur un lac.



Figure 90 Fausse réflexion sur la surface de l'eau d'un lac dans Mario Galaxy

Figure 91 La réflexion du château est en fait un modèle différent qui est distordu par la surface de l'eau

L'avantage majeur de la technique de duplication de géométrie est sa rapidité de rendu et sa simplicité algorithmique car elle ne nécessite pas de calculs spéciaux. Elle a été très utilisée à une époque où les calculs de réflexion étaient trop compliqués pour les machines grand public. Le désavantage majeur de cette technique est qu'elle requiert d'avoir la place nécessaire permettant de placer la scène reflétée dans la scène réelle. Il n'est donc pas possible sans technique de masquage d'afficher un miroir s'il existe une autre pièce derrière le miroir.

Cube maps

On peut utiliser des cube maps pour la réflexion d'un miroir, cette technique ayant l'avantage de ne pas nécessiter de dupliquer la géométrie de la scène. Pour utiliser une cube map pour un miroir, il faut alors placer l'origine de la cube map au centre du miroir pour obtenir un reflet convainquant. Si l'on utilise une cube map statique, les objets dynamiques comme le personnage joué ne sont alors pas visibles dans la réflexion, ce qui peut briser l'immersion du joueur. Pour que les objets tels que le personnage joué soient visibles, il faut donc utiliser des cube maps dynamiques qui sont beaucoup plus lourdes à calculer.



Figure 92 Réflexion incorrecte dans un miroir du jeu Mafia 3 due à l'utilisation de cube maps semi dynamiques

Le jeu *Mafia 3* utilise la technique des cube maps pour afficher des reflets sur les miroirs. Ces cube maps sont hybrides, elles ne se mettent à jour que toutes les 5 à 10 secondes. Cela permet de gagner grandement en performance mais les reflets affichés ont une grande latence par rapport au reste de la scène. On peut également remarquer que la section entre les images des faces de la cube map est visible. Pour ces raisons, on utilise d'autres algorithmes spécialisés pour le calcul des réflexions planaires.

Planar reflections probes

Les planar reflection probes sont un type d'objets spécialisés dans la génération des reflets qui ne se basent pas sur les cube maps. En effet, pour calculer les reflets de l'environnement sur une surface plane, ces objets créent une seconde caméra symétrique par rapport à la caméra principale du jeu. Cette caméra effectue un second rendu de la scène de son point de vue. Ce rendu de l'image reflétée de la scène est alors ajouté sur la surface du miroir pour afficher un reflet réaliste de la scène. Les moteurs de jeux comme *Source Engine*, *Unity* et *Unreal Engine* proposent ce genre d'objets pour augmenter la qualité des réflexions sur les miroirs ou les sols laqués.

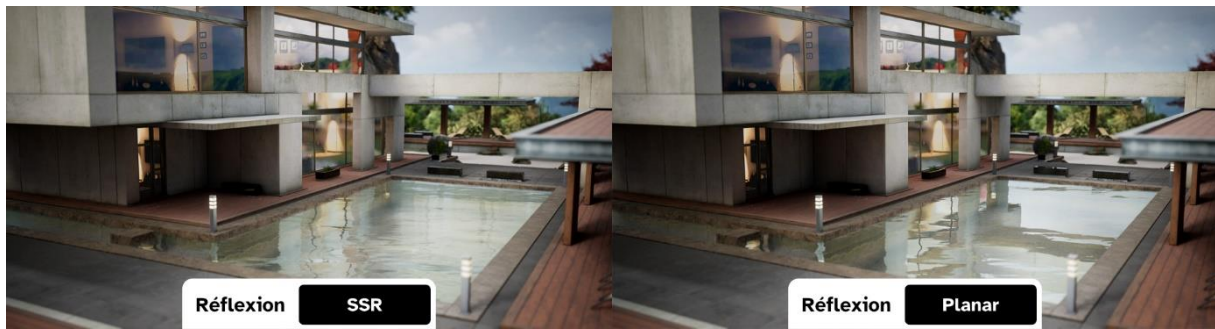


Figure 93 Comparaison des réflexions planaires en utilisant l'algorithme Screen Space Reflection et des Planar Reflection Probes

Cette technique est coûteuse en calculs car il faut effectuer deux fois le rendu de la scène. Sans technique d'optimisation, on doit alors doubler le temps de rendu de la scène, ce qui est inenvisageable pour la plupart des jeux vidéo. Pour gagner en performance, il existe plusieurs solutions. La première consiste à rendre l'image de réflexions dans une résolution plus basse que le rendu de la scène que l'on affiche à l'écran. La deuxième est d'effectuer le rendu de la réflexion à une fréquence plus basse pour avoir moins d'images à rendre. La troisième consiste à rendre la réflexion de la scène en utilisant un niveau de détails plus bas pour les modèles 3D de la scène et pour les ombres. Ces techniques d'optimisation permettent, dans certains cas, de rendre le rendu de la réflexion très rapide.

L'illumination Globale

Description

Jusqu'ici, les techniques d'illuminations abordées se contentent majoritairement de calculer l'impact de la lumière directe sur la scène. Or, la lumière directe n'est qu'une des deux formes de sources d'illumination. La seconde source d'illumination des objets est la lumière indirecte.

Calculer l'impact de la lumière sur un objet de manière réaliste consiste à résoudre l'équation de rendu, une intégrale utilisée pour calculer la luminosité d'une surface du point de vue d'un observateur.

$$L_o(\omega_0) = \int_{\Omega} L_i(\omega_i) \cos\theta_i f_r(\omega_i, \omega_o) d\omega_i$$

Figure 94 Équation du rendu de la lumière

Cette fonction peut être décomposée de la manière suivante : $\int_{\Omega}$ correspond à tous les angles de l'hémisphère orientés dans le même sens que la surface, $f_r(\omega_i, \omega_o)$ correspond à la BRDF de la surface, $L_i(\omega_i)$ correspond à l'intensité de la lumière arrivant depuis l'angle ω_i et $\cos(\theta)$ est le terme de la géométrie de la surface.

$$L_i(\omega_i) = L_{directe}(\omega_i) + L_{indirecte}(\omega_i)$$

Figure 95 Décomposition du terme de l'intensité lumineuse en fonction de l'intensité lumineuse directe et indirecte

Ici $L_{directe}$ correspond à la lumière provenant directement des lampes de la scène tandis que $L_{indirecte}$ correspond à la lumière reflétée par les objets environnants dans la scène. Les techniques d'illumination abordées précédemment permettent de calculer $L_{directe}$ mais d'autres techniques plus complexes ont été développées pour calculer le rebond de la lumière $L_{indirecte}$.

Ces algorithmes d'illumination globale sont donc souvent très sophistiqués et permettent de calculer les rebonds de la lumière dans une scène. Ils sont très lourds à calculer mais augmentent significativement la qualité des rendus 3D en apportant de la cohésion à l'éclairage des volumes et en permettant d'avoir un éclairage ambiant plus doux que l'éclairage direct.

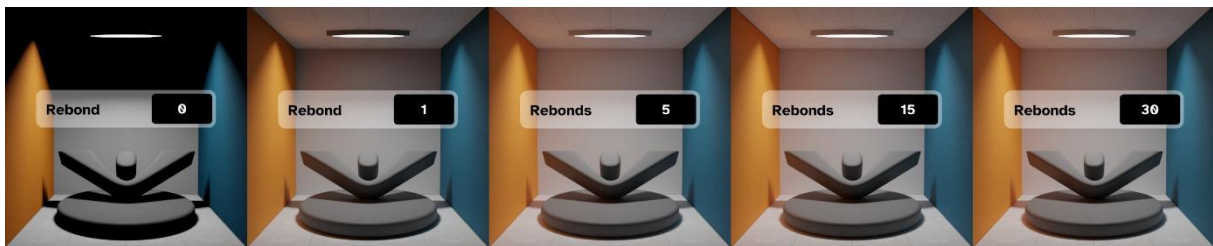


Figure 96 Comparaison du rendu d'une image en fonction du nombre de rebonds accordés à chaque rayon de lumière

Étant donné que le calcul d'illumination globale n'est pas intrinsèquement complexe mais peut être très lent, la majorité des techniques d'illuminations globales dans le jeu vidéo cherche à réduire ce temps de calcul au maximum lors du jeu.

Illumination globale statique

La manière la plus rapide pour afficher l'illumination globale dans le jeu vidéo est de ne pas la calculer pendant le jeu mais bien en amont, on parle alors d'illumination globale statique. Cette technique consiste donc à précalculer l'illumination d'une scène pendant la création du jeu, puis à stocker cette information pour être utilisée durant le jeu.

L'avantage de cette méthode est que le temps de calcul de l'illumination globale peut prendre plusieurs heures, voire plusieurs jours, mais une fois en jeu, les performances seront très bonnes car les informations seront déjà présentes.

Le désavantage de cette technique est que l'éclairage global n'est pas dynamique. Si la position d'une lampe change, seule son illumination directe pourra être recalculée.

Light mappers

On peut utiliser des light mappers pour calculer l'illumination directe des environnements 3D. Cette technique consiste à précalculer l'illumination globale en utilisant des algorithmes similaires à ceux utilisés au cinéma, puis à stocker cette information dans des textures que l'on ajoute aux modèles 3D de la scène.

La grande majorité des moteurs de jeux actuels qui proposent de la 3D comporte une ou plusieurs solutions de calcul d'illumination globale dans leurs lights mappers, comme *Unity*, *Unreal Engine*, *Godot Engine*, etc.

Radiosity

L'un des premiers algorithmes de calcul des rebonds de lumière appliqué au light mapping est l'algorithme Radiosity. Cet algorithme a été développé dans les années 1950 pour le calcul des transferts de chaleur entre les surfaces et a ensuite été modifié pour l'application à l'imagerie numérique par les Universités de Cornell et d'Hiroshima.

Cet algorithme calcule l'illumination d'une scène en la séparant en une multitude de sous-parties que l'on appelle des patches. Puis il calcule l'impact de la réflexion de la lumière de toutes les paires de patches possibles dans la scène que l'on appelle un facteur de transfert de lumière. Une fois ce calcul fait, Radiosity calcule l'éclairage direct de chaque patch et commence le processus de calcul des rebonds de manière itérative. Pour chaque rebond de lumière à calculer, il détermine le transfert de lumière entre chaque paire de patches et applique la nouvelle lumière aux deux patches. Une fois toutes les paires de patches modifiées, on peut recommencer l'opération pour calculer un second rebond de lumière.

Dans l'algorithme de calcul de transfert de lumière naïf, il faut déterminer l'impact de chaque patch sur tous les autres patches et calculer les transferts d'énergie entre chaque patch pour chaque rebond de lumière. La complexité de ce calcul augmente donc de manière quadratique pour chaque nouveau patch et linéairement pour chaque rebond de lumière. Les performances de l'algorithme peuvent être

très mauvaises pour des scènes détaillées, il existe donc différentes techniques pour optimiser ces calculs.

Le jeu *Quake 2* utilise cet algorithme pour son light mapper. Pour augmenter la rapidité du calcul du facteur de transfert d'illumination, la lumière directe est calculée pour chaque pixel de la light map. L'illumination indirecte utilise des patches de plus grande taille et précalcule la visibilité entre les patches et leurs facteurs de transfert. Cela permet de réduire le nombre d'opérations de calculs entre les paires de patches significativement. Enfin, le calcul du transfert d'illumination est effectué de manière classique.

Path tracing

Le path tracing ou lancer de chemin est un algorithme basé sur le lancer de rayon qui fonctionne de la manière suivante : on lance un rayon depuis un observateur, quand le rayon touche un objet on calcule un angle de rebond aléatoirement en se basant sur la distribution de la BSDF du matériau, et on répète l'opération jusqu'à ce que l'on trouve une source de lumière. On calcule ensuite le transfert de lumière depuis la source de lumière vers l'observateur en prenant en compte les propriétés des surfaces à chaque point du chemin.

Comme les chemins sont très irréguliers, l'image obtenue contient beaucoup de grain, on utilise donc la méthode de Monte-Carlo en envoyant une multitude de chemins et en additionnant leurs résultats pour obtenir une image propre. Chaque chemin aléatoire de la lumière est appelé un sample.



Figure 97 Comparaison du grain de l'image en fonction du nombre de chemins parcourus par pixel

Comme on peut le voir dans l'image ci-dessus, pour obtenir une image viable, il faut parcourir des milliers de chemins, ce qui rend l'algorithme très chronophage. Pour augmenter l'efficacité de cet algorithme, on peut utiliser la technique de l'importance sampling, consistant à orienter les rayons de lumière en direction des chemins qui ont un impact majeur sur le rendu final. En priorisant les chemins ayant une grande importance, on peut réduire drastiquement le nombre de rayons "inutiles" et augmenter la rapidité de cet algorithme. On peut également vérifier à chaque point du chemin si ce

point est directement éclairé par une source de lumière, ce qui permet d'augmenter le nombre de samples sans devoir recalculer complètement un chemin.

Les techniques de path tracing sont celles utilisées au cinéma pour le rendu des effets spéciaux. Dans ce cas, l'observateur est une caméra virtuelle. Pour les light mappers dans le jeu vidéo, les observateurs sont les pixels de la light map qui sont plaqués sur les surfaces de la scène. Le light mapping utilisant le path tracing ou des méthodes similaires est le standard actuel pour les moteurs de jeux comme *Unreal Engine*, *Unity Engine* ou encore *Godot Engine*.

Photon mapping

Le photon mapping est un algorithme très similaire au path tracing, il est basé sur le lancer de rayons et utilise la méthode de Monte-Carlo pour obtenir des images sans grain. La spécificité de cet algorithme est qu'il simule des photons en lançant des rayons dans toutes les directions depuis des sources de lumières, en plus de lancer des rayons depuis un observateur. L'algorithme essaye ensuite de relier les rayons venant des sources de lumière et de l'observateur en effectuant des rebonds de lumières.

Volumetric Global Illumination Samplers

Les lights mappers sont très efficaces pour afficher l'illumination globale sur les objets statiques des scènes mais n'affectent pas les objets de la scène qui peuvent se déplacer. En effet, appliquer une light map classique sur un objet mouvant crée généralement des artefacts visuels car l'illumination globale diffère entre les emplacements dans une scène. On ne peut pas donc appliquer l'illumination globale sur des objets dynamiques à l'aide de light maps classiques, ce qui leur donne une apparence différente du reste de l'environnement.

Pour pouvoir appliquer une illumination globale statique sur des objets dynamiques, on utilise alors des techniques qui enregistrent l'illumination globale dans un volume. Les moteurs donnent à leurs méthodes de stockage volumique de la lumière des noms spécifiques, mais elles se basent toutes sur un principe commun. Ces moteurs discrétisent la scène en une grille 3D qui enregistre les informations d'illumination à différents points dans un volume en stockant la lumière arrivant dans toutes les directions dans des fonctions harmoniques sphériques (*Spherical Harmonics Function*). Puis, pour chaque objet, ils calculent l'illumination globale à sa position en interpolant les valeurs des points les plus proches.

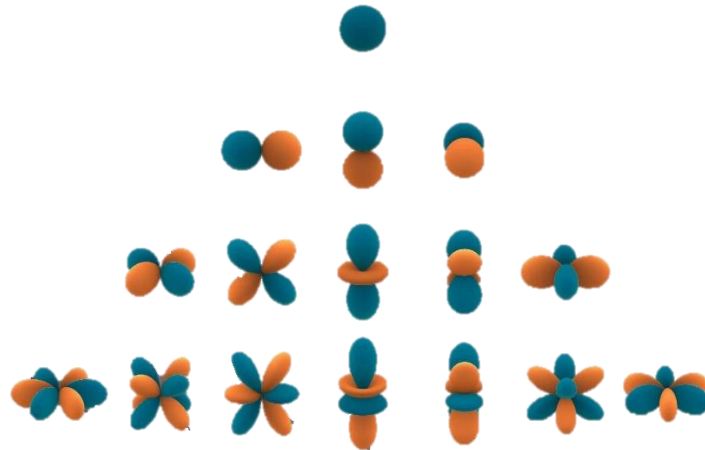


Figure 98 Représentation des premiers niveaux des fonctions harmoniques sphériques, les portions cyan représentent des valeurs positives et les portions orange représentent des valeurs négatives

Unity

Le moteur de jeu *Unity* utilise des objets appelés light probes groups pour afficher l'illumination globale sur des objets dynamiques. Une light probe est un point dans l'espace où l'on calcule l'illumination globale, qui est ensuite stockée dans une fonction harmonique sphérique. Ces light probes sont placées dans les scènes à la main ou automatiquement grâce à des packages d'*Unity* spécifiques.

Les lights probes d'*Unity* peuvent être placées partout et ne sont pas alignées sur une grille, ce qui rend l'interpolation de leurs informations de lumière assez complexe. Dans un contexte 2D, on utilise généralement des triangles pour relier des points dans un volume et interpoler leurs valeurs. Pour son environnement 3D, *Unity* utilise une méthode similaire en reliant les points dans l'espace grâce à des tétraèdres.

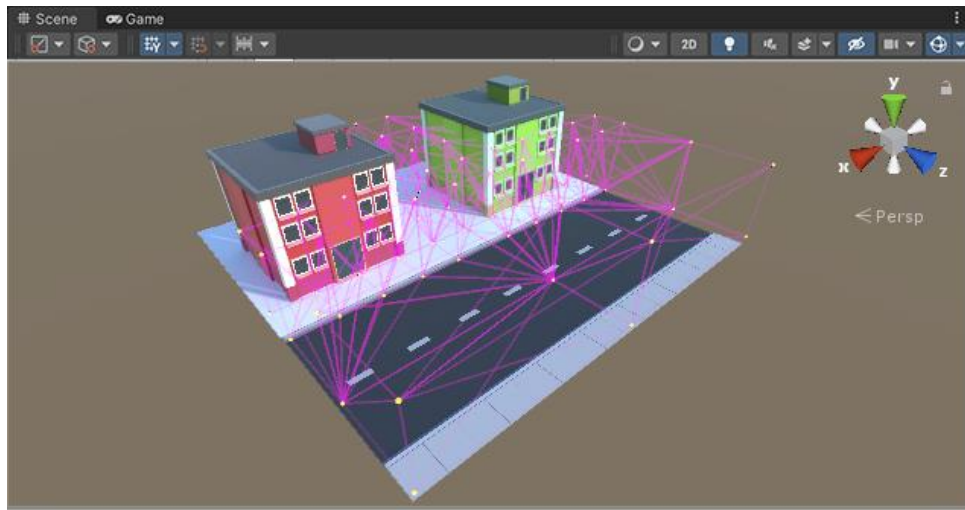


Figure 99 Light probes groups affichés dans Unity, les points jaunes représentent les probes et les lignes roses représentent les connexions entre les probes

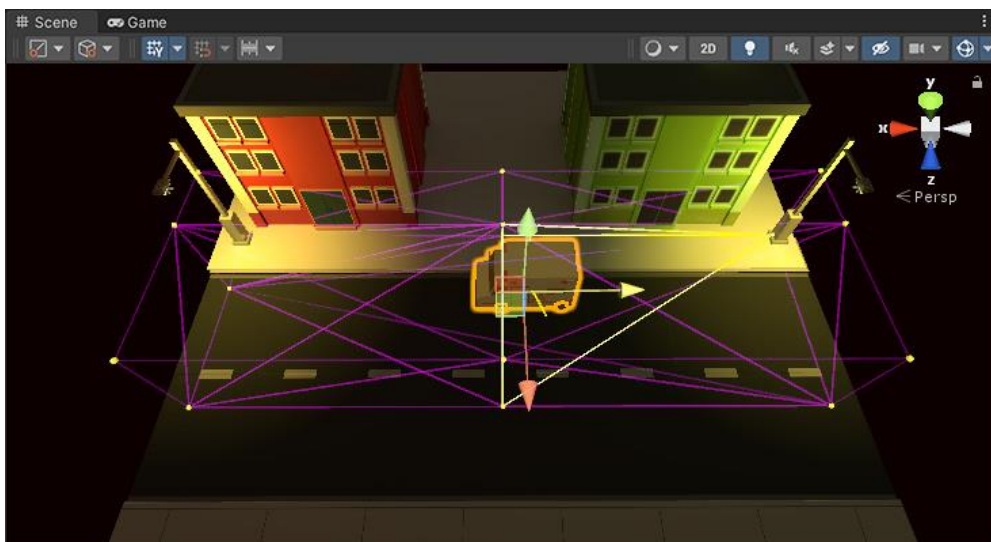


Figure 100 Objet dynamique étant éclairé par les probes qui sont reliées par des lignes jaunes, la proportion de l'impact de chaque probe est calculée en fonction de sa distance avec l'objet

Unreal Engine

Unreal Engine propose une technique appelée volumetric light map pour le calcul de d'illumination globale statique. Cette technique permet d'afficher cette illumination pour les objets dynamiques et les objets statiques.

Les points de calcul de l'illumination globale sont placés sur une grille tridimensionnelle dont la densité peut varier en fonction de zones d'importance. Encore une fois, les informations de lumière sont stockées dans une fonction harmonique sphérique (d'ordre trois).

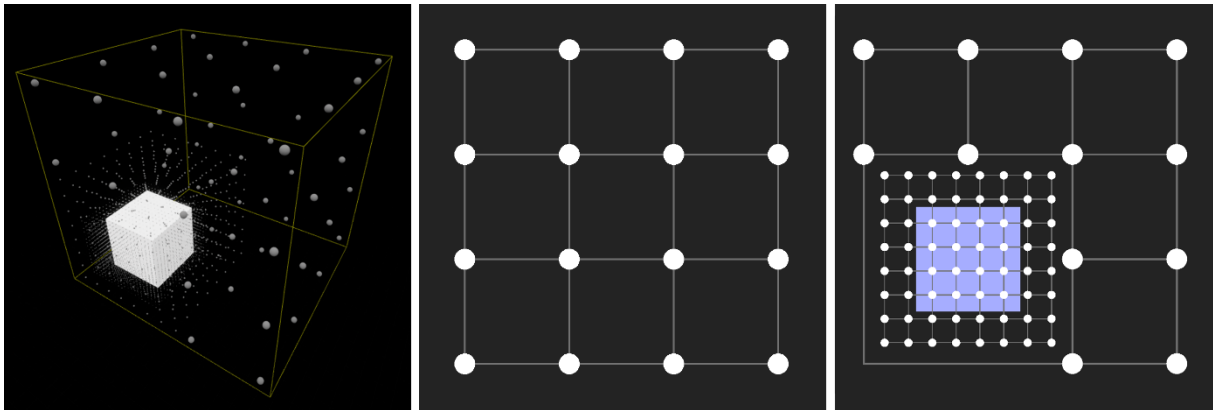


Figure 101 Volumetric light map à la densité variable et schémas en 2D de la grille dont la densité change en fonction de la géométrie de la scène

Godot

Le moteur de jeu *Godot* utilise également une grille volumétrique le light probes générée automatiquement. On peut placer manuellement les lights probes pour un meilleur contrôle de l'illumination. Le moteur utilise aussi des harmoniques sphériques pour stocker les informations d'illumination globale.

Éclairage Dynamique

Lorsqu'un environnement 3D comporte un éclairage qui change au fil du temps ou que l'environnement lui-même est dynamique, on ne peut pas calculer son illumination globale à l'avance. Les techniques d'illumination globale en temps réel doivent donc permettre de calculer l'approximation des rebonds de la lumière de manière très rapide, tout en gardant une fidélité visuelle comparable à des techniques précalculées.

Pour pouvoir effectuer ces calculs en un temps acceptable, il existe de nombreuses techniques qui sont plus ou moins sophistiquées et plus ou moins performantes.

Global light

La technique la plus simple et la plus utilisée pour simuler l'impact de l'illumination globale sur le rendu d'une scène est d'utiliser une lumière globale. Ce genre de lumière n'est pas physiquement réaliste car la lumière ne provient pas d'une source de lumière. Elle est utilisée uniquement comme outil de contrôle artistique sur l'illumination d'une scène car sa lumière est appliquée à tous les objets de la scène et empêche d'avoir des zones d'ombre sans aucune lumière.

Comme ce type de lumière augmente la luminosité globale de la scène, il permet d'accroître la visibilité dans les zones d'ombres, ce qui est très important pour la navigation dans l'espace de jeu. Les lumières globales peuvent être colorées, par exemple pour simuler la lumière du ciel.

Cette technique n'est pas physiquement plausible et n'est pas très convaincante comparativement à des algorithmes plus complexes mais elle a l'avantage majeur de n'avoir virtuellement aucun impact sur les performances des rendus. Elle est cependant indispensable pour garder de la visibilité dans les zones d'ombre si aucune autre technique d'illumination globale n'est utilisée.

Plaçage de lampes dans la scène

Une autre technique pour simuler l'illumination globale dans une scène est de placer à la main des lampes qui illuminent des zones non illuminées par l'éclairage direct, mais qui devraient être éclairées par les rebonds de lumières.

Cette technique peut être chronophage car il faut placer des lampes secondaires supplémentaires dans les environnements 3D mais elle permet en contrepartie de contrôler l'éclairage de manière beaucoup plus granulaire qu'une simple lumière globale.

Ubisoft Deferred Radiance Transfer Volumes (DRTV)

Pour la création du jeu *Far Cry 3* sorti en 2012, *Ubisoft* a mis en place une technique de rendu de l'illumination globale dynamique appelée Deferred Radiance Transfer Volume (DRTV), visant à répondre aux besoins d'un jeu ayant une aire de jeu massive et un cycle jour/nuit dynamique.

Cette technique consiste à placer une grille 3D de points stockant le transfert de lumière en fonction des angles potentiels des sources lumineuses dans une structure appelée Precomputed Radiance Transfer (PRT). Ces PRT permettent au moteur de rendu de connaître les interactions des rebonds de lumière sans connaître l'éclairage d'une scène à l'avance. L'équipe ayant conçu le jeu s'est inspirée d'un papier de recherche du *Siggraph* de 2002 par Sloan et d'exemples d'implémentation des PRT présents dans *DirectX*. Les PRT du jeu *Far Cry 3* stockent également la visibilité du ciel pour pouvoir changer l'éclairage ambiant au fil des heures en se basant sur la couleur dynamique du ciel du jeu.



10

Figure 102 Illustration du changement des rebonds de la lumière en fonction de l'orientation du soleil

Cette technique permet donc d'obtenir une illumination globale dynamique dans un environnement principalement statique, permettant par exemple à *Ubisoft* d'implémenter une forme d'illumination globale dynamique dans son jeu *Open world*, tout en gardant des performances graphiques très élevées. *DRTV* ne consomme que très peu de mémoire, moins d'un mégabit de mémoire système pour le stockage des *PRT*. L'*algorithme* est également très rapide avec un temps d'exécution autour d'une milliseconde.

Nvidia Voxel Global Illumination VXGI

L'*algorithme Voxel Global Illumination (VXGI)* est une technique d'illumination globale développée par *Nvidia* basée sur la représentation de la scène en *voxels*. Un *voxel* ou *volumetric pixel* est un cube aligné sur une grille 3D, l'*algorithme* crée donc une représentation de la scène sous forme de cubes pour accélérer les calculs d'illumination globale, on cherche ici à discrétiser la scène pour réduire sa complexité et donc la complexité du calcul de lumière. Pour augmenter les performances de l'*algorithme*, le niveau de détails ou *LOD* de la grille de voxels est réduit en fonction de la distance. Cet *algorithme* a été présenté à la *GDC 2015*, il est inspiré d'un autre *algorithme* présenté par *Nvidia* en 2013 appelé *SVOGI* qui est aussi basé sur le *voxel*.

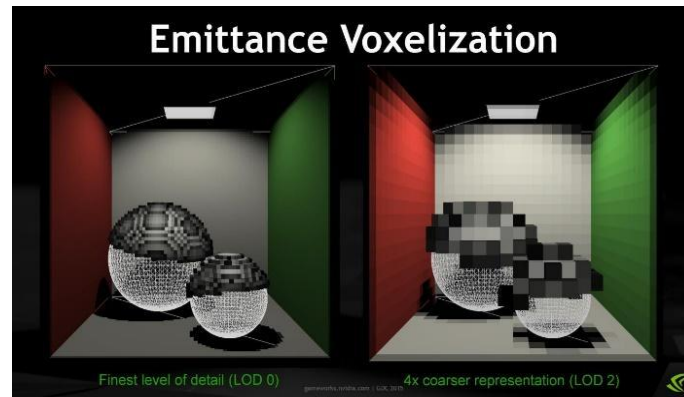


Figure 103 Comparaison de la taille de la grille de voxels de calculs de lumière en fonction du niveau de détail sélectionné

L'algorithme effectue le calcul de transfert de lumière entre les surfaces en effectuant du Cone Tracing, il s'agit d'un algorithme permettant de calculer l'intensité de lumière réfléchiée par une surface dans une direction approximative définie par l'orientation et l'angle du cône.

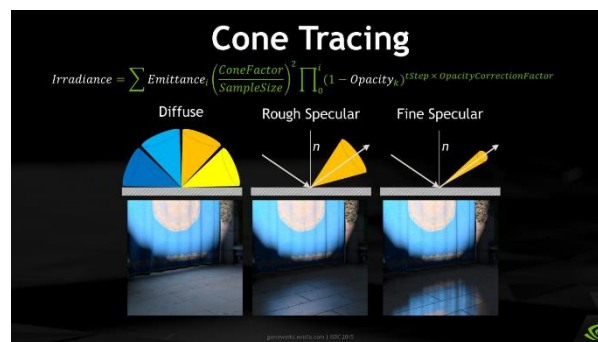


Figure 104 Schéma du calcul des rebonds de lumière à l'aide de l'algorithme de Cone Tracing

Nvidia Reservoir-based Spatio-Temporal Importance Resampling Global Illumination (ReSTIR GI)

L'algorithme Reservoir-based Spatio-Temporal Importance Resampling Global Illumination ou ReSTIR GI développé par Nvidia est une évolution de l'algorithme ReSTIR que nous avons abordé précédemment. Cette évolution est mise en œuvre pour supporter les rebonds de lumière de l'illumination globale contrairement à ReSTIR qui ne supporte que l'illumination directe. ReSTIR augmente significativement l'efficacité du Ray Tracing pour l'illumination directe en ré-utilisant des informations de rayons proches spatialement et temporellement en les stockant dans des réservoirs virtuels. En comparaison, ReSTIR GI augmente la rapidité des algorithmes de path tracing en ré-utilisant des informations proches temporelles et celles proches dans l'espace de l'écran.

Godot Voxel Global Illumination

Le moteur de Jeu *Godot Engine* comporte actuellement trois algorithmes de calculs de l'illumination globale en temps réel et un quatrième est en cours de développement. Le premier d'entre eux, appelé Voxel GI, crée une représentation de la scène en une grille de voxels pour effectuer ses calculs de rebonds de lumière. Cette technique utilise un algorithme de ray marching pour son calcul d'ombre et permet d'obtenir jusqu'à deux rebonds de lumière de manière dynamique. Voxel GI permet également d'obtenir des réflexions dynamiques dans la scène. Pour mettre en place cette méthode, il faut d'abord effectuer un processus de pré-calcul de la géométrie de la scène pour commencer le calcul des réflexions de la lumière en temps réel.

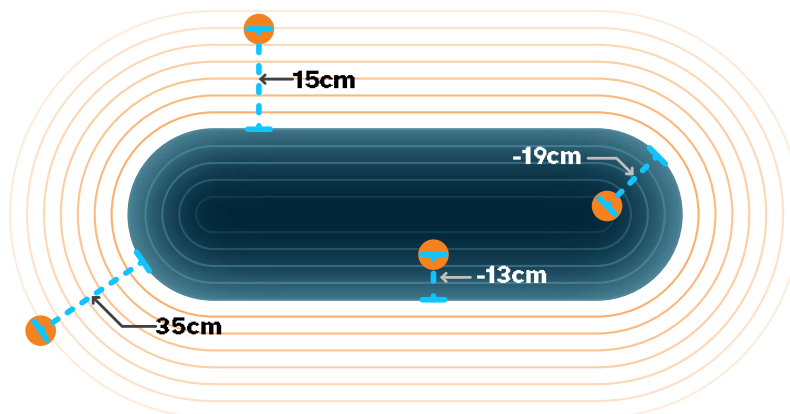


Figure 105 Comparaison d'une scène dans *Godot* sans illumination globale, avec *Voxel GI* et visualisation des voxels

Cette technique est principalement utilisée pour les environnements de taille moyenne comportant des sources lumineuses et des objets dynamiques.

Godot Signed Distance Field Global Illumination (SDFGI)

Le deuxième algorithme de calcul d'illumination globale en temps réel que propose *Godot* s'appelle Signed Distance Field Global Illumination (SDFGI). Un Signed Distance Field (SDF) est une fonction décrivant en tout point de l'espace quelle est la distance avec la surface d'une géométrie quelconque. Sa spécificité est de pouvoir être négative. Lorsque l'on obtient une distance négative de la SDF d'un objet, cela signifie que l'on se trouve à l'intérieur du volume de l'objet.



Signed Distance Field d'une Capsule en 2D

Figure 106 Visualisation d'un Signed Distance Field d'une capsule en 2D, les paliers orange représentent une distance positive de 5 cm tandis que les paliers cyan représentent une distance négative de 5 cm, les distances affichées correspondent à la distance des points par rapport à la surface de la capsule

Godot utilise cette technique pour discrétiser la scène en une multitude de SDFs qui permettent de connaître la distance d'un point avec les différentes parties de l'environnement. Ces distances permettent de calculer plus rapidement l'atténuation de la lumière en fonction de la distance et d'accélérer grandement le calcul de simulation des rebonds de la lumière.

De plus, l'algorithme permet de calculer plus de rebonds de lumière au cours du temps pour améliorer son réalisme sans impacter fortement les performances en jeu. Les informations de rebonds de lumière sont ensuite stockées dans une structure en forme de grille 3D similaire à des voxels, cette structure ayant un niveau de détail dynamique en fonction de la distance pour obtenir un résultat détaillé proche de l'observateur. Enfin, le moteur utilise ces informations de lumière pour générer une grille de light probes qui permettent d'affecter les objets dynamiques de la scène.



Figure 107 Affichages d'une scène dans Godot en visualisation classique, en visualisation de la scène discrétisée par SDFGI et en visualisation de la grille 3D de Light Probes

Godot Screen Space Indirect Lighting (SSIL)

Le dernier algorithme d'illumination globale présent actuellement dans *Godot* est appelé Screen Space Indirect Lighting (SSIL). Cet algorithme, qui fonctionne sur les mêmes bases que de Screen Space Shadows ou de Screen Space Reflections, a été développé pour compléter les autres techniques d'illumination globale au même titre que les autres algorithmes screen space ont été développés pour compléter respectivement les ombres et les réflexions.

Le SSIL fonctionne en effectuant des calculs de rebonds de lumière dans l'espace de l'écran similaire au SSR mais utilise une distribution des rayons basée sur une distribution de Lambert au lieu d'une distribution de réflexions spéculaires comme celle de Phong.

En utilisant cette technique en plus des autres disponibles dans le moteur, on peut augmenter le niveau de détail de l'illumination globale du moteur.

Godot Hierarchical Digital Differential Analyzer Global Illumination (HDDAGI) basé sur le papier de Dreamworks

Juan Linietsky, créateur du moteur *Godot*, a commencé le développement d'un nouvel algorithme d'illumination globale en temps réel nommé Hierarchical Digital Differential Analyzer Global Illumination (HDDAGI). Cet algorithme, censé remplacer l'algorithme SDFGI dans les prochaines versions du moteur de jeu, est basé sur un papier de recherche présenté par *Ken Museth* du studio d'animation *Dreamworks*. Il permet d'accélérer les algorithmes de Ray Marching dans les structures de données représentant des volumes.

Cette amélioration de vitesse de l'algorithme permet à *Godot* d'obtenir une illumination globale plus détaillée en moins de temps et de réduire les algorithmes visuels de la solution actuelle. Elle permet également d'obtenir une illumination globale plus réactive aux changements de lumière, comparativement à SDFGI, et d'avoir des meilleures performances quand la caméra du jeu se déplace rapidement. Ces différentes améliorations permettent d'obtenir un rendu plus fidèle à la vraie vie dans le moteur.



Figure 108 Comparaison du rendu de l'illumination globale dans une scène en les algorithmes SDFGI et HDDAGI

Electronic Arts Global Illumination Based on Surfels (GIBS)

L'algorithme Global Illumination Based on Surfels (GIBS) est une méthode d'illumination globale développée par *Electronic Arts* pour un rendu en temps réel dans son moteur de jeu *Frostbite*. Il a été présenté au *SIGGRAPH* 2021 par Henrik Halen et Andreas Brink.

L'algorithme est basé sur la discrétisation de la scène en utilisant des surfels. Un surfel, ou surface element, est un élément défini par une position, un rayon et une normale. Ces surfels sont placés dynamiquement dans la scène depuis le point de vue de la caméra. Ils sont persistants dans le temps et restent accrochés aux surfaces mouvantes, ce qui permet au moteur d'accumuler les rebonds de lumière au fil du temps. Comme ils sont placés depuis le point de vue de la caméra, leur densité est homogène pour l'observateur. Cela implique d'avoir plus de détails pour les surfaces proches et moins de détails sur les surfaces lointaines, le rayon des surfels augmentant également en fonction de la distance.

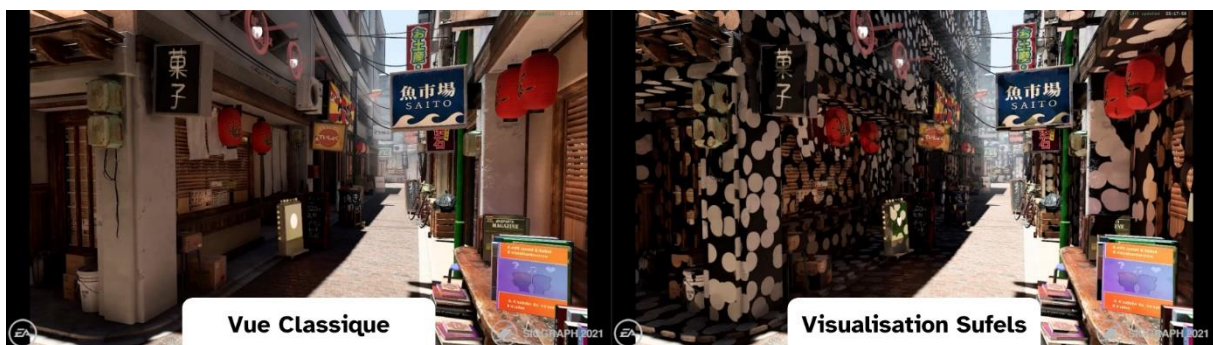


Figure 109 Visualisation des Surfels projetés dans une scène pour calculer en calculer l'illumination globale

Pour effectuer le calcul du rebond de la lumière, le moteur utilise un algorithme de ray tracing au niveau des surfels. Des rayons sont envoyés depuis les surfels dans l'environnement pour capter le niveau d'illumination et sont accumulés au cours du temps. De plus, les rayons peuvent capturer

l'illumination capturée par les autres surfels, ce qui augmente drastiquement le nombre de rayons pris en compte pour l'illumination sans augmenter le nombre de rayons à lancer. Enfin, les surfels peuvent partager leurs informations de lumières avec les surfels environnant pour encore augmenter le nombre de rayons pris en compte lors du calcul de lumière.

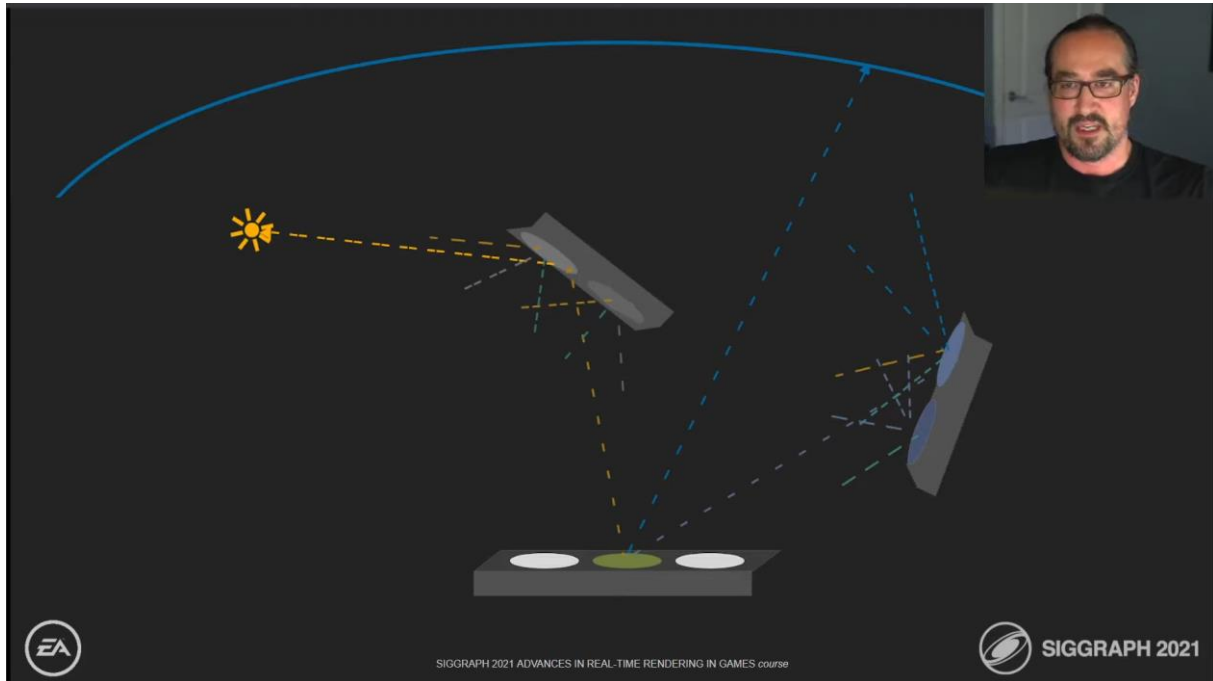


Figure 110 Schéma du fonctionnement de l'algorithme GIBS du moteur Frostbite d'Electronic Arts

L'avantage de cette technique est que le nombre nécessaire de rayons à lancer est bien moindre que de lancer des rayons pour chaque pixel de l'écran comme pour les algorithmes de path tracing au cinéma. Comme on a pu le voir dans le document, les algorithmes de light mapping ont besoin de cinquantaines, voire de centaines de rayons accumulés par pixels pour obtenir un éclairage viable. De plus, EA s'est inspiré du travail effectué par *Nvidia* pour *ReSTIR* en utilisant une structure en réservoir pour l'accélération du lancer de rayons.

Le matériel informatique actuel permet en moyenne de calculer en temps réel un à deux rayons par pixel jusqu'à dix rayons par pixel pour les cartes graphiques les plus puissantes. Comme cet algorithme effectue son lancer de rayon sur des objets discrétisés, le nombre de lancer de rayon par pixel par image est assez bas pour pouvoir tourner en temps réel sur des machines de moyenne gamme. De plus, comme beaucoup d'algorithmes de lancer de rayon, *GIBS* optimise son lancer de rayon grâce à des techniques d'importance sampling.

Pour éclairer la scène en utilisant les surfels, *Frostbyte* les considère comme des lampes virtuelles et effectue une interpolation linéaire de leurs valeurs de luminosité pour les objets de l'environnement 3D. Enfin, pour l'éclairage des objets n'ayant pas d'effet sur l'illumination globale, le moteur utilise une grille 3D d'harmoniques sphériques pour créer un volume d'illumination similaire aux techniques abordées précédemment dans ce document.

Unreal Engine Lumen

Le moteur de jeu *Unreal Engine 5* a introduit une technique d'illumination globale en temps réel appelée Lumen. Cette technique se base sur le lancer de rayon pour calculer les rebonds de lumière dans la scène. L'objectif de cet algorithme est le même que celui de GIBS d'EA : réduire au maximum le nombre de rayons à lancer par pixel pour obtenir une image sans artéfacts comme du grain.



Figure 111 Illumination globale en temps réel dans une scène utilisant l'algorithme Lumen d'Unreal Engine 5

Pour réduire le nombre de rayons nécessaires pour le calcul d'une illumination globale, Lumen regroupe le lancer de rayons dans deux groupes de light probes. Le premier groupe est appelé le Screen Space Radiance Cache, qui place une probe sur la surface de la scène tous les 16 pixels de l'écran mais place des probes supplémentaires dans les zones de fort contraste pour augmenter le niveau de détail de l'illumination, cette méthode de placement adaptative étant appelée Adaptive Sampling. Le second groupe, appelé le World Space Radiance Cache, permet de placer les lights probes dans une grille 3D dans la scène et de moduler le niveau de détails de la grille en fonction de la géométrie de la scène. Le

Screen Space Radiance Cache effectue son lancer de rayon pour les sources de lumières directes et indirectes à courtes et moyennes distances à la manière de GIBS. Les probes en Screen Space sont également décalées à chaque image pour pouvoir calculer l'illumination sur une plus grande surface et éliminer des potentiels artéfacts visuels. Le World Space Radiance Cache est utilisé pour capturer les sources de lumières directes et indirectes très lointaines. Ces deux groupes de lights probes sont utilisés conjointement pour enregistrer les rebonds de lumière générale de la scène, puis le moteur de rendu effectue différents algorithmes d'interpolation et de filtrage pour appliquer l'illumination globale dans la scène.

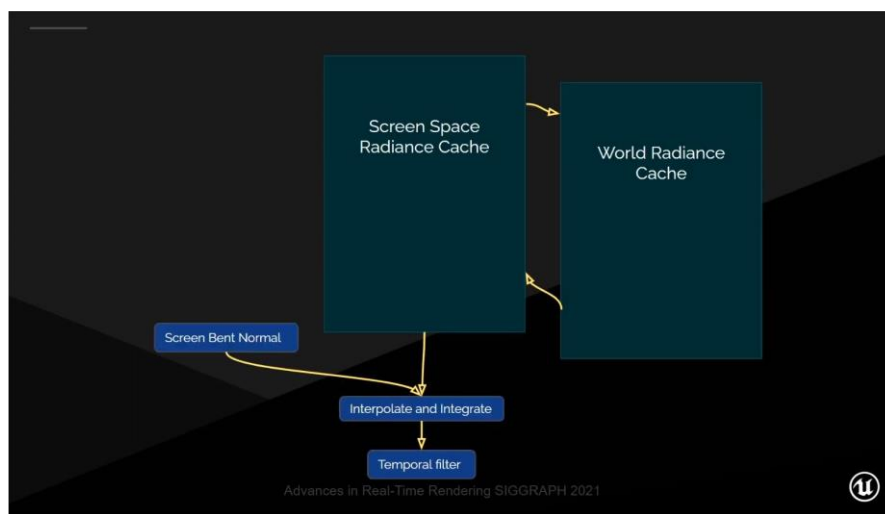


Figure 112 Schéma de l'architecture logicielle de l'algorithme Lumen

Tout comme GIBS, Lumen effectue son calcul de rebonds de lumière au cours du temps pour réduire le nombre de rayons à lancer à chaque instant du jeu et augmenter les performances de rendu. La durée d'accumulation des rebonds de lumière au cours du temps peut être réduite pour les objets se déplaçant rapidement pour éviter que leurs ombres ne restent « collées » à leurs anciens emplacements. De plus, il utilise aussi différents algorithmes pour orienter les rayons afin d'obtenir des résultats significatifs, plus souvent comme Importance Sampling, les informations des sources lumineuses importantes pouvant être partagées entre les probes afin d'accélérer la convergence des calculs de lumière.

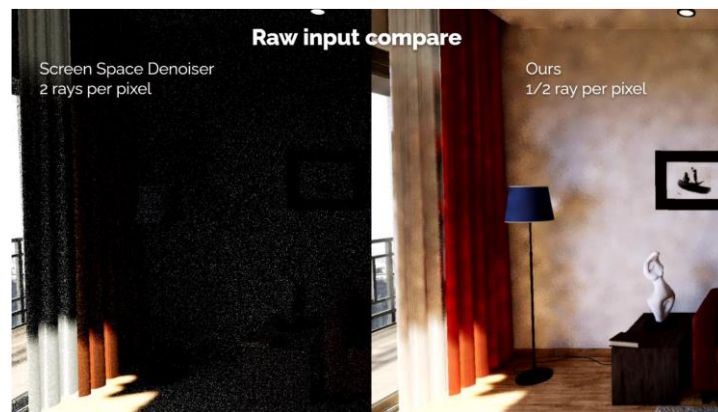


Figure 113 Comparaison de la qualité de rendu d'un algorithme de lancer de rayons classique et de lumen pour un nombre restreint de lancer de rayons par pixels

Toutes ces optimisations permettent à Lumen de calculer l'illumination globale d'une scène en utilisant une moyenne de 0.5 à 1 lancer de rayons par pixel de l'écran pour un coup de 3.74 millisecondes sur Playstation 5 avec une résolution Full HD.

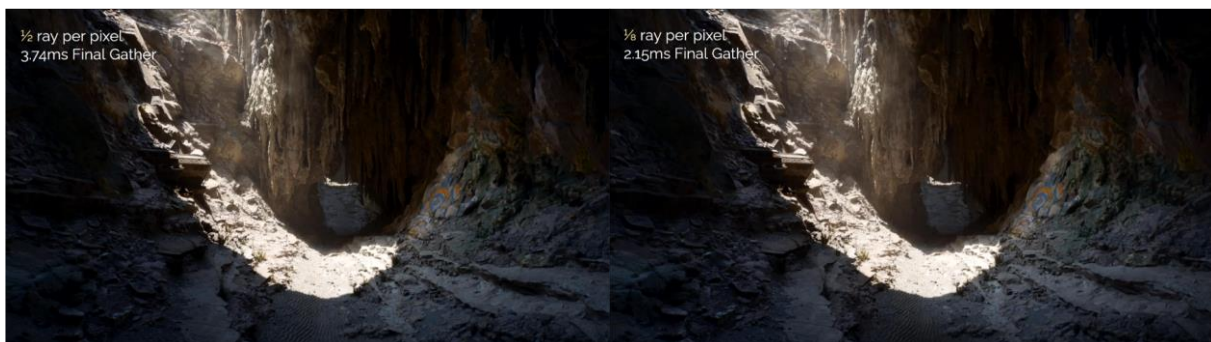


Figure 114 Comparaison de la qualité de l'illumination de Lumen avec un budget de 1/2 lancer de rayons par pixel et un budget de 1/8 lancer de rayons par pixel

Instant Radiosity / Virtual Lights

Instant Radiosity est une technique expérimentale de calcul d'illumination globale qui a été présentée par Alexander Keller au SIGGRAPH en 1997. Cette ancienne technique, peu utilisée, consiste à effectuer un lancer de rayons depuis les sources de lumières de la scène, à placer des nouvelles lampes au points d'intersection de ces rayons avec la scène et à appliquer l'illumination capturée à ce point comme illumination de la nouvelle lampe. En ajoutant ces nouvelles lampes, on simule un rebond de lumière, on peut ensuite recommencer l'opération autant de fois que l'on souhaite simuler des rebonds. Enfin, on ajoute l'impact de toutes les nouvelles lampes de la scène pour calculer l'illumination globale.

Cette technique permet théoriquement de calculer l'illumination globale en utilisant les mêmes de algorithmes que l'illumination directe. Elle est conceptuellement assez simple et peut permettre

d'unifier le rendu de la lumière directe et indirecte sans utiliser de lancers de rayons qui peuvent être très longs à calculer.

Cependant, elle requiert d'augmenter exponentiellement le nombre de lampes dans la scène, ce qui rend le calcul de lumière directe beaucoup plus long, on parle du problème de rendu « *Many Lights* ». Pour cette raison, *Instant Radiosity* est restée principalement expérimentale et d'autres *algorithmes* basés sur le *lancer de rayons* ont pris sa place.

On peut quand même noter qu'avec l'avancée du matériel et des *algorithmes* de rendu de la lumière directe comme *ReSTIR*, on commence à pouvoir effectuer le rendu de la lumière sur un nombre très élevé de lampes très rapidement. Il existe donc peut être une potentielle utilité pour une technique similaire de calcul d'illumination globale dans un futur proche.

NVIDIA / AMD / SONY / Microsoft Denoised Ray Tracing

Depuis les balbutiements de l'imagerie numérique, les *algorithmes* de *lancer de rayons* sont les seuls *algorithmes* permettant de capturer tous les aspects du rendu de la lumière de manière réaliste. C'est pour cette raison que les films d'animation, et plus largement les effets spéciaux utilisent des moteurs de *lancer de rayons* pour le rendu de leurs images : *Arnold*, *Corona*, *Cycles*, *Redshift*, *VRay*, *LuxCor*, *Octane*, etc. L'avantage de ces moteurs est qu'ils permettent d'obtenir des images très réalistes en modélisant des phénomènes physiques de manière cohérente. Les *algorithmes* de calcul de lumière basés sur le *lancer de rayon* sont assez simples conceptuellement, leur lenteur vient du fait qu'il faut généralement un nombre très élevé de rayons pour obtenir une image sans grain.

Jusqu'à récemment, le *lancer de rayon* pour les applications en temps réel était impossible car le nombre de rayons à lancer était bien trop élevé pour pouvoir lancer des rayons à partir de chaque pixel de l'écran sans obtenir des images illisibles à cause du grain de l'image. Cependant, on a pu voir une évolution drastique dans les performances de *ray tracing* sur les nouvelles cartes graphiques et consoles sorties sur le marché. Sur le marché du PC, les nouvelles cartes graphiques supportent maintenant du matériel spécifique pour le *lancer de rayon*, par exemple la série *Nvidia GeForce RTX 2000* sortie en 2018 et la série *Amd Radeon RX 6000* sortie en 2020. Apple a également lancé ses processeurs M3 comportant le support du *ray tracing*. Les premières consoles de jeu à supporter cette technologie en temps réel sont la *Playstation 5* et les *Xbox Series X et S*, toutes les trois sorties en 2020 également.

Ces évolutions drastiques sont dues à trois nouvelles technologies qui sont l'accélération matérielle du lancer de rayon présente dans les nouvelles puces graphiques, l'avancée des outils de dé-bruitage ou denoising et enfin l'avancée des algorithmes de génération de pixels et d'images.

L'ajout de matériels spécifiquement conçus pour le lancer de rayon a permis aux nouvelles cartes graphiques de pouvoir lancer plus de rayons dans un même laps de temps. Comme expliqué précédemment, les cartes graphiques d'aujourd'hui permettent de lancer 10 rayons par pixel par image tout au plus avec une moyenne de 2 à 5 rayons par pixel par image. Comparé aux 0.5 à 2 rayons par image des cartes graphiques des générations précédentes, l'augmentation est significative. Cela n'est toujours pas suffisant pour obtenir des images sans grain mais la quantité d'information sur les rebonds de lumière est tout de même bien plus élevée.

Ces nouvelles informations sont très importantes car elles réduisent potentiellement le grain de l'image par deux. C'est là que qu'entrent en jeu les algorithmes de débruitage. En effet, ces algorithmes sont conçus pour prendre une image contenant beaucoup de grain et de supprimer le grain pour obtenir une image propre. Les algorithmes les plus populaires actuellement utilisés sont Optix Denoiser créé par *Nvidia*, basé sur des algorithmes de Machine Learning, Fidelity FX Denoiser créés par *AMD* et Open Image Denoiser créé par *Intel*. Le denoiser de *Nvidia* ne fonctionne que sur les cartes du constructeur tandis que les autres fonctionnent sur tout type de carte graphique.



Figure 115 Comparaison d'un rendu en Ray Tracing en fonction du nombre de rayons par pixel et de l'activation ou non d'un algorithme de Denoising (ici Optix Denoiser). L'augmentation du nombre de rayons par pixel (samples) augmente la qualité de l'image de base et permet au Denoiser d'approcher l'image de référence avec moins d'artefacts visuels.

Les derniers algorithmes qui permettent d'augmenter drastiquement les performances en jeux sont les algorithmes de génération de pixels et d'images présents dans les frameworks DLSS de *Nvidia* et FSR d'*AMD*. Pour la génération de pixels, les deux frameworks proposent des solutions d'UpScaling qui permettent d'augmenter la résolution des images automatiquement et de régénérer artificiellement des détails dans l'image. Dans leurs dernières versions, les deux frameworks permettent également de fluidifier la fréquence d'images en générant artificiellement des nouvelles images entre celles rendues par les moteurs de jeu. Tandis que le framework de *Nvidia* utilise principalement des algorithmes de

machine learning pour son upscaling et son interpolation d'images, celui d'AMD utilise des algorithmes de filtrage d'images plus classiques.

Ces deux technologies permettent de doubler, voire tripler les performances en jeux. Elles ont permis d'effectuer du lancer de rayon à plus petite résolution et avec une fluidité moindre et d'augmenter cette fluidité et le niveau de détail dans l'image pour obtenir des performances acceptables.

La plupart des jeux supportant le ray tracing en temps réel ne l'utilisent que pour les calculs d'illumination globale et de réflexion dans l'environnement, les méthodes de rendu rastérisées sont pour l'heure beaucoup plus rapides que les méthodes de ray tracing, bien qu'elles soient moins fidèles à la réalité, mais l'avancée constante des technologies supportant le ray tracing laisse à penser que plus d'aspects du rendu de la lumière pourraient être calculés par lancer de rayons, ce qui simplifierait les architectures des moteurs de rendu et offrirait des visuels plus convaincants.

L'occlusion ambiante

Description

L'occlusion ambiante est un effet visuel en infographie qui simule l'assombrissement des zones peu accessibles à la lumière. En effet, dans les surfaces concaves comme les creux ou l'espace entre deux objets, plus difficiles d'accès pour la lumière, la lumière rebondit entre les surfaces et est absorbée en partie.

L'occlusion ambiante crée des ombres douces en prenant en compte les interactions lumineuses entre les surfaces d'une scène 3D. Dans le jeu vidéo, cet effet visuel permet de mieux jauger les volumes des objets et leur distance les uns par rapport à l'autres et d'augmenter le réalisme des matériaux en ajoutant du contraste sur les micro-détails de la surface.

Dans le jeu vidéo, on distingue deux types d'occlusions ambiantes à deux échelles de tailles différentes. La première échelle est celle qui se situe au niveau des micro-détails des matériaux des surfaces (par exemple, un mur de béton contenant des crevasses moins illuminées que le reste de la surface). La seconde échelle est celle des objets (par exemple, un objet creux et l'espace entre deux objets sont des zones qui reçoivent et reflètent moins de lumière que le reste des surfaces).

Les algorithmes qui effectuent les calculs d'occlusion ambiante peuvent être très similaires à ceux utilisés pour le rendu de l'illumination globale, les techniques d'illumination globale permettent d'ailleurs de calculer automatiquement l'occlusion ambiante dans la majorité des cas.

Occlusion Ambiante Statique

Tout comme pour l'illumination globale, le calcul d'occlusion ambiante peut être fait à l'avance pour augmenter les performances en jeu.

Échelle des micros-détails

Il existe plusieurs méthodes pour afficher les zones d'occlusion ambiantes dans les matériaux. Pour cela, on utilise des textures spécifiques de plusieurs types qui ont pour but de stocker ces informations.

Ambient Occlusion Maps

Les textures d'occlusions ambiantes ou AO maps sont les textures les plus utilisées pour les stocker. Il s'agit d'une texture en noir et blanc qui représente l'assombrissement. Pour l'appliquer à un matériau dans sa forme la plus simple, il suffit de multiplier l'albédo du matériau par l'AO map. On calcule généralement cette texture à partir de la texture de reliefs appelée aussi Height Map du matériau qui est une texture en noir et blanc qui représente la hauteur de la surface.

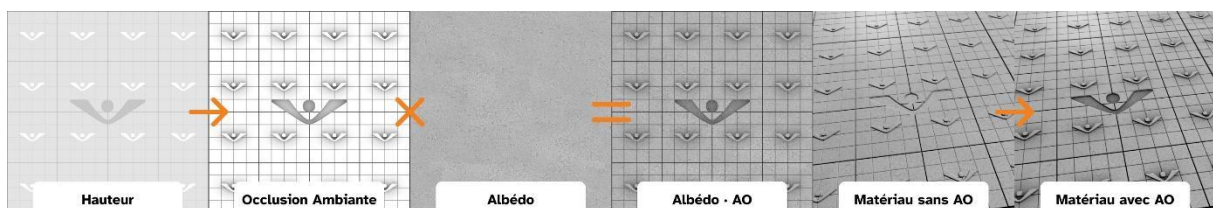


Figure 116 Schéma de la création d'une texture d'occlusion ambiante à partir d'une texture de reliefs qui est ensuite appliquée à l'albédo de la surface et comparaison du matériau sans et avec texture d'occlusion ambiante.

Bump Maps

Les bump maps, ou textures de bosses, sont très similaires aux textures de reliefs car elles représentent la hauteur de la surface en noir et blanc. Leur différence vient du fait qu'elles sont utilisées pour calculer l'ombrage de la surface en fonction des creux qui y sont présents et de l'angle de la lumière. Ces textures permettent également de générer des textures de normales.

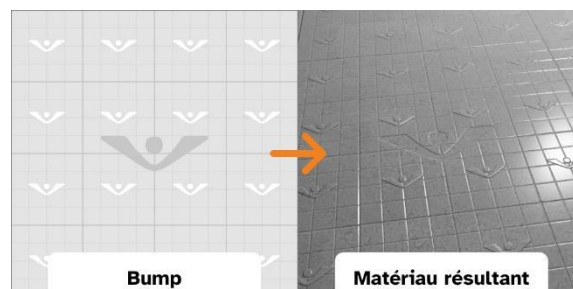


Figure 117 Texture de bosses (bump) et matériau utilisant cette texture qui est éclairé dans la direction de la lumière et sombre dans la direction inverse à la lumière.

Les bump maps ont été le standard pour l'éclairage dans le jeu vidéo mais elles ont été remplacées au cours des années par les textures de normales et les textures d'occlusion ambiante car ces dernières donnent plus d'informations au moteurs 3D pour effectuer les rendus.

Valve Self Shadowing Bump Maps

Les Self Shadowing Bump Maps (SSBump Maps) ou textures de normales à auto-ombrage-directionnel sont un type de texture créé par Valve Software qui combine une texture de normales et une texture d'occlusion ambiante modifiée, prenant en compte l'angle de la source de lumière pour l'affichage de l'ombre. Les SSbump maps ont été présentées en 2007 par Valve dans un papier de recherche, au sein duquel les bump maps sont présentées comme ce que l'on appelle aujourd'hui des normal maps. En effet, les conventions des noms des textures du moteur Source étaient à l'époque différents des standards actuels.

Pour créer ces textures, les informations d'orientation de la surface sont stockées dans le canal rouge (axe X) et le canal vert (axe Y) de la texture comme une normal map classique, mais les SSbump maps remplacent le canal bleu (axe Z) d'orientation par un canal bleu qui représente l'occlusion de la lumière.



Figure 118 Height Map



Figure 119 "Bump Map" en réalité une normal map



Figure 120 normal map stocké dans le canal rouge et le canal vert



Figure 121 SSBump Map qui contient les

Ces textures ont été créées dans le but d'améliorer l'affichage des ombres dans le moteur de jeu Source et ont été utilisées dans le jeu *Half-Life 2 : Episode 2* ainsi que dans *Team Fortress 2*.

Ce type de textures a majoritairement été utilisé dans certains des jeux créés par Valve Software, mais on peut noter qu'il permet de stocker une occlusion ambiante "directionnelle", ce que les textures plus standards ne permettent pas.

Echelle des objets

Pour précalculer l'occlusion ambiante due à la forme des surfaces des objets, on utilise généralement des algorithmes très similaires à ceux utilisés pour le pré-calcul de l'illumination globale, tels qu'un light mapper pour stocker l'occlusion ambiante. Les algorithmes de ces lights mappers sont les mêmes que ceux qui calculent l'illumination globale, par exemple des algorithmes de lancer de rayons classiques.



Figure 122 Modèle 3D ayant une texture d'occlusion ambiante appliquée à son matériau



Figure 123 La texture d'occlusion ambiante qui est appliquée au modèle 3D

Calculs d'occlusion ambiante statique pendant le jeu

Les calculs d'occlusion ambiante statique sont généralement réalisés pendant la création des jeux par les développeurs. Les ombres et lumières ne peuvent donc pas être mises à jour dynamiquement pendant le jeu.

Cette limitation a été problématique pour les jeux de la licence *Halo*. En effet, les jeux *Halo* proposent un mode de jeu appelé *Forge* qui permet aux joueurs et joueuses de modifier les niveaux de jeux en y ajoutant des objets. Le jeu *Halo 3* a souffert graphiquement de cette limitation car les objets ajoutés dans les scènes dans *Forge* avaient un éclairage très rudimentaire par rapport au reste des scènes. Ils ne projetaient pas d'ombres et ne comportaient aucune occlusion ambiante.

Le jeu *Halo 4* a réglé son problème d'illumination non cohérente en ajoutant un système de pré-calcul de la lumière directement dans le jeu. Ce système permettait notamment de calculer l'occlusion ambiante des objets ajoutés au lancement des niveaux, juste avant de commencer à y jouer. Comme

le nombre d'objets ajoutés aux scènes était limité dans *Forge*, le temps de calcul de l'occlusion ambiante ne dépassait pas les 5 secondes et donc ne ralentissait presque pas le lancement des niveaux.



Figure 124 Comparaison des modèles Forge avant et après le calcul d'occlusion ambiante : l'occlusion ambiante permet au modèles Forge de mieux s'intégrer avec le reste de la scène

Occlusion Ambiante Dynamique

Bien que les algorithmes de pré-calcul d'occlusions ambiantes permettent, dans la majorité des cas, d'obtenir des rendus plus fidèles à la réalité, leur nature statique ne leur permet pas d'obtenir des ombres qui réagissent aux changements d'éclairage dans les scènes.

Screen Space Ambient Occlusion

Le l'algorithme Screen Space Ambient Occlusion (SSAO) est une technique d'approximation de l'occlusion ambiante en temps réel qui fonctionne en analysant la profondeur et les normales des pixels dans l'espace écran pour déterminer les zones occluses. L'algorithme SSAO a été développé par *Vladimir Kajalin* en 2007 alors qu'il travaillait chez *Crytek* et a été utilisé pour la première fois dans le jeu vidéo *Crysis*, sorti la même année. Il est présent dans la grande majorité des moteurs 3D aujourd'hui car il est assez rapide à calculer et permet de mieux distinguer les volumes des scènes.

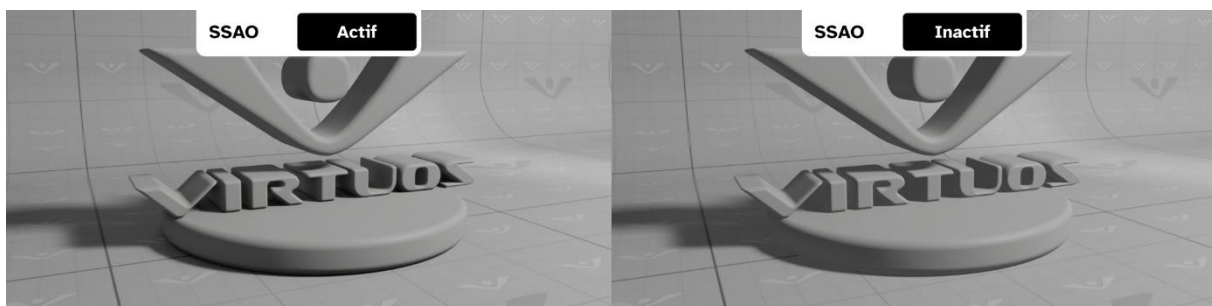


Figure 125 Comparaison d'un modèle 3D affiché avec et sans l'algorithme Screen Space Ambient Occlusion

Il est très similaire aux autres algorithmes en Screen Space comme SSS, SSR et SSIL mentionnés plus haut dans le document. Tout comme les Screen Space Shadows, on récupère les informations de profondeur et de normales de la scène du point de vue de la caméra via les GBuffers. Pour chaque pixel de l'écran, on trace un rayon vers la surface de la scène puis on lance un ou plusieurs rayons dans des directions aléatoires dans un hémisphère orienté dans la direction de la normale de la surface. Enfin, on vérifie si les rayons reflétés entrent en contact avec d'autres surfaces. Si c'est le cas, on assombrit le pixel.

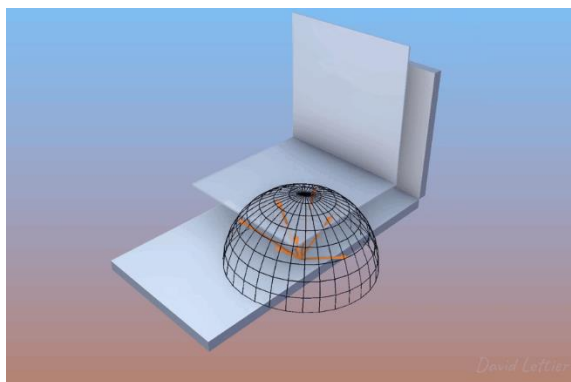


Figure 126 Schéma du fonctionnement de l'algorithme Screen Space Ambient Occlusion

Lorsque l'on utilise plusieurs rayons pour tester la proximité d'autres objets, on parle alors de Multi-sample Screen Space Ambient Occlusion (MSSAO).

On peut améliorer la précision de SSAO en orientant l'hémisphère de calcul d'occlusion en direction des sources de lumières pour savoir si un pixel est caché par un objet proche. Cette technique, nommée Screen Space Directional Occlusion (SSDO), se rapproche d'un effet de Screen Space Shadow.

Une version plus avancée de cet algorithme est appelée Horizon Based Ambient Occlusion (HBAO), elle fonctionne sur le même principe que SSAO mais calcule l'horizon du point de vue de la surface pour calculer son occlusion. Il s'agit de savoir quel est l'angle maximum sans entrer en contact avec un autre objet. Plus l'angle est petit, plus la surface est dans une zone concave donc plus elle est occultée de la lumière. Enfin, un autre algorithme très similaire est appelé High Definition Ambient Occlusion (HDAO), ces deux algorithmes offrent une qualité relativement similaire.

Nvidia Voxel Ambient Occlusion (VXAO)

L'algorithme Voxel Ambient Occlusion (VXAO) est une technique de rendu de l'occlusion ambiante créée par *Nvidia*. Il s'agit d'un sous-ensemble des fonctionnalités offertes par l'algorithme VXGI, qui permet de calculer l'illumination globale d'une scène de manière dynamique. Or, l'occlusion ambiante

fait partie de l'illumination globale car elle simule l'absorption de la lumière due aux rebonds de la lumière. VXAO effectue uniquement la partie du calcul d'illumination globale qui affecte l'occlusion ambiante et est donc 2 à 10 fois plus rapide que VXGI. L'utilisation des voxels pour stocker l'occlusion ambiante lui permet d'avoir moins d'artefacts visuels comparativement au SSAO et autres dérivés. La distribution des ombres de VXGI est également plus réaliste. Cette technique est en moyenne 3 à 4 fois plus lente qu'HBAO+ mais reste très rapide et permet d'afficher une occlusion ambiante plus fidèle à la réalité.

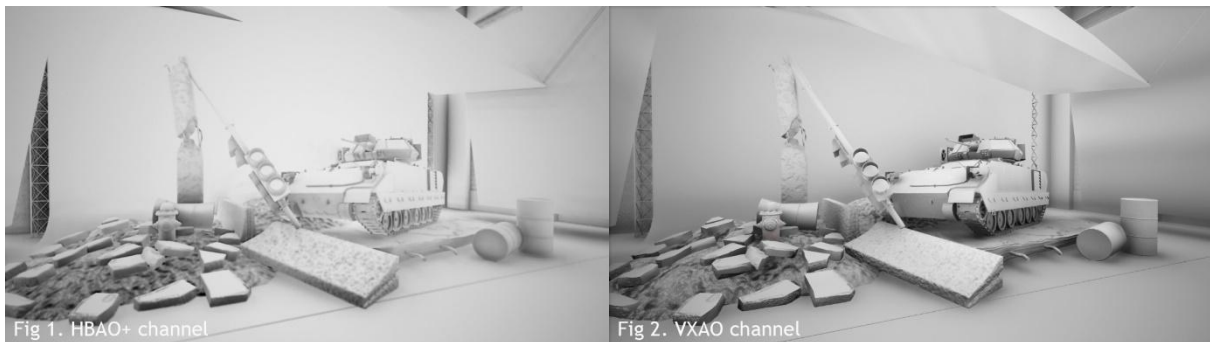


Figure 127 Rendu de l'occlusion ambiante d'une scène en utilisant HBAO

Figure 128 Rendu de l'occlusion ambiante d'une scène en utilisant VXAO

Spéculations sur l'évolution des techniques de rendu en temps réel dans les années à venir dans le jeu vidéo

Ce document a permis d'aborder les différentes techniques d'illuminations et leurs évolutions au cours des années, mais aussi d'observer la complexification et la multiplication des techniques de rastérisation permettant l'affichage de la lumière. Ces techniques ont été créées pour accommoder les architectures des cartes graphiques qui sont optimisées pour les opérations de rastérisation. La rastérisation permet aujourd'hui de représenter une grande partie des phénomènes lumineux mais peine à obtenir des résultats convaincants pour l'éclairage environnemental. En effet, les calculs de reflets, d'illuminations globales et d'occlusions ambiantes nécessitent de prendre en compte les rebonds de la lumière. Les techniques de rastérisation classiques ne permettent pas de calculer ces rebonds de lumière de manière précise, puisque pour obtenir des résultats précis, il faut généralement utiliser des algorithmes simulant le comportement réel de la lumière comme les algorithmes de ray tracing et leurs dérivés.

Depuis quelques années, on peut observer une augmentation de l'utilisation des techniques de ray tracing en temps réel dans le jeu vidéo. On commence à voir l'augmentation des techniques de rastérisation qui sont remplacées par des techniques de lancer de rayon au fur et à mesure de l'avancement du matériel graphique. On peut trouver des similitudes dans cette évolution avec celle de l'utilisation du ray tracing au cinéma, qui a d'abord été utilisé pour augmenter la qualité des images rastérisées, puis les algorithmes ont permis d'uniformiser les techniques d'illumination jusqu'à remplacer totalement les techniques de rastérisation. Il est possible qu'une évolution similaire arrive dans le domaine de l'imagerie numérique en temps réel dans les prochaines décennies. Des moteurs de rendu autrefois seulement basés sur la rastérisation permettent aujourd'hui d'effectuer une illumination hybride, peut être que l'amélioration des performances de ray tracing poussera les constructeurs de moteurs de rendus à consolider les algorithmes de ray tracing pour remplacer complètement la rastérisation.

Ces dernières années ont également permis d'observer la démocratisation des algorithmes de génération et de filtrage d'images, par exemple DLSS de *Nvidia*, FSR d'*AMD* et Optix d'*Intel*. Beaucoup de ces algorithmes utilisent des IAs génératives pour effectuer les traitements de l'image en temps réel. Des projets d'IAs comme le logiciel *Magnific* permettent aujourd'hui de modifier les détails des images de synthèse pour leur donner un côté plus organique et augmenter leur réalisme. On peut

imaginer des futurs algorithmes permettant de convertir des rendus de jeux assez basiques et de les filtrer en images photoréalistes en temps réel.

De nouveaux algorithmes de rendu qui se basent sur une multitude de photographie pour les volumes des scènes ont également vu le jour. On peut par exemple penser aux algorithmes Neural Radiance Field (NeRF) et Gaussian Splatting qui permettent de rendre des scènes statiques photoréalistes en temps réel. Ces algorithmes sont principalement utilisés pour afficher en 3D des objets du monde réel qui ont été scannés. Il est en théorie possible de remplacer les images du monde réel par des images de synthèse pré-rendues pour pouvoir afficher des scènes virtuelles photoréalistes en temps réel.

Ces nouvelles techniques de rendu sont aujourd'hui expérimentales ou encore trop coûteuses pour être utilisées en temps réel. Il est probable qu'elles continuent de s'améliorer au cours des prochaines années mais leur utilisation dans le domaine du jeu vidéo ne repose, à l'heure actuelle, que sur des spéculations. Les méthodes actuelles de rendu 3D vont également continuer à s'améliorer dans le temps et de nouvelles innovations continueront à apparaître, comme en témoigne la rapidité des évolutions des techniques en matière de lumière dans le milieu du jeu vidéo.

Conclusion

Ce document nous a permis d'aborder un grand nombre d'algorithmes et de techniques de rendu 3D, ayant pour objectif est d'obtenir des images les plus réalistes possibles. Ces techniques sont nombreuses et un grand nombre d'autres algorithmes existent pour augmenter la qualité des graphismes. On peut remarquer que les fondamentaux du rendu de la lumière, qui sont L'ASPECT DES MATERIAUX et LES OMBRES PORTEES, utilisent des techniques communes dans tous les moteurs. Les techniques d'ILLUMINATION ENVIRONNEMENTALE comme les REFLETS, l'ILLUMINATION GLOBALE et l'OCCCLUSION AMBIANTE sont, quant à elles, implémentées de manière plus spécifique par chaque moteur de jeu et chaque constructeur de puces graphiques.

Il existe deux autres aspects de la lumière qui n'ont pas été abordés dans ce document car leurs domaine d'utilisation est très spécifique. Ces aspects sont le **rendu volumétrique de la lumière** et les aspects de la lumière sur les **capteurs**. Le **rendu volumétrique de la lumière** comprend le rendu du brouillard, des nuages et de l'absorption de la lumière dans l'eau. Le rendu des aspects de la lumière sur les **capteurs** comprend la profondeur de champ, les reflets optiques, les aberrations chromatiques ou le bloom. Ces deux aspect du rendu de la lumière entrent en jeu dans des situations spécifiques contrairement aux aspects que nous avons abordés dans le document et il est encore rare aujourd'hui qu'ils soient simulés de manière réaliste dans le jeu vidéo.

Ensemble, ces cinq aspects de la lumière permettent d'avoir une vision globale portant sur les techniques de rendu des aspects de la lumière dans l'imagerie numérique.

Il est important de noter que la qualité des graphismes d'un jeu n'est pas la partie la plus importante de ses visuels. Le jeu vidéo est un art, tout comme la qualité d'une peinture ne se mesure pas par son nombre de coups de pinceaux, l'esthétique et la direction artistique d'un jeu est bien plus importante que la résolution de ses ombres ou le réalisme de ces matériaux. L'exemple le plus parlant est le jeu le plus vendu au monde : *Minecraft*. Le jeu utilise des graphismes très basiques mais son esthétique rétro lui permet d'avoir un style visuel iconique. Il est généralement admis que les jeux ayant une esthétique poussée ont des visuels qui vieillissent bien mieux que des jeux se voulant réalistes et même s'ils étaient à la pointe de l'innovation graphique à leur époque.

Il est également important de noter que le jeu vidéo est un art interactif. Par définition, la façon dont il est joué et le plaisir qu'il procure sont les éléments majeurs de sa réussite critique et commerciale. On peut prendre l'exemple de *Counter Strike 2 (CS 2)*, le jeu le plus joué sur *Steam*. Il est la suite du jeu *Counter Strike Global Offensive (CSGO)* et est créé dans le nouveau moteur *Source 2* de *Valve Software*.

Le nouveau jeu supporte des graphismes plus détaillés que son prédécesseur mais utilise une esthétique plus stylisée. Cet ajout de détail peut parfois entraîner des problèmes de visibilité pendant les matchs. Pour cette raison, certains niveaux ont vu leurs détails être retirés, par souci de jouabilité. Il est ainsi souvent préférable de créer un jeu aux graphismes moins impressionnants si cela permet au jeu final d'être plus fluide, plus lisible.

Enfin, la recherche des graphismes toujours plus réalistes est souvent un argument de vente plus qu'une vision artistique. Les constructeurs de consoles et de cartes graphiques repoussent les limites matérielles pour donner envie au public d'acheter leurs matériels. Cela pousse les développeurs à créer des jeux aux graphismes impressionnants pour être mis en avant par les constructeurs de matériel de jeu, et donc d'augmenter leur chiffre d'affaires. Il est vrai que les graphismes impressionnants sont un facteur de vente non négligeable mais augmenter la complexité du rendu 3D peut rendre des jeux injouables sur du matériel ancien et limiter le nombre de ventes potentielles. On peut voir ce paradoxe si l'on observe les jeux les plus vendus et joués dans le monde. Dans le top 10 des jeux les plus vendus au monde, toutes plateformes confondues, seuls trois d'entre eux ont des graphismes "réalistes" : *Grand Theft Auto 5*, *PUBG : Battleground* et *Red Dead Redemption 2*. *Minecraft*, *Tetris*, *Wii Sports*, *Mario Kart 8 – Deluxe*, *Super Mario Bros*, *Human Fall Flat* et *Overwatch* ont quant à eux eu des graphismes très basiques. Parmi les dix jeux les plus joués sur PC en avril 2024, huit ont des graphismes simples et une direction artistique stylisée : *Fortnite*, *Roblox*, *Minecraft*, *Counter Strike*, *League of Legends*, les *Sims 4*, *Valorant* et *Rocket League*.

On peut alors se demander quel est l'intérêt de toujours continuer à innover dans la recherche pour l'imagerie numérique en temps réel. Il s'avère que tous les jeux populaires qui ont été cités doivent en partie leur existence à l'amélioration des technologies de rendu 3D. En effet, la démocratisation des algorithmes complexes de rendu 3D a permis de faciliter le développement de jeux vidéo pour toute l'industrie, qu'il s'agisse des grands éditeurs ou des développeurs indépendants. De plus, les algorithmes qui ont été abordés dans ce document sont tous utilisés par un ou plusieurs des jeux cités plus tôt et sont utilisés dans des centaines de jeux pour compléter leur esthétique. Ainsi, dans le milieu du jeu vidéo, le choix des graphismes est donc une question de compromis entre la vision des créateurs des jeux, les limitations du matériel et l'attente des joueurs et joueuses.

Liste des figures

Figure 1 Comparaison d'une scène de couloir avec un éclairage accueillant et avec un éclairage angoissant.....	17
Figure 2 La scène du couloir avec une lumière rouge signifiant un danger imminent	17
Figure 3 Image du jeu Wolfenstein 3D sorti en 1992	19
Figure 4 Image du jeu Doom sorti en 1993	19
Figure 5 Comparaison d'une scène rendue en utilisant un algorithme de rendu photoréaliste (lancer de rayons) avec sa géométrie triangulaire réelle.....	20
Figure 6 Schéma de l'algorithme de rastérisation et exemple de rendu rastérisé. On voit dans le schéma que la géométrie de l'objet est projetée en 2D sur l'écran.	20
Figure 7 Schéma de l'algorithme lancé de rayons et exemple de rendu en lancé de rayons, on voit dans le schéma que l'on projette des rayons dans la scène depuis la caméra	21
Figure 8 Comparaison du rendu d'un modèle 3D métallique en utilisant la rastérisation et le lancer de rayons	21
Figure 9 Équation de la réflexion diffuse de Lambert	22
Figure 10 Torus polygonal utilisant un éclairage de Lambert	23
Figure 11 Affiche du Jeu I Robot.....	23
Figure 12 Image Issue du jeu I Robot	23
Figure 13 Torus polygonal utilisant un éclairage de Gouraud.....	24
Figure 14 Affiche du jeu Mario 64	24
Figure 15 Image issue du jeu Mario 64	24
Figure 16 Formule de la réflexion spéculaire de Phong	25
Figure 17 Formule du vecteur direction de la lumière reflétée sur une surface de normale N	25
Figure 18 Torus polygonal utilisant un éclairage de Phong	26
Figure 19 Image issue du jeu Titanfall (2014)	27
Figure 20 Image issue du jeu Doom (2016).....	27
Figure 21 Formule de la réflexion spéculaire de Blinn-Phong.....	27
Figure 22 Formule du vecteur milieu M entre la direction vers la caméra C et la direction vers la lumière L	27
Figure 23 Illustration des problèmes de réflexion spéculaire du modèle de Phong comparé au modèle de Blinn-Phong	28
Figure 24 Comparaison des reflets spéculaires du modèle de Phong et du modèle de Blinn-Phong ..	28

Figure 25 Première de couverture de la quatrième édition du livre "Physically Based Rendering"	30
Figure 26 Schéma des différentes Fonctions de Distributions Bidirectionnelles par Electronic Arts ...	30
Figure 27 Historique des BRDF basées sur la physique par Josef Pelikán	32
Figure 28 Historique des BRDF empiriques et expérimentales par Josef Pelikán	32
Figure 29 Simulation de la rugosité par algorithme de micro-facettes	33
Figure 30 Fonction BRDF de Cook-Torrance.....	33
Figure 31 Fonction de réflexion spéculaire de Cook-Torrance	33
Figure 32 Distribution normale des reflets spéculaires en fonction du paramètre de rugosité.....	34
Figure 33 Assombrissement d'une surface en fonction de sa rugosité	34
Figure 34 Distribution de la fonction de Fresnel en fonction de l'Indice de Réfraction et impact sur la réflexion spéculaire d'un matériau	34
Figure 35 Comparaison de matériaux ayant chacun un indice de réfraction différent, la valeur de leur indice de réfraction est affichée.....	35
Figure 36 Equation du rendu de la translucidité d'une surface	36
Figure 37 Comparaison du réalisme de la peau en fonction de la prise en compte de sa translucidité, le modèle éclairé sans translucidité a une apparence de plastique.	37
Figure 38 Différents aspects visuels d'un matériau en fonction de la couleur de son albédo	38
Figure 39 Différents aspects visuels d'un matériau en fonction de son aspect métallique.....	39
Figure 40 Différents aspects visuels d'un matériau en fonction de sa rugosité	39
Figure 41 Différents aspects visuels d'un matériau en fonction de l'intensité de sa texture de normales	39
Figure 42 Comparaison de l'apparence des textures de normales au format OpenGL et DirectX en fonction de la hauteur de la surface, le canal vert de la texture de normale est inversé entre le format OpenGL et le format DirectX	40
Figure 43 Différents aspects visuels d'un matériau en fonction de sa translucidité	40
Figure 44 Différents aspects visuels d'un matériau en fonction de sa transparence	40
Figure 45 Différents aspects visuels d'un matériau en fonction de son indice de réfraction.....	41
Figure 46 Comparaison d'un matériau ayant ou non un éclat de tissu.....	41
Figure 47 Comparaison du rendu d'un matériau utilisant ou non du clear-coat.....	41
Figure 48 Comparaison du rendu d'un matériau anisotropique ou non	41
Figure 49 Comparaison de l'aspect d'un modèle 3D en fonction d'une absence d'ombres portées, d'une ombre obtenue par rasterisation et d'une ombre obtenue par lancer de rayons.	43
Figure 50 Schéma de la distribution de l'ombre, la pénombre et la zone éclairée d'un objet.....	43

Figure 51 Comparaison des ombres projetées sur une surface en fonction de la taille d'une source lumineuse.....	44
Figure 52 Éclairage d'un point lumineux.....	45
Figure 53 Éclairage d'un projecteur	46
Figure 54 Éclairage d'une lumière directionnelle.....	46
Figure 55 Éclairage d'une lumière de zone	46
Figure 56 Éclairage d'une lumière environnementale	47
Figure 57 Éclairage d'une lumière globale	47
Figure 58 Éclairage d'une surface lumineuse.....	47
Figure 59 Création de l'illumination d'une scène du jeu "Kingdom Hearts" grâce à la couleur des sommets des polygones.	48
Figure 60 Schéma de la détection de l'illumination par Ray Casting dans le moteur de Quake, les rayons bloqués par la géométrie sont considérés à l'ombre.....	49
Figure 61 Comparaison de l'ombre calculée sans et avec Anti-Aliasing.	50
Figure 62 Ombre circulaire projetée sous Mario dans "Mario 64"	51
Figure 63 Comparaison d'une shadow map sans filtrage, avec filtrage PCF et avec filtrage PCF aléatoire	52
Figure 64 Comparaison des cascaded shadow maps et des virtual shadow maps dans Fortnite.	53
Figure 65 Unreal Tournament 2004 utilisant un ombrage classique	54
Figure 66 Doom 3 (2004) utilisant des shadow volumes pour son ombrage.....	54
Figure 67 Schéma de l'algorithme de calcul des screen space shadows	56
Figure 68 Comparaison des détails d'une ombre issue de shadow map avec et sans ajout de screen space shadows.....	56
Figure 69 Implémentation des screen space shadows dans les jeux "Quantum Break" et "Control" du studio Remedy Entertainment	57
Figure 70 Image comparative des matériaux utilisant du parallax occlusion mapping avec et sans contact shadows.....	57
Figure 71 Image d'une scène rendue avec ReSTIR, chaque élément lumineux de la scène est comptabilisé dans les calculs d'illumination directe.	58
Figure 72 Comparatif par Nvidia de la qualité de rendu d'image de différents algorithmes de lancer de rayons avec et sans débruitage par l'Optix Denoiser.....	59
Figure 73 Réflexions de d'un environnement sur un des modèles 3D avec cube map qui correspond à l'environnement reflété	61

Figure 74 Environnement enregistré dans une cube map	62
Figure 75 Le même environnement enregistré dans une image panoramique équirectangulaire	62
Figure 76 Comparaison des réflexions probes/captures dans le moteur Unity, le moteur Unreal Engine et le moteur Godot.....	62
Figure 77 Comparaison d'une réflexion utilisant une cube map classique et une cube map ayant sa perspective corrigée dans le moteur Source, la réflexion est bien plus convaincante quand sa perspective est correcte.....	63
Figure 78 Utilisation des G buffers de profondeur, de normales et de rendu classique de la scène pour générer un G buffer de réflexion suivi de l'ajout de ce buffer au rendu de base pour obtenir une image finale.....	64
Figure 79 Schéma de l'algorithme de calcul des Screen Space Reflection.....	64
Figure 80 Image du film Compleat Angler par Turner Whitted en 1979 ayant un rendu prenant un temps considérable pour chaque image	65
Figure 81 Image de la cinématique Reflections en 2020 ayant un rendu en 20 ms en utilisant une Nvidia RTX 2080Ti.....	65
Figure 82 Image de la réflexion sur l'eau en utilisant des Screen Space Reflections dans le jeu Cyberpunk 2077.....	66
Figure 83 Image de la réflexion sur l'eau en utilisant du Ray Tracing dans le jeu Cyberpunk 2077	66
Figure 84 Image d'un miroir capturée dans le jeu Duke Nukem 3D	67
Figure 85 Schéma du ray casting 2D dans la géométrie de la scène de l'image précédente, le moteur lance un rayon par colonne de pixel de l'écran pour afficher la scène.....	67
Figure 86 Faux reflet de miroir dans la salle au miroir dans le jeu Mario 64.....	67
Figure 87 Salle en réalité dupliquée avec une surface semi transparente	67
Figure 88 Faux reflets du plafond sur le bois laqué de la piste de bowling dans Wii Sport.....	68
Figure 89 Salle en réalité symétrique sur sa hauteur et ayant un sol semi transparent pour faire apparaître les reflets.	68
Figure 90 Fausse réflexion sur la surface de l'eau d'un lac dans Mario Galaxy	68
Figure 91 La réflexion du château est en fait un modèle différent qui est distordu par la surface de l'eau	68
Figure 92 Réflexion incorrecte dans un miroir du jeu Mafia 3 due à l'utilisation de cube maps semi dynamiques	69
Figure 93 Comparaison des réflexions planaires en utilisant l'algorithme Screen Space Reflection et des Planar Reflection Probes	70

Figure 94 Équation du rendu de la lumière.....	70
Figure 95 Décomposition du terme de l'intensité lumineuse en fonction de l'intensité lumineuse directe et indirecte	71
Figure 96 Comparaison du rendu d'une image en fonction du nombre de rebonds accordés à chaque rayon de lumière	71
Figure 97 Comparaison du grain de l'image en fonction du nombre de chemins parcourus par pixel	73
Figure 98 Représentation des premiers niveaux des fonctions harmoniques sphériques, les portions cyan représentent des valeurs positives et les portions orange représentent des valeurs négatives .	75
Figure 99 Light probes groups affichés dans Unity, les points jaunes représentent les probes et les lignes roses représentent les connexions entre les probes	76
Figure 100 Objet dynamique étant éclairé par les probes qui sont reliées par des lignes jaunes, la proportion de l'impact de chaque probe est calculée en fonction de sa distance avec l'objet	76
Figure 101 Volumetric light map à la densité variable et schémas en 2D de la grille dont la densité change en fonction de la géométrie de la scène.....	77
Figure 102 Illustration du changement des rebonds de la lumière en fonction de l'orientation du soleil	79
Figure 103 Comparaison de la taille de la grille de voxels de calculs de lumière en fonction du niveau de détail sélectionné	80
Figure 104 Schéma du calcul des rebonds de lumière à l'aide de l'algorithme de Cone Tracing	80
Figure 105 Comparaison d'une scène dans Godot sans illumination globale, avec Voxel GI et visualisation des voxels	81
Figure 106 Visualisation d'un Signed Distance Field d'une capsule en 2D, les paliers orange représentent une distance positive de 5 cm tandis que les paliers cyan représentent une distance négative de 5 cm, les distances affichées correspondent à la distance des points par rapport à la surface de la capsule	82
Figure 107 Affichages d'une scène dans Godot en visualisation classique, en visualisation de la scène discrétisée par SDFGI et en visualisation de la grille 3D de Light Probes.....	82
Figure 108 Comparaison du rendu de l'illumination globale dans une scène en les algorithmes SDFGI et HDDAGI	84
Figure 109 Visualisation des Surfels projetés dans une scène pour calculer en calculer l'illumination globale	84
Figure 110 Schéma du fonctionnement de l'algorithme GIBS du moteur Frostbite d'Electronic Arts .	85

Figure 111 Illumination globale en temps réel dans une scène utilisant l'algorithme Lumen d'Unreal Engine 5	86
Figure 112 Schéma de l'architecture logicielle de l'algorithme Lumen	87
Figure 113 Comparaison de la qualité de rendu d'un algorithme de lancer de rayons classique et de lumen pour un nombre restreint de lancer de rayons par pixels	88
Figure 114 Comparaison de la qualité de l'illumination de Lumen avec un budget de 1/2 lancer de rayons par pixel et un budget de 1/8 lancer de rayons par pixel.....	88
Figure 115 Comparaison d'un rendu en Ray Tracing en fonction du nombre de rayons par pixel et de l'activation ou non d'un algorithme de Denoising (ici Optix Denoiser). L'augmentation du nombre de rayons par pixel (samples) augmente la qualité de l'image de base et permet au Denoiser d'approcher l'image de référence avec moins d'artefacts visuels.	90
Figure 116 Schéma de la création d'une texture d'occlusion ambiante à partir d'une texture de reliefs qui est ensuite appliquée à l'albédo de la surface et comparaison du matériau sans et avec texture d'occlusion ambiante.	92
Figure 117 Texture de bosses (bump) et matériau utilisant cette texture qui est éclairé dans la direction de la lumière et sombre dans la direction inverse à la lumière.	92
Figure 118 Height Map	93
Figure 119 "Bump Map" en réalité une normal map	93
Figure 120 normal map stocké dans le canal rouge et le canal vert	93
Figure 121 SSBump Map qui contient les.....	93
Figure 122 Modèle 3D ayant une texture d'occlusion ambiante appliquée à son matériau	94
Figure 123 La texture d'occlusion ambiante qui est appliquée au modèle 3D	94
Figure 124 Comparaison des modèles Forge avant et après le calcul d'occlusion ambiante : l'occlusion ambiante permet au modèles Forge de mieux s'intégrer avec le reste de la scène.....	95
Figure 125 Comparaison d'un modèle 3D affiché avec et sans l'algorithme Screen Space Ambient Occlusion	95
Figure 126 Schéma du fonctionnement de l'algorithme Screen Space Ambient Occlusion	96
Figure 127 Rendu de l'occlusion ambiante d'une scène en utilisant HBAO	97
Figure 128 Rendu de l'occlusion ambiante d'une scène en utilisant VXAO.....	97

Annexe

Liste de sources

Sources d'information générales:

<https://gdcvault.com/>

<https://siggraph.org>

<https://lousodrome.net/blog/light/>

<https://lettier.github.io/3d-game-shaders-for-beginners/index.html>

<https://google.github.io/filament/Filament.html>

https://www.youtube.com/@cem_yuksel

Aspects des matériaux :

Modèles d'ombrages :

https://en.wikipedia.org/wiki/Lambertian_reflectance

<https://collections.lib.utah.edu/details?id=706615>

<https://lettier.github.io/3d-game-shaders-for-beginners/lighting.html>

<https://lettier.github.io/3d-game-shaders-for-beginners/blinn-phong.html>

<https://learnopengl.com/Lighting/Basic-Lighting>

<https://learnopengl.com/Advanced-Lighting/Advanced-Lighting>

Indice de Réfraction :

<https://pixelandpoly.com/ior.html>

https://pbr-book.org/4ed/Reflection_Models/Dielectric_BSDF

SubSurface Scattering :

<https://gdcvault.com/play/1014536/Approximating-Translucency-for-a-Fast>

<https://dev.epicgames.com/documentation/en-us/unreal-engine/using-subsurface-scattering-in-unreal-engine-materials>

<https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-human-skin-in-unreal-engine>

Physically Based Rendering :

<https://www.pbrt.org/>

<https://www.pbr-book.org/>

https://blog.selfshadow.com/publications/s2013-shading-course/karis/s2013_pbs_epic_notes_v2.pdf

<https://www.trentreed.net/blog/physically-based-shading-and-image-based-lighting/>

https://seblagarde.files.wordpress.com/2015/07/course_notes_moving_frostbite_to_pbr_v32.pdf

<https://cdn2.unrealengine.com/Resources/files/2013SiggraphPresentationsNotes-26915738.pdf>

https://media.disneyanimation.com/uploads/production/publication_asset/48/asset/s2012_pbs_disney_brdf_notes_v3.pdf

<https://sayan1an.github.io/pdfs/references/disneyBrdf.pdf>

<https://cgg.mff.cuni.cz/~pepca/lectures/pdf/pg2-05-brdf.en.pdf>

Self Shadowing Bump Maps :

https://cdn.cloudflare.com/apps/valve/2007/SIGGRAPH2007_EfficientSelfShadowedRadiosityNormalMapping.pdf

Rastérisation et Ray Tracing :

<https://gdcvault.com/play/1016435/Forward-Rendering-Pipeline-for-Modern>

<https://www.pixelsham.com/2019/10/24/whats-the-difference-between-ray-tracing-and-rasterization/>

Ombres portées :

Les types de sources de lumières :

<https://docs.unrealengine.com/4.27/en-US/BuildingWorlds/LightingAndShadows/LightTypes/>

<https://docs.unity3d.com/Manual/Lighting.html>

https://docs.godotengine.org/en/stable/tutorials/3d/lights_and_shadows.html

https://docs.blender.org/manual/en/latest/render/lights/light_object.html

<https://ieslibrary.com/>

Ombres portées statiques :

Light Mapping :

<https://gdcvault.com/play/1017902/Static-Lighting-Tricks-in-Halo>

https://www.youtube.com/watch?v=TFfkX_ahl94

https://developer.valvesoftware.com/wiki/Half-Life_SDK

https://dev.epicgames.com/documentation/en-us/unreal-engine/understanding-lightmapping-in-unreal-engine?application_version=5.0

<https://www.creativeshrimp.com/game-level-texturing-bake-lightmap-in-blender-and-unity-part-55.html>

Quake 1 et Quake 2 :

<https://www.youtube.com/watch?v=bxientPsgRg&list=LL&index=2>

Ombres portées dynamiques :

Dynamic Shadow Maps :

<https://dev.epicgames.com/documentation/en-us/unreal-engine/shadowing-in-unreal-engine>

<https://www.opengl-tutorial.org/intermediate-tutorials/tutorial-16-shadow-mapping/>

<https://docs.unity3d.com/Manual/shadow-cascades.html>

Illustrations Percentage Closer Filtering :

<https://www.youtube.com/watch?v=uueB2kVvbHo>

Virtual Shadow Maps :

https://dev.epicgames.com/documentation/en-us/unreal-engine/virtual-shadow-maps-in-unreal-engine?application_version=5.0

Shadow Volumes :

https://fabiansanglard.net/doom3_documentation/0301002.pdf

<https://developer.nvidia.com/gpugems/gpugems/part-ii-lighting-and-shadows/chapter-9-efficient-shadow-volume-rendering>

https://developer.download.nvidia.com/assets/gamedev/docs/StencilShadows_CEDEC_E.pdf

Reservoir-based Spatio-Temporal Importance Resampling :

<https://developer.nvidia.com/blog/rendering-millions-of-dynamics-lights-in-realtime/>

https://research.nvidia.com/sites/default/files/pubs/2020-07_Spatiotemporal-reservoir-resampling/ReSTIR.pdf

Screen Space Shadows :

https://panoskarabelas.com/posts/screen_space_shadows/

<https://dev.epicgames.com/documentation/en-us/unreal-engine/contact-shadows-in-unreal-engine>

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@13.1/manual/renderer-feature-screen-space-shadows.html>

G-Buffers :

<https://unrealartoptimization.github.io/book/profiling/passes/>

<https://learnopengl.com/Advanced-Lighting/Deferred-Shading>

Illumination environnementale :

Réflexions :

Réflexions Standards :

Cube Maps :

https://dev.epicgames.com/documentation/en-us/unreal-engine/reflections-captures-in-unreal-engine?application_version=5.0

<https://docs.unity3d.com/Manual/class-ReflectionProbe.html>

https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/reflection_probes.html

Cube Map utilisée dans le document :

https://polyhaven.com/a/brown_photostudio_02

Conversion des images panoramique équirectangulaire en Cube Maps classiques : <https://jaxry.github.io/panorama-to-cubemap/>

Screen Space Reflections :

<https://lettier.github.io/3d-game-shaders-for-beginners/screen-space-reflection.html>

https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/reflection_probes.html

https://docs.blender.org/manual/en/latest/render/eevee/render_settings/screen_space_reflections.html

Ray Tracing :

<https://blogs.nvidia.com/blog/ray-tracing-global-illumination-turner-whitted/>

<https://www.youtube.com/watch?v=IMSuGoYcT3s>

<https://www.youtube.com/watch?v=eqXeM4712ps>

<https://developer.nvidia.com/rtx/ray-tracing/>

Réflexions de Miroirs :

Ray Casting 2.5D :

<https://www.youtube.com/watch?v=Shc18-7OxH0&t=72s>

Duplication de modèles 3D :

Captures des jeux : <https://noclip.website>

Cube Maps :

Exemple du jeu *Mafia 3* :

<https://www.youtube.com/watch?v=0GRteWe10xl>

Plannar Reflections Probes :

<https://docs.unrealengine.com/4.27/en-US/BuildingWorlds/LightingAndShadows/PlanarReflections/>

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@12.0/manual/Planar-Reflection-Probe.html>

https://docs.blender.org/manual/en/latest/render/eevee/light_probes/reflection_planes.html

Illumination Globale :

<https://www.youtube.com/watch?v=zIM4bMwqtCk&t=1s>

https://en.wikipedia.org/wiki/Rendering_equation

Illumination Globale Statique :

<https://gdcvault.com/play/1015522/Light-Probe-Interpolation-Using-Tetrahedral>

<https://cseweb.ucsd.edu/~ravir/papers/envmap/envmap.pdf>

<https://docs.unity3d.com/Manual/class-LightProbeGroup.html>

https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/using_light_map_gi.html

<https://dev.epicgames.com/documentation/en-us/unreal-engine/volumetric-lightmaps-in-unreal-engine>

<https://blog.unity.com/engine-platform/global-illumination-updates-in-2022-2>

Illumination Globale Dynamique :

Unreal Engine :

<https://gdcvault.com/play/1016472/Epic-Games-Elemental-Unreal-Engine>

https://dev.epicgames.com/documentation/en-us/unreal-engine/lumen-technical-details-in-unreal-engine?application_version=5.0

<https://www.youtube.com/watch?v=2GYXuM10riw>

<https://www.youtube.com/watch?v=qC5KtatMcUw>

Unity Engine :

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@10.2/manual/Override-Screen-Space-GI.html>

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@10.2/manual/Override-Screen-Space-GI.html>

<https://docs.unity3d.com/Manual/realtime-gi-using-enlighten.html>

Godot Engine :

https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/using_voxel_gi.html

https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/using_sdfgi.html

https://docs.godotengine.org/en/stable/tutorials/3d/environment_and_post_processing.html#screen-space-indirect-lighting-ssil

<https://github.com/godotengine/godot/pull/86267>

https://research.nvidia.com/publication/2021-06_restir-gi-path-resampling-real-time-path-tracing

https://research.dreamworks.com/wp-content/uploads/2018/07/talk_hierarchical_digital_OpenVDB_v4-Edited.pdf

Frostbite Engine :

<https://jcgt.org/published/0005/04/02/paper-lowres.pdf>

Nvidia :

<https://on-demand.gputechconf.com/gtc/2014/presentations/S4552-rt-voxel-based-global-illumination-gpus.pdf>

https://developer.download.nvidia.com/assets/events/GDC15/GEFORCE/VXGI_Dynamic_Global_Illumination_GDC15.pdf

<https://d1qx31qr3h6wln.cloudfront.net/publications/ReSTIR%20GI.pdf>

https://research.nvidia.com/publication/2021-06_restir-gi-path-resampling-real-time-path-tracing

<https://developer.nvidia.com/rtx/ray-tracing/optix>

<https://developer.nvidia.com/optix-denoiser>

AMD :

<https://gpuopen.com/amd-fidelityfx-sdk/>

<https://gpuopen.com/fidelityfx-denoiser/>

Intel :

<https://www.openimagedenoise.org/>

Deferred Radiance Transfert Volumes dans Far Cry 3:

<https://gdcvault.com/play/1015539/Deferred-Radiance-Transfer-Volumes-Global>

Instant Radiosity :

https://agraphicsguynotes.com/posts/instant_radiosity_in_my_renderer/

Metropolis Light Transport :

<https://www.youtube.com/watch?v=sgmWkbk4T3g&list=PL1pqDbFhTzsgVpRjcJaNOSItMLpaalAFd&index=14>

Occlusion Ambiante :

Occlusion Ambiante Statique :

Échelle des matériaux :

<https://dev.epicgames.com/community/learning/tutorials/bOEy/material-ambient-occlusion-ao-in-unreal-engine-5>

<https://helpx.adobe.com/substance-3d-designer/substance-compositing-graphs/nodes-reference-for-substance-compositing-graphs/node-library/filters/effects/ambient-occlusion-hbao-filter-node.html>

<https://help.poliigon.com/en/articles/1712652-texture-maps-explained>

Échelle des modèles 3D :

<https://gdcvault.com/play/1017710/Rendering-Assassin-s-Creed>

<https://viewar.gitbooks.io/sdk-documentation/content/3d-content/3d-content--custom-models/guidelines/guidelines-AO/3d-content--custom-models--guidelines--AO-maya-xnormal.html>

<https://gdcvault.com/play/1017902/Static-Lighting-Tricks-in-Halo>

Occlusion Ambiante Hybride :

<https://gdcvault.com/play/1017766/Relighting-Forge-in-Halo-4>

Occlusion Ambiante Dynamique :

https://www.researchgate.net/publication/228576448_Screen_Space_Ambient_Occlusion

<https://gdcvault.com/play/1014914/Lighting-in-Crysis>

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@10.0/manual/post-processing-ssao.html>

https://developer.download.nvidia.com/presentations/2008/SIGGRAPH/HBAO_SIG08b.pdf

<https://gaming.stackexchange.com/questions/95161/ssao-vs-hbao-vs-hdao-whats-the-real-difference>

<https://www.gdcvault.com/play/1023520/Advanced-Ambient-Occlusion-Methods-for>

<https://lettier.github.io/3d-game-shaders-for-beginners/ssao.html>

Algorithmes de *Machine Learning* :

<https://isl-org.github.io/PhotorealismEnhancement/>

<https://www.matthewtancik.com/nerf>

<https://bertrand-benoit.com/blog/ai-finishing/?lid=lx2x38ulsiux>

Bases de données des jeux vidéo les plus populaires :

<https://steamdb.info/charts/>

<https://fortnite.gg/player-count>

<https://prioridata.com/data/league-of-legends/>

https://fr.wikipedia.org/wiki/Liste_des_jeux_vid%C3%A9o_les_plus_vendus